Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mmet functions

Synopsis

The functions prefixed with Mmet make up the Metrology module. The Metrology module measures features and validates geometric tolerances in a target image. Features and tolerances are first added to a virtual template. Added features can be physically measured from an image or geometrically constructed from other features. Once features and tolerances have been added to the template, the template itself can be positioned at a new location, depending on where features have been found in the target image. Positioning in metrology can even be done automatically using the results of other Aurora Imaging Library modules, such as Model Finder. Measurements are made and results are returned with a high degree of subpixel accuracy.

Functions

MmetAddFeature
MmetAddTolerance
MmetAlloc
MmetAllocResult
MmetCalculate
MmetControl
MmetDraw
MmetFree
MmetGetResult
MmetInquire
MmetName
MmetPut
MmetRestore
MmetSave
MmetSetPosition
MmetSetRegion
MmetStream

MmetAddFeature

Synopsis

Add a feature to, or modify a feature of, the metrology template of a metrology context.

Syntax

```
void MmetAddFeature(
    AIL_ID ContextId,                       //out
    AIL_INT64 FeatureType,                  //in
    AIL_INT64 Geometry,                     //in
    AIL_INT FeatureLabel,                   //in
    AIL_INT64 BuildOperation,               //in
    const AIL_INT *FeatureLabelArrayPtr,    //in
    const AIL_INT *SubFeatureIndexArrayPtr, //in
    AIL_INT SizeOfArray,                    //in
    AIL_INT64 ControlFlag                   //in
)
```

Description

This function adds a feature to the metrology template of the specified metrology context. This function can also modify an existing feature of a metrology template. To delete features, use MmetControl() with M_DELETE.

To add several features to the template, you must call this function for each feature to add.

Features can either be physically measured from the target image or constructed from a set of other features, which can be referred to as base features. When a feature is composed of several instances of the same feature type, it is considered a multiple feature. Every instance of the feature type is a subfeature of the multiple feature. A single feature is a feature that has only one instance of the feature type. All supported features are single features, except for the edgel feature (M_MEASURED or M_CONSTRUCTED with M_EDGEL) and the measured multiple point feature (M_MEASURED with M_POINT). When adding a constructed feature that requires a point, it is possible to isolate and use a specific point subfeature of a measured multiple point feature. This is not true for the edgel feature since it is always used as a group.

Every feature has a position in the metrology template. For a physically measured feature, its position can be considered to be set according to the metrology region in which it is expected to be established (MmetSetRegion()). By default, the metrology region is infinite (the whole target image). For a constructed feature, its position in the template is defined by the position of the features used to construct it (its base features). The position of a feature is relative to a reference coordinate system, referred to as the reference frame. The global frame is the default. If the target image is not calibrated, the global frame's default origin (0,0) is aligned with the center of the top-left corner pixel of the target image. If the target image is calibrated, the global frame's default origin is aligned with the origin of the relative (world) coordinate system. To change the reference frame to a local frame, use MmetControl() with M_REFERENCE_FRAME. Local frames are coordinate systems located anywhere within the metrology template and defined as metrology features.

Once a feature is added to the template, it is possible to change its position using MmetSetPosition(). In addition, you can modify the feature so that it is constructed from a new set of base features and/or a new build operation; to do so, call **MmetAddFeature**() with M_MODIFY. Note that you cannot modify a feature's geometry.

To define the acceptable geometric relationship between several features in the template or the acceptable deviation from the definition of a feature, add a geometric tolerance, using MmetAddTolerance(). To calculate the added features and validate the geometric tolerances in the target image, use MmetCalculate(). To change the settings of a feature in the template, use MmetControl().

When adding a constructed feature with a fit operation, or when adding a constructed edgel feature with the clone operation, you can specify base features that have a delimited metrology region (MmetSetRegion()). By default, Aurora Imaging Library uses an infinite metrology region (M_INFINITE).

Parameters

ContextId

Specifies the identifier of the metrology context containing the template. The metrology context must have been previously allocated on the required system using MmetAlloc().

FeatureType INQ

Specifies the type of feature to add (M_CONSTRUCTED or M_MEASURED) or specifies to modify a feature (M_MODIFY).

See the Parameter associations section for possible values that can be specified.

Geometry INQ

Specifies the geometric shape of the feature to add. If modifying a feature (M_MODIFY), this parameter must be set to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

FeatureLabel

Specifies the label of the feature.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying label values																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that Aurora Imaging Library automatically selects the feature's label value. Once a feature is added, you can inquire its label using MmetInquire() with the feature's index and M_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label value. Each feature must have a unique label otherwise you will get an error. You can have an extremely high number of labels (up to **M_MET_FEATURE_ID_MAX**).	c		e	f	g	a		i		j	k	l	m	n	b

BuildOperation INQ

Specifies the operation to use to build the feature.

See the Parameter associations section for possible values that can be specified.

FeatureLabelArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1)
  Required array size:(SizeOfArray)

Specifies the array that holds the label of every feature to use to construct the feature. Features used to construct a feature are considered to be its base features.

Set this parameter to **M_NULL** when adding measured features or parametrically constructed features (M_PARAMETRIC).

SubFeatureIndexArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1)
  Required array size:(SizeOfArray)

Specifies the array that holds the index of every subfeature to use to construct the feature. Features used to construct a feature are considered to be its base features.

If all features referenced by FeatureLabelArrayPtr are single features, set SubFeatureIndexArrayPtr to **M_NULL**. This is also true when adding measured features or parametrically constructed features (M_PARAMETRIC).

If some features referenced by FeatureLabelArrayPtr are multiple features, set SubFeatureIndexArrayPtr to an array that is the same size as the FeatureLabelArrayPtr array. Set each element of the SubFeatureIndexArrayPtr array to the index of the subfeature to use for the feature at the corresponding element of the FeatureLabelArrayPtr array; for single features, set their corresponding element in the SubFeatureIndexArrayPtr array to 0.

SizeOfArray

Specifies the size of the arrays that hold the information that will be used to construct the feature (FeatureLabelArrayPtr and SubFeatureIndexArrayPtr). When setting the FeatureLabelArrayPtr parameter to **M_NULL**, set this parameter to 0.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the FeatureLabelArrayPtr or SubFeatureIndexArrayPtr parameter.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The tables below list possible values for the **FeatureType**, **Geometry**, and **BuildOperation** parameters.

- For adding a constructed feature
- For adding a measured feature
- For modifying a feature

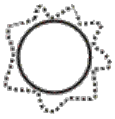
To add a constructed feature, the FeatureType, Geometry, and BuildOperation parameters can be set to the following values. Note that certain build operations for constructed features require base features, which you can specify using the FeatureLabelArrayPtr parameter and, if necessary, the SubFeatureIndexArrayPtr parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

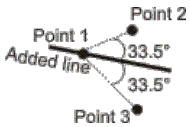
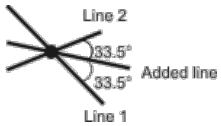
For adding a constructed feature																	
FeatureType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Geometry																
BuildOperation																

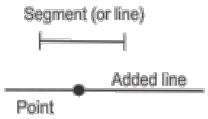
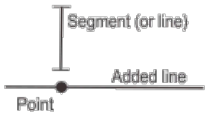
M_CONSTRUCTED	Adds a constructed feature. Constructed features are geometrically built using base features (other features added using **MmetAddFeature**()), or they are defined by their parametric controls (set using MmetControl()). In the latter case, they are referred to as parametrically constructed features (M_PARAMETRIC). Constructed features built using an M_CLONE_FEATURE operation are cloned from the specified base feature (FeatureLabelArrayPtr). For example, an arc can be cloned from an arc feature. Cloned features can be scaled, rotated, and/or moved using MmetControl() with M_CLONE_.... The base features of constructed features built using a fit operation (M_FIT, M_INNER_FIT, or M_OUTER_FIT) can be delimited by a metrology region. You must call MmetSetRegion() to specify the metrology region for each base feature. The base feature of a constructed edgel feature built using M_CLONE_FEATURE can also be delimited by a metrology region. Aurora Imaging Library only uses the edgels or points within the delimited region of the base feature to build the constructed feature; edgels or points outside the region are clipped. For constructed features that require several points or edgels, the actual number of point and edgel features can vary. For example, if 3 points are needed to construct a feature, it is possible to provide one multiple point feature containing three points, three separate point features, or one multiple point feature containing two points and one point subfeature. Only the total number of elements is important, not the number of features. If you do not specify the subfeature indices, then all subfeatures are used. Constructed features built with M_FIT depend on M_FIT_TYPE and the settings related to the type of fit selected. All fit operations will fail if their coverage is less than M_FIT_COVERAGE_MIN. If the constructed feature uses edgels as a source, then the fit operations are also affected by the gradient angle settings. You can adjust these using MmetControl() with M_EDGEL_ANGLE_RANGE and M_EDGEL_RELATIVE_ANGLE. When performing a standard best fit (M_STANDARD_FIT), more importance is typically given to the edgels whose gradient orientation follows the region's orientation.															
M_ARC	Adds a constructed arc feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b

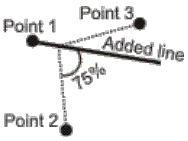
	M_CLONE_FEATURE	Specifies that the arc feature is cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONSTRUCTION	Specifies that the arc feature is built from a geometric construction using the specified base features. You can specify any three of the following as base features: points, circles, arcs, or local frames. For example, you can specify three point features, or one point, one circle, and one local frame feature. In all cases, Aurora Imaging Library requires three points to construct the arc. If you specify a circle or arc feature, Aurora Imaging Library takes its center point. If you specify a local frame feature, Aurora Imaging Library takes its origin point. To build the arc, Aurora Imaging Library uses the first, second, and third point as the center, the start, and the end of the arc, respectively. If the radius formed by the first and second points is not equal to the radius formed by the first and third points, the constructed arc will pass as close as possible to both the second and third points. This occurs because arc features are circular arcs, which are portions of a circle. Therefore, every point on the arc must have the same radius. Note that arcs follow the counter-clockwise convention. 	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIT	Specifies that the arc feature is built from the arc that best fits the specified base features. You can specify three or more edgels or points as base features. It is possible to have a combination of edgels and points. The best fit arc passes as close to as much data as possible while not necessarily touching it. Note that arcs follow the counter-clockwise convention. The angular constraints are ignored for the base features which do not have angle information. 	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies that the arc feature is built from the following MmetControl() parametric controls: M_POSITION_X, M_POSITION_Y, M_ANGLE_START, M_ANGLE_END, and M_RADIUS.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCLE		Adds a constructed circle feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b

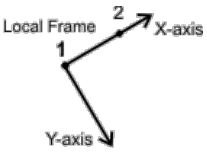
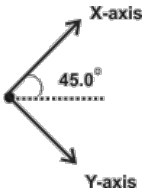
		M_CLONE_FEATURE	Specifies that the circle feature is cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CONSTRUCTION	Specifies that the circle feature is built from a geometric construction using the specified base features. You can specify any two of the following as base features: points, circles, arcs, or local frames. For example, you can specify two point features, or one point and one circle. In all cases, Aurora Imaging Library requires two points to construct the circle. If you specify a circle or arc feature, Aurora Imaging Library takes its center point. If you specify a local frame feature, Aurora Imaging Library takes its origin point. To build the circle, Aurora Imaging Library uses the first point as the center of the circle, while the second point is considered to be on the circle's contour. The distance between the two points is the new circle's radius.	c		e	f	g	a		i		j	k	l	m	n	b
		M_FIT	Specifies that the circle feature is built from the circle that best fits the specified base features. You can specify three or more edgels or points as base features. It is possible to have a combination of edgels and points. The best fit circle passes as close to as much data as possible while not necessarily touching it. The angular constraints are ignored for the base features which do not have angle information. 	c		e	f	g	a		i		j	k	l	m	n	b
		M_INNER_FIT	Specifies that the circle feature is built from the best inner fit of a circle around the specified base features. You can specify three or more edgels or points as base features. It is possible to have a combination of edgels and points. The inner fit for a circle feature is the largest possible circle that does not contain any edgels or points. The circle feature is fit to the edgels or points using M_STANDARD_FIT mode. Therefore, all settings related to M_STANDARD_FIT will affect the M_INNER_FIT circle. The angular constraints are ignored for the base features which do not have angle information. 	c		e	f	g	a		i		j	k	l	m	n	b

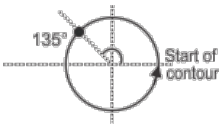
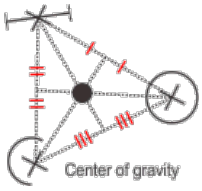
	M_OUTER_FIT	Specifies that the circle feature is built from the best outer fit of a circle around the specified base features. You can specify three or more edgels or points as base features. It is possible to have a combination of edgels and points. The outer fit for a circle feature is the smallest possible circle that contains all the edgels or points. The circle feature is fit to the edgels or points using M_STANDARD_FIT mode. Therefore, all settings related to M_STANDARD_FIT will affect the M_OUTER_FIT circle. The angular constraints are ignored for the base features which do not have angle information. 	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies that the circle feature is built from the following MmetControl() parametric controls: M_POSITION_X, M_POSITION_Y, and M_RADIUS.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL		Adds a constructed edgel feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_COPY_FEATURE_EDGELS.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CLONE_FEATURE	Specifies that the edgel feature is cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COPY_FEATURE_EDGELS	Specifies that the edgel feature is constructed by copying the edgels of the specified base features. You can specify physically measured features or constructed edgel features as base features. For physically measured base features, you can restrict the operation to copy only the active edgels or only the fitted edgels of the base features, using MmetControl() with M_EDGEL_TYPE. For constructed edgel base features, Aurora Imaging Library considers the edgels active (not fitted).	c		e	f	g	a		i		j	k	l	m	n	b
	M_EXTERNAL_FEATURE	Specifies that the edgel feature is constructed from edgels or points established using other Aurora Imaging Library modules or third party software. Use MmetPut() to add the edgels/points to the feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GROUP_EDGELS	Specifies that the edgel feature is constructed by grouping the edgels of the specified base features. By default, Aurora Imaging Library uses all edgels. To change this (for example, to group edgels according to the distance to a point), call MmetControl() with M_GROUP_SELECTION_MODE.	c		e	f	g	a		i		j	k	l	m	n	b

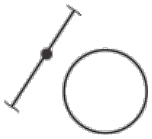
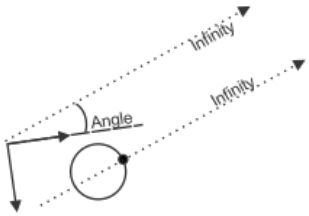
M_LINE	Adds a constructed line feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE	Specifies that the line feature is built at a specified angle from the specified linear base feature (segment or line), and passing through the specified point base feature. You can specify one or two base features. If one base feature is specified, it must be a point. In this case, the reference line from which the angle is measured is the X-axis of the reference frame. If two base features are specified, one must be a point and the other must be a line or a segment. The order in which you specify the base features does not matter. To specify the angle, use MmetControl() with M_ANGLE and set a value between 0° and 360°. The angle is measured counter-clockwise, relative to the linear feature (from start to end). 	c		e	f	g	a		i		j	k	l	m	n	b
M_BISECTOR	Specifies that the line feature is built by dividing an angle formed by the specified base features. You can specify three points or two lines as base features. By default, Aurora Imaging Library builds the feature by dividing the angle, formed by the base features, in two equal parts. If three points are specified, the first point is used as the vertex of the angle.  If two lines are specified, the bisector line is built where it divides the angle formed by the first line and the second line in the counter-clockwise direction.  If specifying two lines as base features, you can use MmetControl() with M_LINE_BISECTOR_TYPE to alter how the new line is built. For example, you can build the bisector line at the middle of the smallest or biggest angle formed by the two line base features.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLONE_FEATURE	Specifies that the line feature is cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b

		M_CONSTRUCTION	Specifies that the line feature is built from a geometric construction using the specified base features. You can specify any two of the following as base features: points, circles, arcs, or local frames. For example, you can specify two point features, or one point and one circle. In all cases, Aurora Imaging Library requires two points to construct the line. If you specify a circle or arc feature, Aurora Imaging Library takes its center point. If you specify a local frame feature, Aurora Imaging Library takes its origin point. To build the line, Aurora Imaging Library ensures that it passes through the two points specified.	c		e	f	g	a		i		j	k	l	m	n	b
		M_FIT	Specifies that the line feature is built from the resulting best fit of the specified base feature. You can specify a segment as the base feature.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PARALLEL	Specifies that the line feature is built parallel to the specified linear base feature (segment or line), and passing through the specified point base feature. You can specify one or two base features. If one base feature is specified, it must be a point. In this case, the line feature is built parallel to the X-axis of the reference frame. If two base features are specified, one must be a point and the other must be a line or a segment. The order in which you specify the base features does not matter. 	c		e	f	g	a		i		j	k	l	m	n	b
		M_PARAMETRIC	Specifies that the line feature is built from the following MmetControl() parametric controls: M_LINE_A, M_LINE_B, and M_LINE_C.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PERPENDICULAR	Specifies that the line feature is built perpendicular to the specified linear base feature (segment or line), and passing through the specified point base feature. You can specify one or two base features. If one base feature is specified, it must be a point. In this case, the line feature is built perpendicular to the X-axis of the reference frame. If two base features are specified, one must be a point and the other must be a line or a segment. The order in which you specify the base features does not matter. 	c		e	f	g	a		i		j	k	l	m	n	b

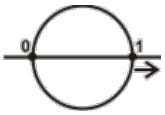
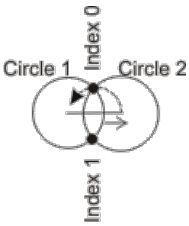
	M_SECTOR_RELATIVE	Specifies that the line feature is built where it divides, in two parts, an angle formed by the specified base features. You can specify three points as base features. The first point is used as the vertex of the angle. Specify the size of the angle between the line and the first pair of points (first and second points) using MmetControl() with M_ANGLE. Express the angle as a percentage of the total angle of all three points. 	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCAL_FRAME		Adds a constructed local frame feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CLONE_FEATURE	Specifies that the local frame feature is cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b

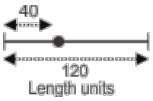
		M_CONSTRUCTION	Specifies that the local frame feature is built from a geometric construction using one or two specified base features. You can specify one or two of the following as base features: points, circles, arcs, or local frames. You also specify a segment if it is the only base feature. When you specify two base features, the local frame's origin is set at the center of the first base feature, and the local frame's X-axis is set to pass through and in the direction of the center of the second base feature. If you specify a local frame as a base feature, Aurora Imaging Library uses its origin to construct the local frame.  You can also build a local frame by specifying only one base feature. Aurora Imaging Library uses the center of the specified feature as the origin of the local frame. Aurora Imaging Library sets the local frame's X-axis at a specified angle from the X-axis of the reference frame; use MmetControl() with M_ANGLE to specify the local frame's angle. The angle is applied in the counter-clockwise direction. When the base feature is a segment, the origin of the local frame is set at the segment's center; the local frame's X-axis is aligned with the segment, pointing in the direction from the segment's first point to its second point.  	c		e	f	g	a		i		j	k	l	m	n	b
		M_PARAMETRIC	Specifies that the local frame feature is built from the following MmetControl() parametric controls: M_POSITION_X, M_POSITION_Y, and M_ANGLE. Note that the angle is measured counter-clockwise, and is relative to the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POINT		Adds a constructed point feature.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b

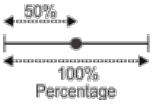
		M_ANGLE_ABSOLUTE	Specifies that the point feature is built at the specified absolute angle, on the contour of the specified base feature. You can specify either an arc or a circle as the base feature. The angle, in degrees, is relative to the start of the contour of the specified feature and is set using MmetControl() with M_ANGLE. The angle is measured counter-clockwise. Note that the start of a circle's contour is at its rightmost side, or 0°. This is the point where the circle's positive X-axis meets the circle, given a local origin at the circle's center. 	c		e	f	g	a		i		j	k	l	m	n	b
		M_ANGLE_RELATIVE	Specifies that the point feature is built at the specified relative angle, on the contour of the specified base feature. You can specify either an arc or a circle as the base feature. Specify the angle as a percentage, relative to the start of the contour using MmetControl() with M_ANGLE. Note that the start of a circle's contour is at its rightmost side, or 0°. This is the point where the circle's positive X-axis meets the circle, given a local origin at the circle's center. 	c		e	f	g	a		i		j	k	l	m	n	b
		M_CENTER	Specifies that the point feature is built at the center of the specified base feature(s). You can specify one, or a combination, of the following as base features: arc, circle, edgel, local frame, point, or segment. If multiple features are provided, the point feature is built at the center of gravity of the centers of all the features.  M_CENTER does not necessarily build a point on the feature's edge. Use M_MIDDLE if you want to build a point at the edge's midpoint.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CLONE_FEATURE	Specifies that the point feature is cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b

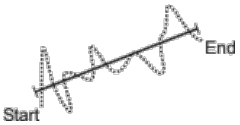
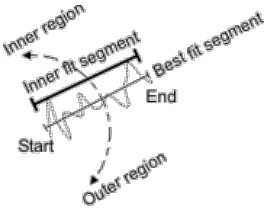
		M_CLOSEST	Specifies that the point feature is built at the point on the first specified base feature that is, by default, closest to the other (one or more) specified base features.  You can specify any features as the base features. You can explicitly specify an angle at which to build the closest point between the features, by calling MmetControl() with M_DISTANCE_MODE. This can be seen as a type of caliper feature. Aurora Imaging Library considers the specified distance mode angle at which to measure the closest distance to be relative to the first base feature. Aurora Imaging Library applies the angle counter-clockwise.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CLOSEST_TO_INFINITE_POINT	Specifies that the point feature is built, at the position on the base feature, that is the closest to an infinite point. This is a virtual point that Aurora Imaging Library establishes, at infinity, along the direction (X-axis) of the local frame. To change the angle along which to establish the infinite point, call MmetControl() with M_ANGLE. Aurora Imaging Library measures this angle counter-clockwise, relative to the X-axis of the local frame.  The base features can be any feature except line, which is itself infinite (unbounded). You can, for example, use M_CLOSEST_TO_INFINITE_POINT to establish the boundaries of an edgel feature by fetching the coordinates of the constructed closest infinite points at 0°, 90, 180°, and 270° angles.	c		e	f	g	a		i		j	k	l	m	n	b

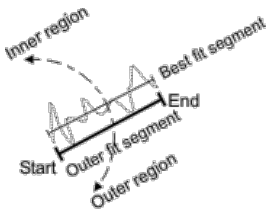
		M_EXTENDED_INTERSECTION	Specifies that the point feature is built at the point of intersection between the extension of the first and second specified base features. Note that the following can be extended: a segment, which is extended into a line, and an arc, which is extended into a circle. You can specify the following as base features: • A segment feature and a segment, arc, circle, line, segment, or edgel feature. • An arc feature and a segment, arc, circle, line, or edgel feature. • A circle feature and a segment or arc feature. • A line feature and a segment or arc feature. • An edgel feature and a segment or arc feature. If there is more than one intersection candidate, specify the index of the intersection at which to build the feature, using MmetControl() with M_OCCURRENCE; the intersection is relative to the first feature that has a direction. The default intersection is the one indexed as 0. 	c		e	f	g	a		i		j	k	l	m	n	b

		M_INTERSECTION	Specifies that the point feature is built at the point of intersection between the first and second specified base features. You can specify the following as base features: • A segment feature and a segment, arc, circle, line, or edgel feature. • An arc feature and a segment, arc, circle, line, or edgel feature. • A circle feature and a segment, arc, circle, or edgel feature. • A line feature and a segment, arc, line, or edgel feature. • An edgel feature and a segment, arc, circle, line, or edgel feature. If there is more than one intersection candidate, specify the index of the intersection at which to build the feature, using MmetControl() with M_OCCURRENCE; the intersection is relative to the first feature that has a direction. The default intersection is the one indexed as 0.  For two circle intersections, specify the intersections relative to the segment that can be created by joining the circles' centers. The first intersection obtained by scanning the first circle counter-clockwise from the segment is indexed as 0; the other is indexed at 1. 	c		e	f	g	a		i		j	k	l	m	n	b
		M_MAX_DISTANCE_POINT	Specifies that the point feature is built at the point on the first specified base feature that is, by default, farthest from any one point on the other (one or more) specified base features. You can specify any feature except line as the first base feature, and any feature as the other base features.	c		e	f	g	a		i		j	k	l	m	n	b

		M_MAX_OF_MIN_DISTANCE_POINT	Specifies that the point feature is built at the point on the first specified base feature that is the farthest of all the closest points on the second base feature. This point can be considered the maximin (the greatest of all the smallest). You can specify any feature except a line as the first base feature, and any feature except an edgel as the second base feature.	c		e	f	g	a		i		j	k	l	m	n	b
		M_MIDDLE	Specifies that the point feature is built at the midpoint of the contour (edge) of the specified base feature. You can specify a segment, arc, circle, or edgel as the base feature. The middle of the contour of a circle is where the circle intersects the negative X-axis at 180°. When using an edgel as the base feature, the point feature is built at the position of the edgel occupying the center position in the group (rounded down). For example, for an edgel feature containing 5 edgels, [Edgel0, Edgel1, Edgel2, Edgel3, Edgel4], the point is built at Edgel2's position. M_MIDDLE builds a point, located on the feature's edge, that splits the edge at the halfway mark. Use M_CENTER if you want to build the point feature at the center point of the specified features and not necessarily on the feature's edge.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PARAMETRIC	Specifies that the point feature is built from the following MmetControl() parametric controls: M_POSITION_X and M_POSITION_Y.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSITION_ABSOLUTE	Specifies that the point feature is built at the specified position on the contour of the specified base feature. You can specify a segment, arc, circle, or edgel feature as the base feature. The position is relative to the start of the contour of the specified feature and is set using MmetControl() with M_POSITION. In the example below, M_POSITION has a value of 40 and the provided segment feature has a length of 120.  Note that the start of a circle's contour is at its rightmost side, or 0°. This is the point where the circle's positive X-axis meets the circle, given a local origin at the circle's center. The start of the contour of an edgel feature is the first edgel subfeature of the edgel feature.	c		e	f	g	a		i		j	k	l	m	n	b

		M_POSITION_END	Specifies that the point feature is built at the end of the contour of the specified base feature. You can specify a segment, arc, circle, or edgel feature as the base feature. Note that the end of a circle's contour is at its rightmost side, or 360°. This is the point where the circle's positive X-axis meets the circle, given a local origin at the circle's center, and is the same point as its start of contour. The end of the contour of an edgel feature is the last edgel subfeature of the edgel feature.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSITION_RELATIVE	Specifies that the point feature is built at the specified relative position on the contour of the specified base feature. You can specify a segment, arc, circle, or edgel feature as the base feature. Specify the position as a percentage of the total length of the specified feature's contour, relative to the start of the contour, using MmetControl() with M_POSITION. In the example below, M_POSITION is set to 50%.  Note that the start of a circle's contour is at its rightmost side, or 0°. This is the point where the circle's positive X-axis meets the circle, given a local origin at the circle's center. The start of the contour of an edgel feature is the first edgel subfeature of the edgel feature.	c		e	f	g	a		i		j	k	l	m	n	b
		M_POSITION_START	Specifies that the point feature is built at the start position of the contour of the specified base feature. You can specify a segment, arc, circle, or edgel feature as the base feature. Note that the start of a circle's contour is at its rightmost side, or 0°. This is the point where the circle's positive X-axis meets the circle, given a local origin at the circle's center. The start of the contour of an edgel feature is the first edgel subfeature of the edgel feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEGMENT		Adds a constructed segment feature.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CLONE_FEATURE	Specifies that the segment feature is cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b

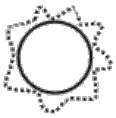
		M_CONSTRUCTION	Specifies that the segment feature is built from a geometric construction using the specified base features. You can specify any two of the following as base features: points, circles, arcs, or local frames. For example, you can specify two point features, or one point and one circle. In all cases, Aurora Imaging Library requires two points to construct the segment. If you specify a circle or arc feature, Aurora Imaging Library takes its center point. If you specify a local frame feature, Aurora Imaging Library takes its origin point. To build the segment, Aurora Imaging Library uses the first point as the start of the segment and the second as the end of the segment.	c		e	f	g	a		i		j	k	l	m	n	b
		M_FIT	Specifies that the segment feature is built from the line that best fits the specified base features. You can specify two or more edgels or points as base features. The best fit segment passes as close to as much data as possible while not necessarily touching it. The start of the segment is the extremity closest to the origin of the reference frame. The angular constraints are ignored for the base features which do not have angle information. 	c		e	f	g	a		i		j	k	l	m	n	b
		M_INNER_FIT	Specifies that the segment feature is built from the segment that has the best inner fit in the specified base features. You can specify two or more edgels or points as base features. The edgels used to find the inner fit segment are the active edgels on the counter-clockwise side of the best fit segment (from start to end). These edgels are considered active edgels since they satisfy the edgel constraints set using MmetControl(). The best fit segment, whose starting point is the extremity closest to the origin of the global frame, separates the inner and outer fit regions. The inner fit segment passes through the two farthest edgels of the inner fit region such that all the edgels of the inner fit region are located on the same side of the inner fit segment. All the active edgels can be projected on the segment, so the segment's extremities are determined by the two farthest active edgels. The angular constraints are ignored for the base features which do not have angle information. 	c		e	f	g	a		i		j	k	l	m	n	b

		M_OUTER_FIT	Specifies that the segment feature is built from the outer fit of the specified base features. You can specify two or more edgels or points as base features. The edgels used to find the outer fit segment are the active edgels on the clockwise side of the best fit segment, relative to its direction. These edgels are considered active edgels since they satisfy the edgel constraints set using MmetControl(). The best fit segment, whose starting point is the extremity closest to the origin of the global frame, separates the inner and outer fit regions. The outer fit segment passes through the two farthest edgels of the outer fit region such that all the edgels of the outer fit region are located on the same side of the outer fit segment. All the active edgels can be projected on the segment, so the segment's extremities are determined by the two farthest active edgels. The angular constraints are ignored for the base features which do not have angle information. 	c		e	f	g	a		i		j	k	l	m	n	b
		M_PARAMETRIC	Specifies that the segment feature is built from the following MmetControl() parametric controls: M_POSITION_START_X, M_POSITION_START_Y, M_POSITION_END_X, and M_POSITION_END_Y.	c		e	f	g	a		i		j	k	l	m	n	b

To add a measured feature, the FeatureType, Geometry, and BuildOperation parameters can be set to the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For adding a measured feature																		
FeatureType			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	Geometry																	
		BuildOperation																

M_MEASURED	Adds a physically measured feature. Measured features are built using data (contour edgels) extracted from the target image within the metrology region. Measured features built with M_FIT depend on M_FIT_TYPE and the settings related to the type of fit selected. All fit operations will fail if their coverage is less than M_FIT_COVERAGE_MIN. Fit operations for measured features are also affected by the gradient angle settings, which you can control using M_EDGEL_ANGLE_RANGE and M_EDGEL_RELATIVE_ANGLE. When performing a standard best fit (M_STANDARD_FIT), more importance is typically given to the edgels whose gradient orientation follows the region's orientation.															
M_ARC	Adds a physically measured arc feature. In this case, you must use an infinite or a ring-sector metrology region (MmetSetRegion()).	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FIT.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT	Specifies that the measured arc feature is built using a best fit operation applied to the data (edgels) extracted from the metrology region. The best fit arc passes as close to as much data as possible while not necessarily touching it. 	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCLE	Adds a physically measured circle feature. In this case, you must use an infinite or a ring metrology region (MmetSetRegion()).	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FIT.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT	Specifies that the measured circle feature is built using a best fit operation applied to the data (edgels) extracted from the metrology region. The best fit circle passes as close to as much data as possible while not necessarily touching it. 	c		e	f	g	a		i		j	k	l	m	n	b
M_INNER_FIT	Specifies that the measured circle feature is built using the best inner fit operation applied to the data (edgels) extracted from the metrology region. The inner fit for a circle feature is the largest possible circle that does not contain any edgels. The circle feature is fit to the edgels using M_STANDARD_FIT mode. Therefore, all settings related to M_STANDARD_FIT will affect the M_INNER_FIT circle. 	c		e	f	g	a		i		j	k	l	m	n	b

	M_OUTER_FIT	Specifies that the measured circle feature is built using the best outer fit operation applied to the data (edgels) extracted from the metrology region. The outer fit for a circle feature is the smallest possible circle that contains all the edgels. The circle feature is fit to the edgels using M_STANDARD_FIT mode. Therefore, all settings related to M_STANDARD_FIT will affect the M_OUTER_FIT circle.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL		Adds a physically measured edgel feature. In this case, you must use an infinite, rectangle, ring, or ring-sector metrology region (MmetSetRegion()).	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies that the measured edgel feature is built by extracting all the edgels in the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT		Adds a physically measured point feature. In this case, you must use an arc, infinite, or segment metrology region (MmetSetRegion()).	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_FIT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIT	Specifies that the measured point feature is built using a best fit operation applied to the data (edgels) extracted from the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEGMENT		Adds a physically measured segment feature. In this case, you must use an infinite, rectangle, or ring-sector metrology region (MmetSetRegion()).	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_FIT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIT	Specifies that the measured segment feature is built from the line that best fits the data (edgels) extracted from the metrology region. The best fit segment passes as close to as much data as possible while not necessarily touching it. The start of the segment is the extremity closest to the origin of the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INNER_FIT	Specifies that the measured segment feature is built from the segment that has the best inner fit applied to the data (edgels) extracted from the metrology region. For more information on how an inner fit segment differs from a best fit segment, see M_CONSTRUCTED with M_SEGMENT and M_INNER_FIT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OUTER_FIT	Specifies that the measured segment feature is built from the segment that has the best outer fit applied to the data (edgels) extracted from the metrology region. For more information on how an outer fit segment differs from a best fit segment, see M_CONSTRUCTED with M_SEGMENT and M_OUTER_FIT.	c		e	f	g	a		i		j	k	l	m	n	b

To modify a feature, the FeatureType, Geometry, and BuildOperation parameters can be set to the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For modifying a feature																	
FeatureType / Geometry / BuildOperation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MODIFY	Modifies the build operation of the specified feature, and/or for a constructed feature, modifies its base features, which is the set of features used to build the constructed feature. To specify the new features from which to construct the specified feature, use FeatureLabelArrayPtr and SubFeatureIndexArrayPtr.															
M_DEFAULT	Specifies to use the current geometry of the feature. The selected feature's geometry cannot be changed.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to change only the features from which the selected feature is built (its base features) and that the selected feature's build operation should not be changed.	c		e	f	g	a		i		j	k	l	m	n	b
BuildOperation	Specifies a new build operation for the selected feature. The new build operation must be appropriate for the feature's geometry. See the For adding a constructed feature or For adding a measured feature table for possible build operations given the feature's geometry. > When modifying a constructed feature, you must pass the list of base features from which to construct the specified feature (FeatureLabelArrayPtr and SubFeatureIndexArrayPtr), even if you only need to modify the build operation.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetAddTolerance

Synopsis

Add a geometric tolerance to or modify a geometric tolerance of the metrology template of the metrology context.

Syntax

```
void MmetAddTolerance(
    AIL_ID ContextId,                       //out
    AIL_INT64 ToleranceType,                //in
    AIL_INT ToleranceLabel,                 //in
    AIL_DOUBLE ValueMin,                    //in
    AIL_DOUBLE ValueMax,                    //in
    const AIL_INT *FeatureLabelArrayPtr,    //in
    const AIL_INT *SubFeatureIndexArrayPtr, //in
    AIL_INT SizeOfArray,                    //in
    AIL_INT64 ControlFlag                   //in
)
```

Description

This function adds a geometric tolerance to the metrology template of the specified metrology context. This function can also modify an existing tolerance of a metrology template. To delete geometric tolerances, use MmetControl() with M_DELETE.

To add several geometric tolerances, you must call this function for each tolerance to add.

A geometric tolerance defines the acceptable deviation from the definition of a feature, or the acceptable relationship between multiple features. For example, you can add a geometric tolerance to the metrology template to specify that one feature must be a certain length, or that two features must be perpendicular. To add a feature, use MmetAddFeature().

When adding a geometric tolerance that requires a point, you can use a specific point subfeature of a measured multiple point feature by specifying its index with the SubFeatureIndexArrayPtr parameter. Note that you cannot do this for other features, including the measured multiple edgel feature, as it is always used as a group.

When adding a geometric tolerance, you must also set its minimum and maximum limit values with the ValueMin and ValueMax parameters. When defining the tolerance of one feature, these limits represent the acceptable deviation from the definition of the feature. For example, a segment's length tolerance can be between 90 and 100 pixels. For multiple features, these limits represent the valid range of acceptable values between the features. For example, the perpendicular tolerance between two segments can be +/- 0.05° (that is, 89.95° to 90.05°). You can also set warning values to alert you when the tolerance is close to its minimum and maximum limits, using MmetControl() with M_VALUE_WARNING_MIN and M_VALUE_WARNING_MAX.

When you call MmetCalculate(), the tolerance is calculated (for example, 95 pixels) and a status is assigned (for example, M_PASS). To retrieve the tolerance value or the status, use MmetGetResult() with M_TOLERANCE_VALUE or M_STATUS, respectively.

You can modify a geometric tolerance (using M_MODIFY) so that it uses a new set of features or subfeatures. In this case, you can also modify the minimum and maximum limit values of a tolerance.

If a camera calibration context is associated with the target image, set values in real-world units (for example, tolerance values and warning values, and positional values). Otherwise use pixel units. For more information, see the Camera calibration - overview section of Chapter 28: Calibrating your camera setup.

Parameters

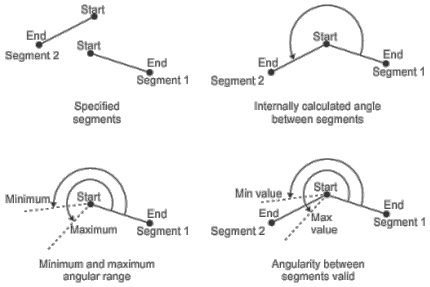
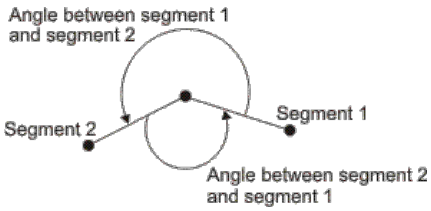
ContextId

Specifies the identifier of the metrology context. The metrology context must have been previously allocated on the required system using MmetAlloc().

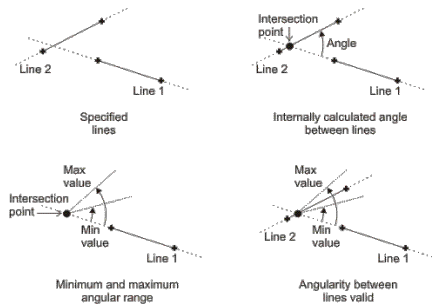
ToleranceType INQ

Specifies the type of geometric tolerance to add. The icon (image symbol) of every tolerance type is in its description. You can draw this icon using MmetDraw() with M_DRAW_TOLERANCE.

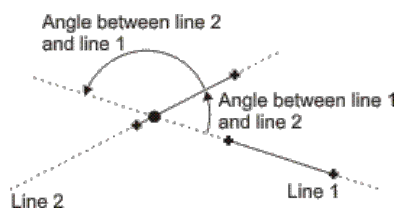
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For adding geometric tolerances																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ANGULARITY	Adds an angularity tolerance. The icon for this tolerance is: An angularity tolerance validates that the angle between two features falls within the specified angular range (for example, that two lines intersect at an angle between 25° and 35°). An angularity tolerance can be applied between the following features. • Two segment features. To validate the angularity between two segment features, Aurora Imaging Library internally connects each segment at their start point and considers the angle between them to be, counter-clockwise, from the first segment to the second segment. This angle is valid if it falls within the angular range set by the minimum and maximum tolerance limits. To designate the valid angular range, the minimum and maximum tolerance limits are angles applied counter-clockwise around the start point of the first segment. Note that Aurora Imaging Library does not physically rearrange the specified segments, it only orients them internally to measure the required angles.  Each segment's start point, as well as the order you list the segments in the FeatureLabelArrayPtr parameter, can affect your results. For example, the angle between segment 1 and segment 2 is different than the angle between segment 2 and segment 1.  • Two line features. To validate the angularity between two line	c		e	f	g	a		i		j	k	l	m	n	b

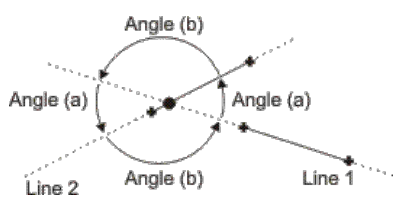
features, Aurora Imaging Library considers the angle between them to be, counter-clockwise, from the first line to the second line. This angle is valid if it falls within the angular range set by the minimum and maximum tolerance limits. To designate the valid angular range, the minimum and maximum tolerance limits are angles applied counter-clockwise, from the first line to the second line, around the intersection point of the two lines.



The order you list the lines in the FeatureLabelArrayPtr parameter can affect your results. For example, the angle between line 1 and line 2 is different than the angle between line 2 and line 1.

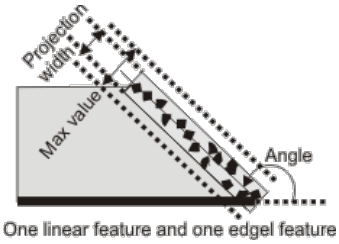


Since lines are infinite, there are actually two ways to measure the angle between line 1 and line 2 (indicated as angle (a) in the following diagram) and between line 2 and line 1 (indicated as angle (b) in the following diagram). Note that this is inconsequential since each pair of angles is equivalent.



- A linear feature (segment or line) and an edgel feature.

  For one linear feature (segment or line) and one edgel feature, M_ANGULARITY validates the width of the projection of the edgel feature, along the nominal angle specified using MmetControl() with M_ANGLE. The angle is in the counter-clockwise direction relative to the linear feature. The minimum and maximum tolerance parameters set the valid projection width. Typically, you should set the minimum width value to 0. The width of the projection is in pixel or world units. The second feature specified in the FeatureLabelArrayPtr parameter must be edgel.

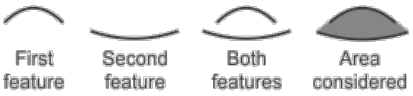
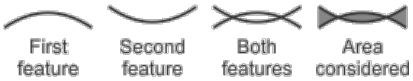


One linear feature and one edgel feature

- Three point features.

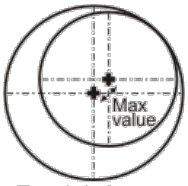
  To validate the angularity between three point features, Aurora Imaging Library considers the first point as the center of the angularity tolerance, the second point as the *StartAngle* (in the counter-clockwise direction relative to the first point), and the third point as the *EndAngle* (in the counter-clockwise direction relative to the first point). M_ANGULARITY validates the angle that results from *EndAngle* - *StartAngle*.

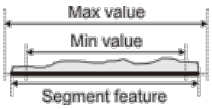
Note that an angularity tolerance between a segment and a line produces too many ambiguities and therefore cannot be calculated.

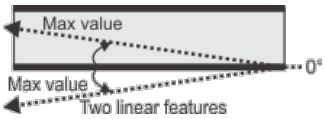
M_AREA_BETWEEN_CURVES	Adds an area tolerance for the area between two curves of edgels. The icon for this tolerance is: To use M_AREA_BETWEEN_CURVES, you must specify two edgel features (FeatureLabelArrayPtr). Aurora Imaging Library internally joins the edgels of each edgel feature to form two curves, between which the area is validated.  When specifying an edgel feature with this tolerance, Aurora Imaging Library internally sorts and filters the edgels, and uses a maximum gap between them, to determine whether they are part of the same curve. To specify an explicit gap size, call MmetControl() with M_CURVE_EDGEL_GAP_SIZE. By default, M_AREA_BETWEEN_CURVES assumes each edgel (curve) feature is unbounded (open), and does not expect the edgels to follow a specific direction. To modify this behavior and potentially speed up calculations and accuracy, call MmetControl() with M_CURVE_INFO. If the two curves cross over each other, there will be areas on both sides of the curves.  By default, Aurora Imaging Library adds all the areas. To modify this behavior (for example, to calculate the area on only one side of a curve), call MmetControl() with M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY or M_AREA_BETWEEN_CURVES_OPPOSITES_SUBTRACT. If you maintain the default behavior of M_AREA_BETWEEN_CURVES, the order in which you specify the curves is inconsequential. To draw the actual area that Aurora Imaging Library uses to define this tolerance, call MmetDraw() with M_DRAW_TOLERANCE_AREA....	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_CONVEX_HULL	Adds an area tolerance, based on the convex hull, for the surface of an edgel, point, or circle feature. The icon for this tolerance is: M_AREA_CONVEX_HULL validates that the surface area (convex hull) of the specified feature (FeatureLabelArrayPtr) falls within the minimum and maximum limit values. To draw the actual area that Aurora Imaging Library uses to define this tolerance, call MmetDraw() with M_DRAW_TOLERANCE_AREA....	c		e	f	g	a		i		j	k	l	m	n	b

M_AREA_SIMPLE	Adds a simple area tolerance for the surface of an edgel, point, or circle feature. The icon for this tolerance is: M_AREA_SIMPLE validates that the surface area of the specified feature (FeatureLabelArrayPtr) falls within the minimum and maximum limit values. To draw the actual area that Aurora Imaging Library uses to define this tolerance, call MmetDraw() with M_DRAW_TOLERANCE_AREA....	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_UNDER_CURVE_MAX	Adds an area tolerance for the maximum area under a curve of edgels or points. The icon for this tolerance is: M_AREA_UNDER_CURVE_MAX validates the maximum area between a curve (edgel or point) feature and a line feature (FeatureLabelArrayPtr). Aurora Imaging Library considers the maximum area to be between an internally established curve under the furthest set of edgels or points and the line. The order of the features specified with the FeatureLabelArrayPtr parameter is inconsequential, since Aurora Imaging Library always validates the curve (edgel or point) feature against the line feature. M_AREA_UNDER_CURVE_MAX is typically used to validate the entire area under a curve (edgel or point) feature and a line, regardless of any interfering edges or noise. When specifying an edgel feature with this tolerance, Aurora Imaging Library internally sorts and filters the edgels, and uses a maximum gap between them, to determine whether they are part of the same curve. To specify an explicit gap size, call MmetControl() with M_CURVE_EDGEL_GAP_SIZE. When specifying a point feature with this tolerance, Aurora Imaging Library does not manipulate the points and chains all of them, assuming the resulting curve is well defined; for example, Aurora Imaging Library assumes the chained points do not create a degenerate polygonal figure, where curves cross over each other. If the specified curve (edgel or point) feature that you are validating crosses over the specified line feature, you are considered to have opposing areas (that is, there are areas on both sides of the line), and the tolerance fails, by default. You can allow Aurora Imaging Library to consider opposing (negative) areas when establishing the area tolerance, by calling MmetControl() with M_AREA_UNDER_CURVE_ALLOW_NEGATIVE. In general, the curve (edgel or point) feature should remain on one side of the line. To draw the actual area that Aurora Imaging Library uses to define this tolerance, call MmetDraw() with M_DRAW_TOLERANCE_AREA....	c		e	f	g	a		i		j	k	l	m	n	b

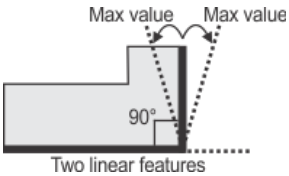
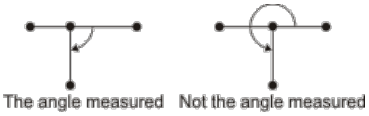
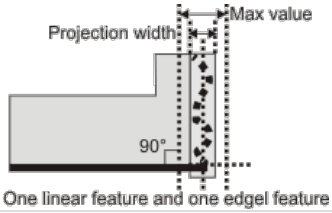
		c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_UNDER_CURVE_MIN	Adds an area tolerance for the minimum area under a curve of edgels. The icon for this tolerance is: M_AREA_UNDER_CURVE_MIN validates the minimum area between a curve (edgel or point) feature and a line feature (FeatureLabelArrayPtr). Aurora Imaging Library considers the minimum area to be between an internally established curve under the furthest set of edgels or points and the line. The order of the features specified with the FeatureLabelArrayPtr parameter is inconsequential, since Aurora Imaging Library always validates the curve (edgel or point) feature against the line feature. M_AREA_UNDER_CURVE_MIN is typically used to validate the exclusive area under a curve (edgel or point) feature and a line, which excludes the area under the curve and any interfering edges or noise. When specifying an edgel feature with this tolerance, Aurora Imaging Library performs an internal sort and filtering of the edgels, and uses a maximum gap between edgels to determine whether they are part of the same curve. To specify an explicit gap size, call MmetControl() with M_CURVE_EDGEL_GAP_SIZE. When specifying a point feature with this tolerance, Aurora Imaging Library does not manipulate the points and chains all of them, assuming the resulting curve is well defined; for example, Aurora Imaging Library assumes the chained points do not create a degenerate polygonal figure, where curves cross over each other. If the specified curve (edgel or point) feature that the tolerance validates crosses over the specified line feature, you are considered to have opposing areas (that is, there are areas on both sides of the line), and the tolerance fails, by default. You can allow Aurora Imaging Library to consider opposing (negative) areas when establishing the area tolerance, by calling MmetControl() with M_AREA_UNDER_CURVE_ALLOW_NEGATIVE. In general, the curve (edgel or point) feature should remain on one side of the line. To draw the actual area that Aurora Imaging Library uses to define this tolerance, call MmetDraw() with M_DRAW_TOLERANCE_AREA....														

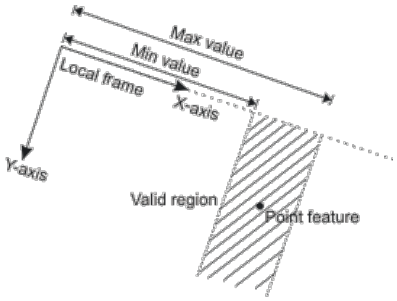
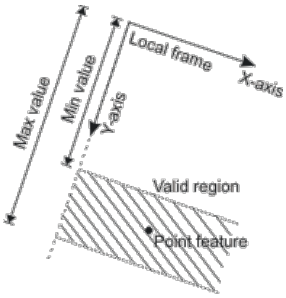
M_CONCENTRICITY	Adds a concentricity tolerance. The icon for this tolerance is: M_CONCENTRICITY validates the concentricity between a combination of two of the following features: arc or circle. This tolerance validates the distance between the centers of the features. Ideally, concentric features have the same center. The order of the features specified in the FeatureLabelArrayPtr parameter is inconsequential.  Two circle features	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_MAX	Adds a maximum distance tolerance. The icon for this tolerance is: M_DISTANCE_MAX validates that a specified maximum distance is separating any two features except lines. By default, Aurora Imaging Library uses the distance at which the features are the farthest. In this case, the order of the features in the FeatureLabelArrayPtr parameter is inconsequential. Maximum distance examples You can explicitly specify an angle at which to apply the maximum distance tolerance between two features, by calling MmetControl() with M_DISTANCE_MODE. This can be seen as a type of caliper tolerance between the features. In this case, the order of the features in the FeatureLabelArrayPtr parameter can affect your results, since Aurora Imaging Library considers the specified angle at which to measure the maximum distance to be relative to the first feature. Aurora Imaging Library applies the angle counter-clockwise.	c		e	f	g	a		i		j	k	l	m	n	b

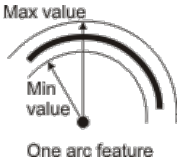
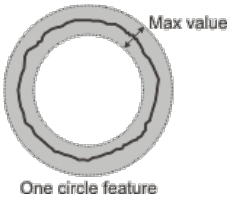
M_DISTANCE_MIN	Adds a minimum distance tolerance. The icon for this tolerance is: M_DISTANCE_MIN validates that a specified minimum distance is separating any two features. By default, Aurora Imaging Library uses the distance at which the features are the closest. In this case, the order of the features in the FeatureLabelArrayPtr parameter is inconsequential. Minimum distance examples You can explicitly specify an angle at which to apply the minimum distance tolerance between two features, by calling MmetControl() with M_DISTANCE_MODE. This can be seen as a type of caliper tolerance between the features. In this case, the order of the features in the FeatureLabelArrayPtr parameter can affect your results, since Aurora Imaging Library considers the specified angle at which to measure the minimum distance to be relative to the first feature. Aurora Imaging Library applies the angle counter-clockwise.	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH	Adds a length tolerance. When used with an arc or circle, the icon for this tolerance is: When used with a segment or edgel, the icon for this tolerance is: M_LENGTH validates the length of one of the following features: segment, edgel, circle, or arc. The length of a circle is its circumference.  You should only use M_LENGTH with an edgel feature if it represents one continuous curve (path) that does not cross over itself.	c		e	f	g	a		i		j	k	l	m	n	b

M_PARALLELISM	Adds a parallelism tolerance. The icon for this tolerance is: A parallelism tolerance validates the degree to which two features are parallel. A parallelism tolerance can be applied between the following features: • Two linear features (segments and lines). • A linear feature (segment or line) and an edgel feature. For any combination of 2 linear features (segments and lines), M_PARALLELISM validates that the angle of the two features, relative to the global reference frame, is the same. The maximum tolerance parameter sets the valid angular range, from the nominal angle (0°) (that is, 0° +/- ValueMax). In this case, you should only set the maximum tolerance parameter, and set the minimum tolerance parameter to 0. The order of the features specified in the FeatureLabelArrayPtr parameter, as well as the start/end of the segments, is inconsequential.  Note that, for parallelism tolerances between two linear features, angles are remapped to 0°; that is, the angle that is measured, and the angular range that you must provide, is the angle that is closest to 0°. For a linear feature (segment or line) and an edgel feature, M_PARALLELISM validates the width of the projection of the edgel feature on a theoretical line that is perpendicular to the segment or line feature. The minimum and maximum tolerance values set the valid projection width. The smaller the width, the more the edgel feature is parallel to the given segment or line feature. Typically, the minimum width value is set to 0. The width of the projection is in pixel or world units. The second feature specified in the FeatureLabelArrayPtr parameter must be edgel. 	c		e	f	g	a		i		j	k	l	m	n	b
M_PERIMETER_CONVEX_HULL	Adds a perimeter tolerance, based on the convex hull, for the surface of an edgel, point, or circle feature. The icon for this tolerance is: M_PERIMETER_CONVEX_HULL validates that the surface perimeter (convex hull) of the specified feature (FeatureLabelArrayPtr) falls within the minimum and maximum limit values.	c		e	f	g	a		i		j	k	l	m	n	b

M_PERIMETER_SIMPLE	Adds a simple perimeter tolerance for the surface of an edgel, point, or circle feature. The icon for this tolerance is: M_PERIMETER_SIMPLE validates that the surface perimeter of the specified feature (FeatureLabelArrayPtr) falls within the minimum and maximum limit values.	c		e	f	g	a		i		j	k	l	m	n	b

		c		e	f	g	a		i		j	k	l	m	n	b
M_PERPENDICULARITY	Adds a perpendicularity tolerance. The icon for this tolerance is: A perpendicularity tolerance validates the degree to which two features are perpendicular. A perpendicularity tolerance can be applied between the following features: • Two linear features (segments and lines). • A linear feature (segment or line) and an edgel feature. For any combination of 2 linear features (segments and lines), M_PERPENDICULARITY validates that the angle between the two features is 90°. The maximum tolerance parameter sets the valid angular range, from the nominal angle (90°) (that is, 90° +/- ValueMax). In this case, you should only set the maximum tolerance parameter, and set the minimum tolerance parameter to 0. The order of the features specified in the FeatureLabelArrayPtr parameter, as well as the start/end of the segments, is inconsequential.  Two linear features Note that, for perpendicularity tolerances between two linear features, angles are remapped to 90°; that is, the angle that is measured, and the angular range that you must provide, is the angle that is closest to 90°.  The angle measured Not the angle measured For a linear feature (segment or line) and an edgel feature, M_PERPENDICULARITY validates the width of the projection of the edgel feature on a theoretical line that is parallel to the segment or line feature. The minimum and maximum tolerance values set the valid projection width. The smaller the width, the more the edgel feature is perpendicular to the given segment or line feature. Typically, the minimum width value is set to 0. The width of the projection is in pixel or world units. The second feature specified in the FeatureLabelArrayPtr parameter must be edgel.  One linear feature and one edgel feature														

M_POSITION_X	Adds a positioning tolerance, along the X-direction of the specified reference frame. The icon for this tolerance is: A positioning tolerance can be used to validate the geometric relationship between a reference frame and one of the following features: local frame, point, or circle. For a circle feature, only the position of its center is validated, not its contour. If you only specify one feature, Aurora Imaging Library validates the geometric relationship between that feature and its reference frame. When using a positioning tolerance, the first element provided to the FeatureLabelArrayPtr parameter must be the label value of the reference frame, while the second element must be the label value of the feature to validate. If you only specify one feature, Aurora Imaging Library validates the geometric relationship between that feature and its reference frame (every feature is associated to a reference frame).  X-position tolerance	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_Y	Adds a positioning tolerance, along the Y-direction of the specified reference frame. The icon for this tolerance is: A positioning tolerance can be used to validate the geometric relationship between a reference frame and one of the following features: local frame, point, or circle. For a circle feature, only the position of its center is validated, not its contour. When using a positioning tolerance, the first element provided to the FeatureLabelArrayPtr parameter must be the label value of the reference frame, while the second element must be the label value of the feature to validate. If you only specify one feature, Aurora Imaging Library validates the geometric relationship between that feature and its reference frame (every feature is associated to a reference frame).  Y-position tolerance	c		e	f	g	a		i		j	k	l	m	n	b

M_RADIUS	Adds a radius tolerance. The icon for this tolerance is: M_RADIUS validates the radius of the specified arc or circle feature. The radius is the linear length between the circle or arc's center and perimeter. 	c		e	f	g	a		i		j	k	l	m	n	b
M_ROUNDNESS	Adds a roundness tolerance. The icon for this tolerance is: M_ROUNDNESS validates the roundness of one of the following features: arc, circle, or edgel. Aurora Imaging Library establishes a feature's roundness by calculating the distance between the inner and outer circular curves formed by the given feature. Note that the inner and outer curves are concentric. 	c		e	f	g	a		i		j	k	l	m	n	b
M_STRAIGHTNESS	Adds a straightness tolerance. The icon for this tolerance is: M_STRAIGHTNESS validates the straightness of one of the following features: segment or edgel. This is done by calculating the distance between the two parallel lines that are formed by the inner and outer boundaries of the feature. The angle of the two parallel lines is chosen such that the distance between them is the smallest. 	c		e	f	g	a		i		j	k	l	m	n	b

To modify a geometric tolerance, the ToleranceType parameter can be set to the following value. In this case, you must set the ToleranceLabel parameter to the label of an existing tolerance.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For modifying a geometric tolerance																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MODIFY	Modifies the features that the tolerance uses, and the tolerance's minimum and maximum limit values. You cannot modify only the features or only the limits. Use FeatureLabelArrayPtr, SubFeatureIndexArrayPtr, ValueMin, and ValueMax to specify the new features and limits.	c		e	f	g	a		i		j	k	l	m	n	b

ToleranceLabel

Specifies the label of the target tolerance.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For labeling the tolerance																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that Aurora Imaging Library automatically selects the tolerance's label value. Once a tolerance is added, you can inquire its label using MmetInquire() with the tolerance's index and M_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label value. Each tolerance must have a unique label otherwise you will get an error. You can have an extremely high number of labels (up to **M_MET_TOLERANCE_ID_MAX**).	c		e	f	g	a		i		j	k	l	m	n	b

ValueMin

Specifies the minimum value of the geometric tolerance.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the minimum geometric tolerance																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b

ValueMax

Specifies the maximum value of the geometric tolerance.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the maximum geometric tolerance																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b

FeatureLabelArrayPtr
Accepts the address of one of the following:

- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1)
  Required array size:(SizeOfArray)

Specifies the array that holds the label values of the features whose relationship to validate. The features must have already been added to the metrology template of the metrology context. To add a feature, use MmetAddFeature(). The specified features, whose relationship you want to validate, can be considered the base features of the tolerance.

SubFeatureIndexArrayPtr

Accepts the address of one of the following:

- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1)
  Required array size:(SizeOfArray)

Specifies the array that holds the index values of the multiple features' subfeatures whose relationship the added tolerance will validate. For non-multiple features (single features), specify 0 as the index. If the tolerance validates the relationship between only single features (there are no multiple features), set SubFeatureIndexArrayPtr to **M_NULL**.

SizeOfArray

Specifies the size of the arrays that hold the information that will be used to add the geometric tolerance (FeatureLabelArrayPtr and SubFeatureIndexArrayPtr).

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the FeatureLabelArrayPtr or SubFeatureIndexArrayPtr parameter.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetAlloc

Synopsis

Allocate a metrology context or a derived metrology region object.

Syntax

```
AIL_ID MmetAlloc(
    AIL_ID SysId,                  //in
    AIL_INT64 Type,                //in
    AIL_ID *ContextOrRegionMetIdPtr //out
)
```

Description

This function allocates a metrology context or a derived metrology region object on the specified system. A metrology context contains all the information necessary to perform an MmetCalculate() operation, including global processing settings and a metrology template; the metrology template defines the set of features and geometric tolerances against which MmetCalculate() validates and measures objects in an image. A derived metrology region object is an optional Aurora Imaging Library object that stores specialized information about metrology regions (ROIs).

When you allocate a metrology context, Aurora Imaging Library automatically creates a global frame, assigns it an index of 0, and labels it M_GLOBAL_FRAME. The global frame is the first coordinate system used to define the features to add to the metrology template of a metrology context. The global frame always exists and you cannot delete it. If the target image is not calibrated, the global frame's default origin (0,0) is aligned with the center of the top-left corner pixel of the target image. If the target image is calibrated, the global frame's default origin is aligned with the origin of the relative (world) coordinate system.

To add features (either physically measured or constructed) or geometric tolerances to the metrology template of a metrology context, use MmetAddFeature() or MmetAddTolerance(), respectively. To adjust metrology context, feature, and geometric tolerance settings, use MmetControl().

For physically measured features, you must also use MmetSetRegion() to set the metrology region that delimits the area in the target image from which to establish the feature. You can set the metrology region using explicit values or a 2D graphics list; you can also derive a metrology region using other features. If you are using an explicitly-defined or a 2D graphics list metrology region, you just need to allocate a metrology context, and specify it when you call MmetSetRegion(). If you are deriving a metrology region using other features, you must also allocate a derived metrology region object and specify it, as well as the metrology context, when you call MmetSetRegion().

When the metrology context or the derived metrology region object is no longer required, release it using MmetFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the metrology context or the derived metrology region object's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the metrology context or the derived metrology region object.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b

System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

Type

Specifies whether to allocate a metrology context or a derived metrology region object. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to allocate a metrology context or a derived metrology region object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Specifies a metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DERIVED_GEOMETRY_REGION	Specifies a derived metrology region object. This object is only required when a geometry setting of a metrology region must be derived from one or more features. This type of metrology region is referred to as a derived metrology region. Use MmetSetRegion() with M_FROM_DERIVED_GEOMETRY_REGION to associate the derived metrology region with a measured feature.	c		e	f	g	a		i		j	k	l	m	n	b

ContextOrRegionMetIdPtr

Specifies the address of the variable in which to write the metrology context or the derived metrology region object identifier or specifies the data type that the function should use to return the metrology context or the derived metrology region object identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated metrology context or the derived metrology region object ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated metrology context or the derived metrology region object ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_MET_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the metrology context or the derived metrology region object (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the metrology context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated metrology context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the metrology derived geometry region identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated metrology derived region object. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the metrology context or the derived metrology region object identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_MET_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetAllocResult

Synopsis

Allocate a metrology result buffer.

Syntax

```
AIL_ID MmetAllocResult(
    AIL_ID SysId,          //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ResultIdPtr    //out
)
```

Description

This function allocates a metrology result buffer, on the specified system, to store results obtained from an MmetCalculate() operation.

When the metrology result buffer is no longer required, release it using MmetFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the metrology result buffer's lifetime and you must not manually free it.

Parameters

SysId

Specifies the system on which to allocate the metrology result buffer.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ResultIdPtr

Specifies the address of the variable in which to write the metrology result buffer identifier or specifies the data type that the function should use to return the metrology result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated metrology result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated metrology result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_MET_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the metrology result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated metrology result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the metrology result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_MET_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetCalculate

Synopsis

Calculate features and validate geometric tolerances.

Syntax

```
void MmetCalculate(
    AIL_ID ContextId,     //in
    AIL_ID TargetBufId,   //in
    AIL_ID ResultId,      //out
    AIL_INT64 ControlFlag //in
)
```

Description

This function calculates features and validates geometric tolerances. For physically measured features, **MmetCalculate**() uses the target image's edge map. For more information on edge maps, see the Extracting the edges section of Chapter 10: Edge Finder.

If a metrology context contains only constructed features, the TargetBufId parameter can be set to **M_NULL**. This is useful, for example, when performing calculations on edgel features constructed from edgels or points established using other Aurora Imaging Library modules or third party software.

To control the Metrology module's calculation and validation behavior, you can set context constraints using MmetControl(). You can also use MmetControl() to alter settings for features and tolerances that you have already added to the context. To add features to the context, use MmetAddFeature(). To add geometric tolerances to the context and define geometric relationships between features, use MmetAddTolerance().

Results of a metrology calculation are stored in the specified result buffer and can be obtained using MmetGetResult().

If the target image is not calibrated, results are calculated in the pixel coordinate system (pixel units). If the target image is calibrated, results are calculated in the world coordinate system (real-world units), but they can be retrieved in either world or pixel units. Note that, if the target image is calibrated, feature values, metrology regions, and tolerances are considered as real-world values and are retrieved in world units.

Note that in the presence of distortion, some results are meaningless when converted from real-world to pixel units (for example, angle and scale results). For example, if a feature appears warped in the target image, but the camera calibration context compensates for this during the calculation, the resulting angle is meaningful in the real-world coordinate system, and meaningless in the pixel coordinate system.

If complex distortions exist, you must first correct the image before using it with the Metrology module.

Parameters

ContextId

Specifies the identifier of the metrology context. The metrology context must have been previously allocated on the required system using MmetAlloc().

TargetBufId

Specifies the identifier of the target image buffer from which to extract edges to calculate physically measured features and validate geometric tolerances. If a metrology context contains features which are only constructed, you can set this parameter to **M_NULL**. If you don't provide a target image and the context contains any measured features, an error is generated.

Typically, the target image buffer should be 1-band 8-bit unsigned. Other buffer depths and types are generally accepted, but can slightly decrease performance. When the target image buffer is 32-bit float, metrology uses floating-point precision calculations.

This image buffer must not have a region of interest (ROI) associated with it (MbufSetRegion()). Using an image buffer with an ROI will cause an error. You can only use metrology regions established with MmetSetRegion().

ResultId

Specifies the metrology result buffer in which to write the results of the calculation. The metrology result buffer must have been previously allocated on the required system using MmetAllocResult().

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetControl

Synopsis

Control a setting for a metrology context, a derived metrology region object, features, tolerances, or a result buffer.

Syntax

```
void MmetControl(
    AIL_ID MetId,           //out
    AIL_INT LabelOrIndex,   //in
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function sets the specified control for a metrology context, a derived metrology region object, features, tolerances, or a result buffer. These settings control the execution of MmetCalculate(), and affect how results are retrieved by MmetGetResult(). You can also use this function to specify whether features can be drawn with MmetDraw(). Most of the control type settings in **MmetControl**() can be inquired using MmetInquire().

If a camera calibration context is associated with the target image, set values in real-world units (for example, tolerance values, warning values, and positional values). Otherwise, use pixel units.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MmetControl()** internally calls the *AIL_DOUBLE* version of this function (MmetControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MmetControlInt64()).

Parameters

MetId

Specifies the identifier of the metrology context, the derived metrology region object, or the result buffer whose settings you want to control. The metrology context or the derived metrology region object must have been previously allocated on the required system using MmetAlloc(). The metrology result buffer must have been previously allocated on the required system using MmetAllocResult().

LabelOrIndex

Specifies to control the metrology context, the derived metrology region object, features, tolerances, or the result buffer. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to control																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_INDEX(AIL_INT FeatureIndex)	Specifies the index value of an existing individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

	FeatureIndex This parameter specifies the index of the individual feature to which the control settings will be applied. You can set this parameter to the following:														
	Value >= 0 — Specifies the index of the individual feature. Note that the index of the global frame is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL(AIL_INT FeatureLabel)	Specifies the label value of an existing individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	FeatureLabel This parameter specifies the label of the individual feature to which the control settings will be applied. You can set this parameter to the following:														
	Value > 0 — Specifies the label of the individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_INDEX(AIL_INT ToleranceIndex)	Specifies the index value of an existing individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	ToleranceIndex This parameter specifies the index of the individual tolerance to which the control settings will be applied. You can set this parameter to the following:														
	Value >= 0 — Specifies the index of the individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_LABEL(AIL_INT ToleranceLabel)	Specifies the label value of an existing individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b

	Parameters															
	ToleranceLabel This parameter specifies the label of the individual tolerance to which the control settings will be applied. You can set this parameter to the following:															
	Value > 0 — Specifies the label of the individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_FEATURES	Applies the specified control setting to all features. This value should only be used with M_DELETE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_TOLERANCES	Applies the specified control setting to all tolerances.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Controls a setting of the metrology context, which has been set using the MetId parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_DERIVED_GEOMETRY_REGION	Controls a setting of the derived metrology region object, which has been set using the MetId parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Controls a setting of a metrology result buffer. The setting is applied to all feature and tolerance results in the metrology result buffer. When using M_GENERAL, MetId must specify a metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_FRAME	Applies the specified control setting to the global frame of the context. In this case, MetId must specify a metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEASURED_FEATURES	Applies the specified control setting to all measured features. In this case, MetId must specify a metrology context.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the control.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For controlling general context settings

- For controlling global processing settings
- For controlling features
- For controlling constructed edgel features
- For setting the geometry of the derived metrology region
- For setting the geometry data of the derived metrology region
- For controlling geometric tolerances
- For controlling features or geometric tolerances
- For controlling clone transformations
- For controlling results
- For deleting features and tolerances

The following ControlType and corresponding ControlValue parameter settings are used to control general metrology context settings. For controlling general context settings, the LabelOrIndex parameter must be set to M_CONTEXT and the MetId parameter must be set to a metrology context.

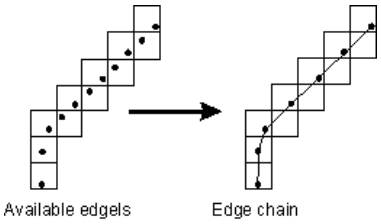
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

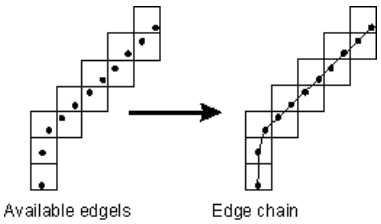
For controlling general context settings																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_ASSOCIATED_CALIBRATION		Associates the specified camera calibration context with the template reference of the metrology context. To specify the template reference, use **MmetControl**() with M_TEMPLATE_REFERENCE_ID. When saving the metrology context, if you do not save the camera calibration context associated with the template reference (MmetSave() or MmetStream() with M_WITH_CALIBRATION), you must re-associate the template reference with its camera calibration context after restoring the context (MmetRestore() or MmetStream()). Note that you can also associate a camera calibration context to the template reference using other Aurora Imaging Library mechanisms (Mcal... ()); in this case, M_ASSOCIATED_CALIBRATION is ignored. You cannot associate the template reference with a calibrated context whose Y-axis has been inverted using McalGrid() with M_Y_AXIS_COUNTER_CLOCKWISE. INQ															
	M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Removes the association to the camera calibration.	c		e	f	g	a		i		j	k	l	m	n	b
	Calibration context identifier	Specifies the identifier of the camera calibration context to associate with the template reference of the metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_REFERENCE_ID		Associates a template reference to the context. The template reference is a companion buffer (typically an image) useful for training the metrology context and drawing the results on a typical target image. Note this control type creates an internal copy of the buffer associated with the metrology context. Freeing the buffer passed to **MmetControl**() (using MbufFree()) will not effect this internal copy. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_NULL	Releases the template reference buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Image identifier	Specifies the identifier of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT		Sets the maximum measurement and validation time for MmetCalculate(), in msec. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies an infinite amount of measurement and validation time.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum measurement and validation time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control processing settings for a measured feature. For controlling processing settings, the LabelOrIndex parameter can be set to M_MEASURED_FEATURES or an existing measured feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX(). The MetId parameter must be set to a metrology context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling global processing settings																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CHAIN_ALL_NEIGHBORS		Sets whether edge chains are built with as much edgel information as possible. An edge chain refers to the set of connected edgels that form an edge. Edge chains are built using an 8-connected lattice. INQ															
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that edge chains are built with the least amount of edgel information possible. 	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies that edge chains are built with as much edgel information as possible.  Enabling M_CHAIN_ALL_NEIGHBORS can result in greater accuracy, since edge chains can contain more information, though calculations can take slightly longer.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_SCALE	Sets the scale of the image at which to do the edge extraction. Once the extraction is complete, the results are scaled to the original scale of the image. An extraction scale less than one can speed calculations but can be less reliable. The default extraction scale setting is typically sufficient. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the extraction scale.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_GAP_ANGLE	Sets the aperture angle where the feature extraction searches for edgel extremity candidates when filling gaps in features. That is, M_FILL_GAP_ANGLE, along with M_FILL_GAP_DISTANCE, define a region where two chain extremities can be linked. Note that two chain extremities can only be linked if both are included in the search region of the other. For more information on filling gaps, see the Filling the edge gaps subsection of the Customizing the edge extraction settings section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 90.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the aperture angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_GAP_DISTANCE	Sets the maximum distance radius where the feature extraction searches for edgel extremity candidates when filling gaps in features. That is, M_FILL_GAP_DISTANCE, along with M_FILL_GAP_ANGLE, define a region where two chain extremities can be linked. Note that two chain extremities can only be linked if both are included in the search region of the other. For more information on filling gaps, see the Filling the edge gaps subsection of the Customizing the edge extraction settings section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value. Typically, the default value is 0.0 pixels, however for an M_MEASURED M_POINT feature, the default value is 3.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies an infinite maximum distance radius.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum distance radius, in pixels. Note that generally, you should not set the distance to a large value. Typically, you would set M_FILL_GAP_DISTANCE to a value in the range of 2 to 5 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_SMOOTHNESS		Sets the degree of smoothness (strength of denoising) applied by the edge extraction filter during the neighborhood operation. The smoothing operation evens out rough edges and reduces noise; in effect, the smoothness factor affects which edges are extracted. For more information on smoothing edges, see the Smoothing for IIR filters subsection of the Customizing the edge extraction settings section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the smoothness value. A value of 100.0 results in a strong noise reduction effect, while a value of 0.0 has almost no noise reduction effect.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_TYPE		Sets the type of filter used to extract edges. The type of filter determines the distribution of the neighborhood's influence. It establishes the contribution of each neighboring pixel to the edge calculation. For more information on filtering, see the Filter type subsection of the Customizing the edge extraction settings section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SHEN.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DERICHE	Specifies a Deriche infinite support filter.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FREI_CHEN	Specifies a Frei Chen filter.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PREWITT	Specifies a Prewitt filter.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SHEN	Specifies a Shen-Castan infinite support exponential filter.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SOBEL	Specifies a Sobel filter.	c		e	f	g	a		i		j	k	l	m	n	b

M_FLOAT_MODE	Sets whether to perform all edge extraction operations using floating-point precision calculations. For more information on using floating-point precision when extracting edges, see the Calculating and retrieving results section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that all edge extractions are not forced to be performed using floating-point precision calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that all edge extractions are forced to be performed using floating-point precision calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAGNITUDE_TYPE	Sets how to calculate the magnitude of the edge at each edgel position. For more information on the magnitude, see the Magnitude type subsection of the Customizing the edge extraction settings section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SQR_NORM.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORM	Specifies that the magnitude will be used.	c		e	f	g	a		i		j	k	l	m	n	b
M_SQR_NORM	Specifies that the square of the magnitude will be used.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ACCURACY_HIGH	Sets whether to use high accuracy to define the metrology region associated with a feature, when dealing with a calibrated image. When using high accuracy, the metrology region closely follows the image distortion; when using low accuracy, the Metrology module approximates the distortion. For example, a rectangular metrology region remains a rectangle and a circular metrology region becomes the best approximated elliptic region. M_REGION_ACCURACY_HIGH also applies when drawing the metrology region, using MmetDraw() with Operation set to M_DRAW_REGION. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that high accuracy is not used when defining a metrology region. This value is recommended when the image distortion of the calibrated image is minor. This typically results in a quicker processing time.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that high accuracy is used when defining a metrology region.	c		e	f	g	a		i		j	k	l	m	n	b

M_THRESHOLD_MODE	Sets the threshold of the edge extraction. For more information on thresholding edges, see the Thresholding subsection of the Customizing the edge extraction settings section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_HIGH.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies no threshold. All edges are extracted.	c		e	f	g	a		i		j	k	l	m	n	b
M_HIGH	Specifies a high threshold. M_HIGH should be used for images with some contrast variations, noise, and non-uniform illumination. M_HIGH always results in a lower (or equal) number of edgels than M_MEDIUM.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOW	Specifies a low threshold. All edges over a minimum noise-based estimated threshold are extracted. M_LOW always results in a lower (or equal) number of edgels than M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIUM	Specifies a medium threshold. M_MEDIUM should be used for multi-contrast images, or for images with a lot of noise or non-uniform illumination. M_MEDIUM always results in a lower (or equal) number of edgels than M_LOW.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies that the threshold values will be user-defined. Set the threshold values with M_THRESHOLD_VALUE_LOW and M_THRESHOLD_VALUE_HIGH.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERY_HIGH	Specifies a very high threshold. Only the strongest edges in the image are extracted. M_VERY_HIGH always results in a lower (or equal) number of edgels than M_HIGH.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_TYPE	Sets the type of hysteresis threshold used when performing the edge extraction. Edge chains are built such that the magnitude values of all connected edgels are stronger than a lower bound threshold value and such that at least one of the edgel's magnitude in each edge chain is stronger than a upper bound threshold value. Note that M_THRESHOLD_TYPE sets how to use the lower and upper threshold bounds, while M_THRESHOLD_MODE defines what the bounds are. For more information on thresholding, see the Thresholding subsection of the Customizing the edge extraction settings section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_HYSTERESIS.	c		e	f	g	a		i		j	k	l	m	n	b
M_FULL_HYSTERESIS	Specifies that the lower bound threshold value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

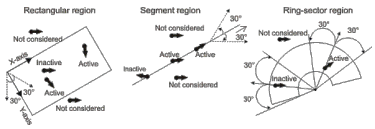
M_HYSTERESIS	Specifies that both the lower bound threshold value and the upper bound threshold value will be used.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_HYSTERESIS	Specifies that the lower bound threshold value is equal to the upper bound threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE_HIGH	Sets the upper bound of the hysteresis threshold value. M_THRESHOLD_VALUE_HIGH is ignored unless **MmetControl**() with M_THRESHOLD_MODE is set to M_USER_DEFINED. Note that lower threshold values result in a more sensitive edgel detection; that is, a higher (or equal) number of edgels are detected. For more information on thresholding, see the Thresholding subsection of the Customizing the edge extraction settings section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the upper bound of the hysteresis threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE_LOW	Sets the lower bound of the hysteresis threshold value. M_THRESHOLD_VALUE_LOW is ignored unless **MmetControl**() with M_THRESHOLD_MODE is set to M_USER_DEFINED. Note that lower threshold values result in a more sensitive edgel detection; that is, a higher (or equal) number of edgels are detected. For more information on thresholding, see the Thresholding subsection of the Customizing the edge extraction settings section of Chapter 10: Edge Finder. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the lower bound of the hysteresis threshold.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control how the features are calculated. For controlling features, the LabelOrIndex parameter can be set to the following: M_GLOBAL_FRAME or an existing feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX(). The MetId parameter must be set to a metrology context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

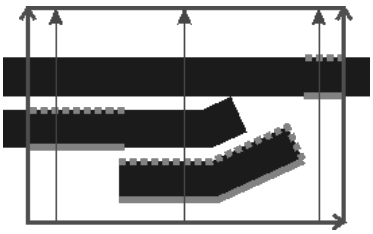
For controlling features																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_ACTIVE	Sets whether to enable or disable the feature for calculation. INQ															

	M_DISABLE	Specifies not to calculate the feature. In this case, the feature is not calculated and the status of the feature (M_STATUS) will return M_FAIL. This is also the case for the status of all the derived features and tolerances, if any.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the feature. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE_END		Sets the end angle of a constructed arc. The angle is relative to the reference frame's X-axis and follows the counter-clockwise convention. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 360.0°.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE_START		Sets the start angle of a constructed arc. The angle is relative to the reference frame's X-axis and follows the counter-clockwise convention. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* < 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEL_ANGLE_RANGE	Sets the angular range within which an edgel's gradient angle must fall for Aurora Imaging Library to consider it an active edgel. Active edgels apply to physically measured features and constructed features based on physically measured edgels or based on external edgels. Note that this control type is ignored for points (edgels with no angle information) in an external edgel feature. By default, Aurora Imaging Library measures the specified angular range, in the counter clockwise direction, relative to the metrology ROI's orientation. To change the relative angle from which to measure the angular range, use M_EDGEL_RELATIVE_ANGLE. In the following examples, the angular range is 60.0° and the relative angle is the same as the region's orientation. Note that for the ring-sector region, the orientation is radial.  Note that constructed edgel features are not built from a region, but from a specified measured feature. Therefore in this case, the region's orientation refers to the orientation of the region from which its specified measured feature was built. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 180.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angular range, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_DENOISING_MODE	Sets how to denoise edgels. Denoising refers to establishing and eliminating noisy (unwanted) edgels. This setting applies to all physically measured features and constructed edgel features. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies no denoising.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAUSSIAN	Specifies a Gaussian type of denoising.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN	Specifies a mean type of denoising.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIAN	Specifies a median type of denoising.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEL_DENOISING_RADIUS	Sets the radius by which to denoise edgels. This setting only has an effect if M_EDGEL_DENOISING_MODE is set to a value other than its default (which is M_NULL). This setting applies to all physically measured features and constructed edgel features. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the radius, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_DENOISING_USE_ORDER	Sets whether Aurora Imaging Library should use and maintain the order of the edgels when denoising them. This setting only has an effect if M_EDGEL_DENOISING_MODE is set to a value other than its default (which is M_NULL). This setting applies to all physically measured features and constructed edgel features. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that Aurora Imaging Library should not use and maintain the order of the edgels when denoising them.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that Aurora Imaging Library should use and maintain the order of the edgels when denoising them. This can improve the speed and accuracy of the denoising. This also ensures that the denoising itself does not reorder the edgels; keeping the original order can be useful for future operations. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEL_RELATIVE_ANGLE	Sets the relative angle from which to measure the gradient angle range (M_EDGEL_ANGLE_RANGE). You can use M_EDGEL_RELATIVE_ANGLE, along with M_EDGEL_ANGLE_RANGE, to select the active edgels of a physically measured feature or the active edgels of a constructed feature based on physically measured edgels or based on external edgels. Typically, the relative angle is the same as the region's orientation. For an example on how M_EDGEL_RELATIVE_ANGLE is applied, along with the gradient angle range, see M_EDGEL_ANGLE_RANGE. Note that this control type is ignored for points (edgels with no angle information) in an external edgel feature. Note that constructed edgel features are not built from a region, but from a specified measured feature. Therefore in this case, the region's orientation refers to the orientation of the region from which its specified measured feature was built. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_REVERSE	Specifies that the gradient angle range is measured relative to the reverse angle (orientation) of the region of interest (+ 180°).	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME	Specifies that the gradient angle range is measured relative to the same angle (orientation) as the region of interest.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME_OR_REVERSE	Specifies that the gradient angle range is measured relative to either the same or the reverse angle (orientation) of the region of interest.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEL_SELECTION_RANK	Sets the edgels to select relative to the orientation of the metrology region. The metrology region is scanned column by column, and the specified edgel encountered within a column is selected. Note that this control type takes effect after M_EDGEL_RELATIVE_ANGLE and M_EDGEL_ANGLE_RANGE are satisfied. M_EDGEL_SELECTION_RANK applies to M_RECTANGLE derived metrology regions. In the following example, there are multiple edges in the metrology region. The solid gray line illustrates the edgels selected when M_EDGEL_SELECTION_RANK is set to 1; the dotted gray line illustrates the edgels selected when M_EDGEL_SELECTION_RANK is set to 2. INQ 	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that all edgels are selected.	c		e	f	g	a		i		j	k	l	m	n	b
M_LAST	Selects the last edgel in each column, relative to the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies which edgels to select, based on their rank, relative to the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_TYPE	Sets the type of edgels to use when building constructed edgel features from a measured base feature using MmetAddFeature() with M_COPY_FEATURE_EDGELS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACTIVE_EDGELS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACTIVE_EDGELS	Specifies to use the active edgels of the measured feature's metrology region. Active edgels must satisfy all edgel constraints (M_EDGEL_ANGLE_RANGE and M_EDGEL_RELATIVE_ANGLE).	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_EDGELS	Specifies to use all the edgels of the measured feature's metrology region.	c		e	f	g	a		i		j	k	l	m	n	b

M_FITTED_EDGELS	Specifies to use the fitted edgels of the measured feature's metrology region. The fitted edgels are those active edgels used by the fit operation to define a measured feature. Fitted edgels must satisfy all edgel constraints (M_EDGEL_ANGLE_RANGE and M_EDGEL_RELATIVE_ANGLE) and fit constraints (M_FIT_...).	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_COVERAGE_MIN	Sets the approximate portion of the feature that must be covered by fitted edgels. In the following example, about 50.0% of the segment feature covers the edgels. The actual coverage that results from this setting depends on the scale of the image at which the feature edges were extracted, as specified with M_EXTRACTION_SCALE; a scale of 1 provides the best estimation. M_FIT_COVERAGE_MIN is valid for all fit operations for physically measured or constructed features. For constructed features using a fit operation performed on multiple physically measured edgel features, the accuracy of the minimum fit coverage is based on the average of each feature's extraction scale. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the minimum coverage, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_DISTANCE_MAX	Sets the greatest possible gap between an active edgel and an iterative fit for the edgel to be considered during the next iteration. The higher the value, the farther away an active edgel can be. M_FIT_DISTANCE_MAX is valid for all fit operations for physically measured or constructed features. You can specify M_FIT_DISTANCE_MAX when M_FIT_TYPE is set to M_STANDARD_FIT. This setting can alter the precision of the edge that is ultimately fit; however, it might not reliably discard outliers (outlier edgels/points) when they significantly impact the initial fit (the first iteration of the fit). When dealing with a lot of outliers, use a robust best fit (M_ROBUST_FIT). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_INFINITE	Specifies no maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the maximum distance, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_DISTANCE_OUTLIERS		Sets whether to automatically determine the distance that establishes outliers (edgels/points), or to use a user-defined distance. Outliers are excluded from the robust best fit. This setting only applies to features built using MmetAddFeature() with M_FIT; also, M_FIT_TYPE must be set to M_ROBUST_FIT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies that Aurora Imaging Library automatically determines the distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies that the distance is set with M_FIT_DISTANCE_OUTLIERS_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_DISTANCE_OUTLIERS_VALUE		Sets an explicit distance value that Aurora Imaging Library uses to establish outliers (outlier edgels/points), which are excluded from the robust best fit. This only applies to features built using MmetAddFeature() with M_FIT; also, M_FIT_TYPE must be set to M_ROBUST_FIT, and M_FIT_DISTANCE_OUTLIERS must be set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 4.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the distance; value must be greater than 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_INTERNAL_NUMBER_OF_POINTS		Sets the minimum number of points from which Aurora Imaging Library performs the robust fit to build the feature. This only applies to features built using MmetAddFeature() with M_FIT; also, M_FIT_TYPE must be set to M_ROBUST_FIT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies that Aurora Imaging Library automatically determines the number of points. For a segment, Aurora Imaging Library uses 2 points. For a circle or arc, Aurora Imaging Library uses 3.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the number of points. Less points typically result in faster fit calculations. For a segment, there is a 2 point minimum, and for a circle or arc, there is a 3 point minimum.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_ITERATIONS_MAX	Sets the maximum number of fit iterations used to compute the feature. The more iterations, the better the fit, but the slower the calculation. M_FIT_ITERATIONS_MAX is valid for all fit operations for physically measured or constructed features. This setting can alter the precision of the edge that is ultimately fit; however, it might not reliably discard outliers (outlier edgels/points) when they significantly impact the initial fit (the first iteration of the fit). When dealing with a lot of outliers, use a robust best fit (M_ROBUST_FIT). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the maximum number of fit iterations will be automatically determined.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the maximum number of fit iterations. Only integer values are accepted. For an M_STANDARD_FIT, a setting of one will consider all edgels/points in the fit. Settings higher than one will progressively eliminate outlying edgels (outlier edgels/points) in the fit.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_TYPE	Sets the type of best fit with which to build the feature. This only applies to features built using MmetAddFeature() with M_FIT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_STANDARD_FIT.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROBUST_FIT	Specifies a robust best fit. This can be effective when dealing with outliers (outlier edgels/points). When you use a robust best fit, you can further control how the feature is built, by specifying M_FIT_DISTANCE_OUTLIERS, M_FIT_DISTANCE_OUTLIERS_VALUE, M_FIT_INTERNAL_NUMBER_OF_POINTS, and M_FIT_ITERATIONS_MAX. This is not a valid control value for features of type M_POINT or M_EDGEL.	c		e	f	g	a		i		j	k	l	m	n	b

	M_STANDARD_FIT	Specifies a standard best fit. In this case, more importance is typically given to the edgels whose gradient orientation follows the region's orientation. However, when adding a segment feature in an M_INFINITE metrology region with an M_RADIAL orientation, or in an M_RING_SECTOR metrology region, more importance is given to the edgels whose gradient orientation is more perpendicular to the radial direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_VARIATION_MAX		Sets the maximum allowable difference in the value of the feature's underlying coefficients, from one fit iteration to the next. If at least one coefficient of the feature varies more, between two consecutive fit iterations, than the specified value, the iteration process will continue. Note that: • An arc feature has 5 coefficients: the X-coordinate of its center, the Y-coordinate of its center, its radius, its start angle, and its end angle. • A circle feature has 3 coefficients: the X-coordinate of its center, the Y-coordinate of its center and its radius. • A segment feature has 4 coefficients: the X-coordinate of its start point, the Y-coordinate of its start point, the X-coordinate of its end point, and the Y-coordinate of its end point. These coefficients are the same as those required when building the corresponding features with a parametric operation. M_FIT_VARIATION_MAX is valid for all fit operations for physically measured or constructed features. You can specify M_FIT_VARIATION_MAX when M_FIT_TYPE is set to M_STANDARD_FIT. This setting can alter the precision of the edge that is ultimately fit; however, it might not reliably discard outliers (outlier edgels/points) when they significantly impact the initial fit (the first iteration of the fit). When dealing with a lot of outliers, use a robust best fit (M_ROBUST_FIT). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies that the maximum variation will be determined automatically.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the maximum variation, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

M_LINE_A	Sets the coefficient *A* of the line equation for a parametrically constructed line. The line equation is: $A\,x + B\,y + C = 0$. If you use the default for all coefficients (*A* = 0, *B* = 1, and *C* = 0), Aurora Imaging Library constructs a line that runs along the X-axis of the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the coefficient's value.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_B	Sets the coefficient *B* of the line equation for a parametrically constructed line. The line equation is: $A\,x + B\,y + C = 0$. If you use the default for all coefficients (*A* = 0, *B* = 1, and *C* = 0), Aurora Imaging Library constructs a line that runs along the X-axis of the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the coefficient's value.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_BISECTOR_TYPE	Sets how to construct a line feature when building it as a bisector. This only applies when using MmetAddFeature() with M_CONSTRUCTED set to M_LINE and M_BISECTOR, and specifying two lines as the base features. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FEATURE_ORDER.	c		e	f	g	a		i		j	k	l	m	n	b
M_BIGGEST_ANGLE	Specifies that the newly constructed line is built at the middle of the biggest angle formed by the two lines specified as base features.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_ORDER	Specifies that the newly constructed line is built according to the order in which you specified the two line base features; that is, at the middle of the angle formed between the first and second specified lines.	c		e	f	g	a		i		j	k	l	m	n	b
M_SMALLEST_ANGLE	Specifies that the newly constructed line is built at the middle of the smallest angle formed by the two lines specified as base features.	c		e	f	g	a		i		j	k	l	m	n	b

M_LINE_C	Sets the coefficient *C* of the line equation for a parametrically constructed line. The line equation is: $A\,x + B\,y + C = 0$. If you use the default for all coefficients (*A* = 0, *B* = 1, and *C* = 0), Aurora Imaging Library constructs a line that runs along the X-axis of the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
-0.5 * *AIL_INT32_MAX* <= *Value* <= 0.5 * *AIL_INT32_MAX*	Specifies the coefficient's value. > If the value that you need to specify for C is out of range, you can multiply A, B, and C by (0.5 **AIL_INT32_MAX*) / *C* to normalize the coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_MAX	Sets the maximum number of subfeatures to calculate for a multiple feature. Note that M_NUMBER_MAX is valid for point features only. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum number of subfeatures. This value must be greater than 1 for measured multiple point features.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_MIN	Sets the minimum number of subfeatures to calculate for a multiple feature. Note that M_NUMBER_MIN is valid for point features only. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the minimum number of subfeatures. This value must be greater than 1 for measured multiple point features.	c		e	f	g	a		i		j	k	l	m	n	b
M_OCCURRENCE	Sets the point to use if there is more than one candidate when adding a constructed point feature built at the intersection of two base features (MmetAddFeature() with M_POINT and M_INTERSECTION or M_EXTENDED_INTERSECTION). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0	Specifies the index value.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION	Sets the position, along the contour of the specified base feature, at which to construct the point feature (MmetAddFeature() with M_POINT and M_POSITION_ABSOLUTE or M_POSITION_RELATIVE). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the position value. Set the position as a linear value when using M_POSITION_ABSOLUTE and as a percentage when using M_POSITION_RELATIVE.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_END_X	Sets the X-coordinate of the end point of a segment constructed using MmetAddFeature() with M_PARAMETRIC. If you use the default for all positions (M_POSITION_START_X = -1.0, M_POSITION_START_Y = 0.0, M_POSITION_END_X = 1.0, and M_POSITION_END_Y = 0), Aurora Imaging Library constructs a segment with a length of 2 (-1.0 to 1.0); the segment is centered on the origin of the reference frame and runs along the reference frame's X-axis. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_END_Y	Sets the Y-coordinate of the end point of a segment constructed using MmetAddFeature() with M_PARAMETRIC. If you use the default for all positions (M_POSITION_START_X = -1.0, M_POSITION_START_Y = 0.0, M_POSITION_END_X = 1.0, and M_POSITION_END_Y = 0), Aurora Imaging Library constructs a segment with a length of 2 (-1.0 to 1.0); the segment is centered on the origin of the reference frame and runs along the reference frame's X-axis. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_START_X		Sets the X-coordinate of the start point of a segment constructed using MmetAddFeature() with M_PARAMETRIC. If you use the default for all positions (M_POSITION_START_X = -1.0, M_POSITION_START_Y = 0.0, M_POSITION_END_X = 1.0, and M_POSITION_END_Y = 0), Aurora Imaging Library constructs a segment with a length of 2 (-1.0 to 1.0); the segment is centered on the origin of the reference frame and runs along the reference frame's X-axis. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is -1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_START_Y		Sets the Y-coordinate of the start point of a segment constructed using MmetAddFeature() with M_PARAMETRIC. If you use the default for all positions (M_POSITION_START_X = -1.0, M_POSITION_START_Y = 0.0, M_POSITION_END_X = 1.0, and M_POSITION_END_Y = 0), Aurora Imaging Library constructs a segment with a length of 2 (-1.0 to 1.0); the segment is centered on the origin of the reference frame and runs along the reference frame's X-axis. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X		Sets the X-coordinate of the center of a constructed circle or arc, or sets the X-coordinate of a constructed local frame or point. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b

M_POSITION_Y	Sets the Y-coordinate of the center of a constructed circle or arc, or sets the Y-coordinate of a constructed local frame or point. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_RADIUS	Sets the radius of a constructed circle or arc. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the radius, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFERENCE_FRAME	Sets the feature's reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that the reference frame is the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT	Sets the sorting order that Aurora Imaging Library applies to the index of subpoints, when retrieving results of multiple point features. A multiple point feature is a point feature that contains more than one point. By default, the subpoints (points within the point feature) are sorted in ascending order, from the subpoint with the lowest index to the subpoint with the highest index. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SORT_UP.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_DOWN	Specifies that results of multiple point features are sorted in descending order, according to the subpoint's index.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_UP	Specifies that results of multiple point features are sorted in ascending order, according to the subpoint's index.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control constructed edgel features. In this case, the LabelOrIndex parameter must be set to the label or index of the feature, using M_FEATURE_LABEL() or M_FEATURE_INDEX(), and the MetId parameter must be set to a metrology context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling constructed edgel features																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																	
M_EDGEL_PROVIDED_ORDER		Sets whether the egdels/points used to construct the edgel feature are listed in a meaningful order. This setting applies to edgel features that are constructed using externally established edgels/points (MmetAddFeature() with M_EXTERNAL_FEATURE), which can come from other Aurora Imaging Library modules or third party software. Such edgels/points are specified using one or more arrays in MmetPut(). The order in which the edgels/points are specified in the arrays is considered meaningfully when, for example, they are listed according to their orientation (such as when they come from a laser line profile) or according to a coherent path (such as when they follow each other in a circular shape). Specifying ordered edgels/points can be useful when, for example, denoising or resampling edgels. INQ																
	M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies unordered edgels/points.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEQUENTIAL	Specifies ordered edgels/points. In this case, Aurora Imaging Library maintains the order in which the edgels/points were specified using MmetPut().	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_RESAMPLING_LINE_ANGLE		Sets the angle of a line along which to resample edgels. For example, you can resample along a laser line. This setting only has an effect if M_EDGEL_RESAMPLING_MODE is set to a value other than its default (which is M_NULL). This setting applies to constructed edgel features that are built by copying the edgels of the specified base features (MmetAddFeature() with M_COPY_FEATURE_EDGELS). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable resampling along a line. In this case, the resampling that Aurora Imaging Library performs (and subsequent operations) can be slower, as more internal computations are required.	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0.0	Specifies the angle of the line along which to resample edgels. The angle is relative to the reference frame's X-axis and follows the counter-clockwise convention. If you specify an angle of 0.0, Aurora Imaging Library resamples the edgels along a line at that angle. If you want Aurora Imaging Library to resample without using a line, set M_EDGEL_RESAMPLING_LINE_ANGLE to M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_RESAMPLING_MODE		Sets how to resample edgels. Resampling refers to using internal estimation techniques (mean or median) to modify and improve the edgels that Aurora Imaging Library uses. This setting applies to constructed edgel features that are built by copying the edgels of the specified base features (MmetAddFeature() with M_COPY_FEATURE_EDGELS). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies no resampling.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEAN	Specifies a mean type of resampling.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIAN	Specifies a median type of resampling.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_RESAMPLING_RADIUS		Sets the radius by which to resample edgels. This setting only has an effect if M_EDGEL_RESAMPLING_MODE is set to a value other than its default (which is M_NULL). This setting applies to constructed edgel features that are built by copying the edgels of the specified base features (MmetAddFeature() with M_COPY_FEATURE_EDGELS). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the radius.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_RESAMPLING_USE_ORDER		Sets whether Aurora Imaging Library should use and maintain the order of the edgels when resampling them. This setting only has an effect if M_EDGEL_RESAMPLING_MODE is set to a value other than its default (which is M_NULL). This setting applies to constructed edgel features that are built by copying the edgels of the specified base features (MmetAddFeature() with M_COPY_FEATURE_EDGELS). INQ	c		e	f	g	a		i		j	k	l	m	n	b

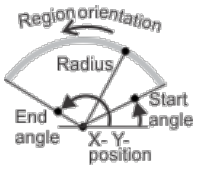
	M_DISABLE	Specifies that Aurora Imaging Library should not use and maintain the order of the edgels then resampling them.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that Aurora Imaging Library should use and maintain the order of the edgels when resampling them. This can improve the speed and accuracy of the resampling. This also ensures that the resampling itself does not reorder the edgels; keeping the original order can be useful for future operations. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUP_EDGEL_GAP_SIZE		Sets the maximum allowable gap (distance) between edgels for Aurora Imaging Library to consider them part of the same group. This setting applies to constructed edgel features built by calling MmetAddFeature() with M_GROUP_EDGELS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies that Aurora Imaging Library should automatically determine the gap size.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the gap size, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUP_MIN_NUMBER_OF_EDGELS		Sets the minimum number of edgels a group must have. Aurora Imaging Library removes all groups that do not meet this minimum number. This setting applies to constructed edgel features built by calling MmetAddFeature() with M_GROUP_EDGELS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies that there is no minimum number of edgels that a group must have.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the minimum number of edgels that a group must have.	c		e	f	g	a		i		j	k	l	m	n	b

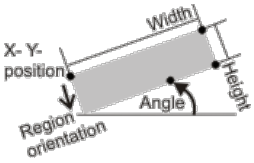
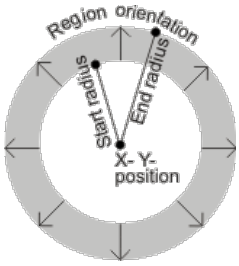
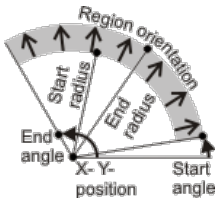
M_GROUP_SELECTION_MODE		Sets how to group edgels. This setting applies to constructed edgel features built by calling MmetAddFeature() with M_GROUP_EDGELS. Aurora Imaging Library initially establishes the groups based on the gaps (distance) between the edgels. By default, Aurora Imaging Library automatically determines the gap size to group edgels. Edgels that are closer to each other than the gap size are considered part of the same group. To explicitly specify the gap size, use M_GROUP_EDGEL_GAP_SIZE. Aurora Imaging Library then eliminates any potential group, if it does not have the minimum number of edgels, which you can set with M_GROUP_MIN_NUMBER_OF_EDGELS (by default, there is no minimum requirement). Once these basic groups are formed, you can either select one group or all groups using M_GROUP_SELECTION_MODE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_OCCURRENCE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CLOSEST_TO_POINT	Specifies to select one group, based on its proximity to a point or the local frame's origin. By default, Aurora Imaging Library selects the group (from the initially established groups) that is closest to the global frame's origin. To select the group that is closest to a different point, use M_GROUP_SELECTION_POINT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OCCURRENCE	Specifies to select one or all group(s), based on the index of the group of edgels. By default, Aurora Imaging Library selects all of the initially established groups. The group index is determined at runtime when creating the groups. To select only one group based on its index, use M_GROUP_SELECTION_OCCURRENCE.	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUP_SELECTION_OCCURRENCE		Sets whether to select one group of edgels or all groups of edgels. This setting applies to constructed edgel features built by calling MmetAddFeature() with M_GROUP_EDGELS. This setting only has an effect if M_GROUP_SELECTION_MODE is set to M_OCCURRENCE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies to use all groups of edgels.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the index of the group of edgels to select. The index number of the groups is determined at runtime.	c		e	f	g	a		i		j	k	l	m	n	b

M_GROUP_SELECTION_POINT		Sets the point or local frame feature from which to select the group of edgels. This setting applies to constructed edgel features built by calling MmetAddFeature() with M_GROUP_EDGELS. This setting only has an effect if M_GROUP_SELECTION_MODE is set to M_CLOSEST_TO_POINT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_GLOBAL_FRAME.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GLOBAL_FRAME	Specifies a virtual point at the location (origin) of the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label of an existing point feature or local frame feature.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to set the geometry of the derived metrology region. In this case, the LabelOrIndex parameter must be set to M_DERIVED_GEOMETRY_REGION and the MetId parameter must be set to a derived metrology region object. You must also call MmetSetRegion() with M_FROM_DERIVED_GEOMETRY_REGION and specify the label or index of the measured feature that will be established from the area delimited by the derived metrology region.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting the geometry of the derived metrology region																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_REGION_GEOMETRY		Sets the geometry of the derived metrology region. Each M_REGION_GEOMETRY setting states the required set of M_REGION_... controls that fully define the derived metrology region and its orientation. INQ															
	M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ARC	Specifies that the feature will be established with an arc metrology region. You can use M_ARC to establish point features. To define an arc metrology region, you must set its end angle (M_REGION_ANGLE_END), start angle (M_REGION_ANGLE_START), position (M_REGION_POSITION or M_REGION_POSITION_X and M_REGION_POSITION_Y), and radius (M_REGION_RADIUS). 	c		e	f	g	a		i		j	k	l	m	n	b

	M_INFINITE	Specifies that the feature will be established with an infinite metrology region. An infinite metrology region is an X- Y-plane, which makes it similar to a rectangle metrology region. However, a rectangle is bounded by its width and height, while an infinite metrology region has no such boundary. You can use M_INFINITE to establish any feature. To define an infinite metrology region, you must set its angle (M_REGION_ANGLE) and position (M_REGION_POSITION or M_REGION_POSITION_X and M_REGION_POSITION_Y). The orientation of an infinite metrology region is established by its angle.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RECTANGLE	Specifies that the feature will be established with a rectangle metrology region. You can use M_RECTANGLE to establish segment or edgel features. To define a rectangle metrology region, you must set its angle (M_REGION_ANGLE), height (M_REGION_HEIGHT), position (M_REGION_POSITION or M_REGION_POSITION_X and M_REGION_POSITION_Y), and width (M_REGION_WIDTH). 	c		e	f	g	a		i		j	k	l	m	n	b
	M_RING	Specifies that the feature will be established with a ring metrology region. You can use M_RING to establish circle or edgel features. To define a ring metrology region, you must set its position (M_REGION_POSITION or M_REGION_POSITION_X and M_REGION_POSITION_Y), end radius (M_REGION_RADIUS_END), and start radius (M_REGION_RADIUS_START). 	c		e	f	g	a		i		j	k	l	m	n	b
	M_RING_SECTOR	Specifies that the feature will be established with a ring-sector metrology region. You can use M_RING_SECTOR to establish radial segment, arc, or edgel features. To define a ring-sector metrology region, you must set its end angle (M_REGION_ANGLE_END), start angle (M_REGION_ANGLE_START), position (M_REGION_POSITION or M_REGION_POSITION_X and M_REGION_POSITION_Y), end radius (M_REGION_RADIUS_END) and start radius (M_REGION_RADIUS_START). 	c		e	f	g	a		i		j	k	l	m	n	b

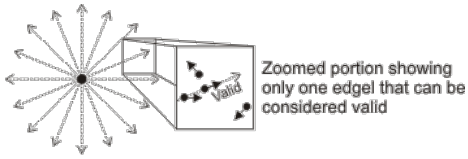
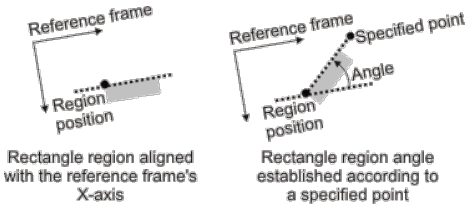
	M_SEGMENT	Specifies that the feature will be established with a linear segment metrology region. You can use M_SEGMENT to establish point features. To define a linear segment metrology region, you must set its end point (M_REGION_END or M_REGION_END_X and M_REGION_END_Y), and start point (M_REGION_START or M_REGION_START_X and M_REGION_START_Y). 	c		e	f	g	a		i		j	k	l	m	n	b

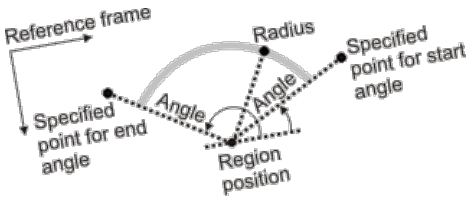
The following ControlType and corresponding ControlValue parameter settings are used to set the geometry data required to define a derived metrology region. In this case, the LabelOrIndex parameter must be set to M_DERIVED_GEOMETRY_REGION and the MetId parameter must be set to a derived metrology region object.

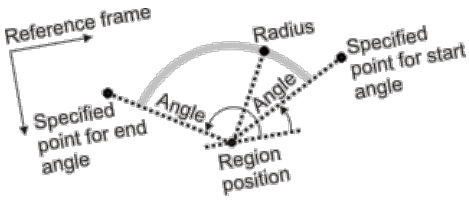
The controls in the following table that you must set depend on the geometry specified with M_REGION_GEOMETRY.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

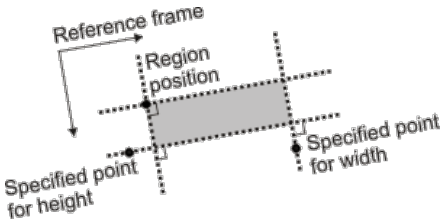
For setting the geometry data of the derived metrology region																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_REGION_ANGLE		Sets the angle of the derived metrology region, relative to the reference frame. M_REGION_ANGLE applies to M_INFINITE and M_RECTANGLE derived metrology regions, unless otherwise specified. INQ															
	M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b

	M_RADIAL	Specifies that the infinite metrology region has a radial orientation, from 0.0° to 360.0°, based on the specified position (M_REGION_POSITION). M_RADIAL applies to M_INFINITE only.  In such a region, edgels can only be considered part of the physically measured feature if their gradient angle conforms to the direction of a radii pointing out from the region's position. M_RADIAL is usually used to define a metrology region for a physically measured circular feature whose radius is unknown. Typically the position of this metrology region is placed at the feature's center, resulting in valid edgels only being possible if they encircle the position.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the angle, in degrees. In this case, you must set M_REGION_ANGLE_TYPE to M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label of a point feature, which metrology uses to set the angle of the derived metrology region. In this case, you must set M_REGION_ANGLE_TYPE to M_FEATURE_LABEL_VALUE. When specifying the label of a point, Aurora Imaging Library considers the angle to be between the X-axis along the metrology region's position (for example, M_REGION_POSITION), and a theoretical straight line that connects that position to the specified point feature. By default, the X-axis along the metrology region's position is aligned along the X-axis of the reference frame. The following example shows how a specified point establishes the angle of a rectangle metrology region.  	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ANGLE_END		Sets the end angle of the derived metrology region, relative to the reference frame. M_REGION_ANGLE_END applies to M_ARC or M_RING_SECTOR derived metrology regions. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 360.0°.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the angle, in degrees. In this case, you must set M_REGION_ANGLE_END_TYPE to M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0	Specifies the label of a point feature, which metrology uses to set the end angle of the metrology region. In this case, you must set M_REGION_ANGLE_END_TYPE to M_FEATURE_LABEL_VALUE. When specifying the label of a point, Aurora Imaging Library considers the end angle to be between the X-axis along the metrology region's position (for example, M_REGION_POSITION), and a theoretical straight line that connects that position to the specified point feature. By default, the X-axis along the metrology region's position is aligned along the X-axis of the reference frame. The following example shows how a specified point establishes the end angle of an arc metrology region. The example also shows how a specified point establishes the start angle of an arc metrology region (M_REGION_ANGLE_START).  	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ANGLE_END_TYPE		Sets whether you are using a feature or an explicit value to specify the end angle of the derived metrology region (M_REGION_ANGLE_END). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ANGLE_START		Sets the start angle of the derived metrology region, relative to the reference frame. M_REGION_ANGLE_START applies to M_ARC or M_RING_SECTOR derived metrology regions. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the angle, in degrees. In this case, you must set M_REGION_ANGLE_START_TYPE to M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0	Specifies the label of a point feature, which metrology uses to set the start angle of the metrology region. In this case, you must set M_REGION_ANGLE_START_TYPE to M_FEATURE_LABEL_VALUE. When specifying the label of a point, Aurora Imaging Library considers the start angle to be between the X-axis along the metrology region's position (for example, M_REGION_POSITION), and a theoretical straight line that connects that position to the specified point feature. By default, the X-axis along the metrology region's position is aligned along the X-axis of the reference frame. The following example shows how a specified point establishes the start angle of an arc metrology region. The example also shows how a specified point establishes the end angle of an arc metrology region (M_REGION_ANGLE_END).  	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ANGLE_START_TYPE	Sets whether you are using a feature or an explicit value to specify the start angle of the derived metrology region (M_REGION_ANGLE_START). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ANGLE_TYPE	Sets whether you are using a feature or an explicit value to specify the angle of the derived metrology region (M_REGION_ANGLE). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_END	Sets the end point of the derived metrology region. M_REGION_END applies to M_SEGMENT derived metrology regions. To use M_REGION_END, you must set M_REGION_END_TYPE to M_FEATURE_LABEL_VALUE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of a point feature, which metrology uses to set the end point of the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b

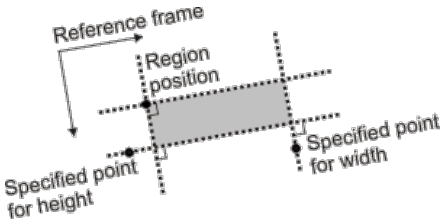
M_REGION_END_TYPE	Sets whether you are using a feature or an explicit value to specify the end point of the derived metrology region (M_REGION_END, or M_REGION_END_X and M_REGION_END_Y). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_END_X	Sets the X-coordinate of the end point of the derived metrology region, relative to the reference frame. M_REGION_END_X applies to M_SEGMENT derived metrology regions. To use M_REGION_END_X, you must set M_REGION_END_TYPE to M_PARAMETRIC. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the end point's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_END_Y	Sets the Y-coordinate of the end point of the derived metrology region, relative to the reference frame. M_REGION_END_Y applies to M_SEGMENT derived metrology regions. To use M_REGION_END_Y, you must set M_REGION_END_TYPE to M_PARAMETRIC. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the end point's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_HEIGHT	Sets the height of the derived metrology region. M_REGION_HEIGHT applies to M_RECTANGLE derived metrology regions. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0	Specifies the label of a point feature, which metrology uses to set the rectangle's height. In this case, you must set M_REGION_HEIGHT_TYPE to M_FEATURE_LABEL_VALUE. When specifying the label of a point, Aurora Imaging Library considers the height to be at the perpendicular intersection point between the Y-axis along the metrology region's position (for example, M_REGION_POSITION), and the X-axis along the specified point feature. By default, the X- and Y-axes along the metrology region's position and along the specified point are aligned with the reference frame. The example also shows how a specified point establishes the width of a rectangle metrology region (M_REGION_WIDTH). 	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the rectangle's height, in pixel or world units. In this case, you must set M_REGION_HEIGHT_TYPE to M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_HEIGHT_TYPE	Sets whether you are using a feature or an explicit value to specify the height of the derived metrology region (M_REGION_HEIGHT). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_POSITION	Sets the position of the derived metrology region. M_REGION_POSITION applies to all derived metrology regions except M_SEGMENT. To use M_REGION_POSITION, you must set M_REGION_POSITION_TYPE to M_FEATURE_LABEL_VALUE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of a point feature, which metrology uses to set the position of the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_POSITION_TYPE	Sets whether you are using a feature or an explicit value to specify the position of the derived metrology region (M_REGION_POSITION, or M_REGION_POSITION_X and M_REGION_POSITION_Y). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b

	M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_POSITION_X		Sets the X-coordinate of the position of the derived metrology region, relative to the reference frame. M_REGION_POSITION_X applies to all derived metrology regions except M_SEGMENT. To use M_REGION_POSITION_X, you must set M_REGION_POSITION_TYPE to M_PARAMETRIC. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the position's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_POSITION_Y		Sets the Y-coordinate of the position of the derived metrology region, relative to the reference frame. M_REGION_POSITION_Y applies to all derived metrology regions except M_SEGMENT. To use M_REGION_POSITION_Y, you must set M_REGION_POSITION_TYPE to M_PARAMETRIC. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the position's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS		Sets the radius of the derived metrology region. M_REGION_RADIUS applies to M_ARC derived metrology regions. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label of a point feature, which metrology uses to set the arc's radius. In this case, you must set M_REGION_RADIUS_TYPE to M_FEATURE_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the radius, in pixel or world units. In this case, you must set M_REGION_RADIUS_TYPE to M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS_END		Sets the end radius of the derived metrology region. M_REGION_RADIUS_END applies to M_RING or M_RING_SECTOR derived metrology regions. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0	Specifies the label of a point feature, which metrology uses to set the end radius. In this case, you must set M_REGION_RADIUS_END_TYPE to M_FEATURE_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the end radius, in pixel or world units. In this case, you must set M_REGION_RADIUS_END_TYPE to M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS_END_TYPE	Sets whether you are using a feature or an explicit value to specify the end radius of the derived metrology region (M_REGION_RADIUS_END). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS_START	Sets the start radius of the derived metrology region. M_REGION_RADIUS_START applies to M_RING or M_RING_SECTOR derived metrology regions. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of a point feature, which metrology uses to set the start radius. In this case, you must set M_REGION_RADIUS_START_TYPE to M_FEATURE_LABEL_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the start radius, in pixel or world units. In this case, you must set M_REGION_RADIUS_START_TYPE to M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS_START_TYPE	Sets whether you are using a feature or an explicit value to specify the start radius of the derived metrology region (M_REGION_RADIUS_START). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS_TYPE	Sets whether you are using a feature or an explicit value to specify the radius of the derived metrology region (M_REGION_RADIUS). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b

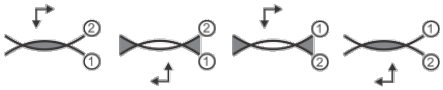
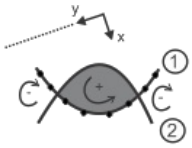
	M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_START		Sets the start point of the derived metrology region. M_REGION_START applies to M_SEGMENT derived metrology regions. To use M_REGION_START, you must set M_REGION_START_TYPE to M_FEATURE_LABEL_VALUE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label of a point feature, which metrology uses to set the start point of the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_START_TYPE		Sets whether you are using a feature or an explicit value to specify the start point of the derived metrology region (M_REGION_START or M_REGION_START_X and M_REGION_START_Y). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_START_X		Sets the X-coordinate of the start point of the derived metrology region, relative to the reference frame. M_REGION_START_X applies to M_SEGMENT derived metrology regions. To use M_REGION_START_X, you must set M_REGION_START_TYPE to M_PARAMETRIC. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the start point's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_START_Y		Sets the Y-coordinate of the start point of the derived metrology region, relative to the reference frame. M_REGION_START_Y applies to M_SEGMENT derived metrology regions. To use M_REGION_START_Y, you must set M_REGION_START_TYPE to M_PARAMETRIC. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the start point's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b

M_REGION_WIDTH	Sets the width of the derived metrology region. M_REGION_WIDTH applies to M_RECTANGLE derived metrology regions. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of a point feature, which metrology uses to set the rectangle's width. In this case, you must set M_REGION_WIDTH_TYPE to M_FEATURE_LABEL_VALUE. When specifying the label of a point, Aurora Imaging Library considers the width to be at the perpendicular intersection point between the X-axis along the metrology region's position (for example, M_REGION_POSITION), and the Y-axis along the specified point feature. By default, the X- and Y-axes along the metrology region's position and along the specified point are aligned with the reference frame. The example also shows how a specified point establishes the height of a rectangle metrology region (M_REGION_HEIGHT).  	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the rectangle's width, in pixel or world units. In this case, you must set M_REGION_WIDTH_TYPE to M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_WIDTH_TYPE	Sets whether you are using a feature or an explicit value to specify the width of the derived metrology region (M_REGION_WIDTH). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies to use a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies to use an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control the geometric tolerances calculated for a metrology context. For controlling geometric tolerances, the LabelOrIndex parameter can be set to M_ALL_TOLERANCES or an existing tolerance label or index using M_TOLERANCE_LABEL() or M_TOLERANCE_INDEX(). The MetId parameter must be set to a metrology context. Unless otherwise specified, the following settings apply to any geometric tolerance (MmetAddTolerance()).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling geometric tolerances																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																

M_AREA_BETWEEN_CURVES_DEFINE_SIDE	Sets the reference frame that determines the side of the curve on which to validate the area, when you specify an M_AREA_BETWEEN_CURVES tolerance, and enable the M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY setting. M_AREA_BETWEEN_CURVES_DEFINE_SIDE is typically used if the two specified curves (edgel features) that the M_AREA_BETWEEN_CURVES tolerance validates cross over each other, causing areas on both sides of the curves. In this case, when enabling M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY, Aurora Imaging Library must establish the side on which the area is considered positive, and must add the areas on that side only. Areas on the other side (which is considered the negative side) are ignored. The positive and negative sides are determined by whether the Y-axis of the reference frame (by default, the global frame) initially intersects with either the first or second curve that is specified with the M_AREA_BETWEEN_CURVES tolerance. If the Y-axis initially intersects with the first specified curve, all the areas on the side of the first curve facing the direction of the Y-axis are considered positive and Aurora Imaging Library adds them; all the areas on the other side of the first curve are considered negative and Aurora Imaging Library ignores them. If the Y-axis initially intersects with the second specified curve, all the areas on the side of the second curve facing the direction of the Y-axis are considered negative and Aurora Imaging Library ignores them; all the areas on the other side of the second curve are considered positive and Aurora Imaging Library adds them.  Shaded areas considered positive If neither curve intersects the Y-axis, or if only one curve intersects the Y-axis, the sign of the areas between the curves is computed based on the order in which the two curves are defined. If the points delimiting an area on the first specified curve appear in a clockwise order, then this area will be negative. If they appear in a counter-clockwise order, then this area will be positive.  Shaded area considered positive Note that both the reference frame, and the order in which you specify the two curves with M_AREA_BETWEEN_CURVES, determine the side on which to validate the tolerance. INQ														
M_DEFAULT	Same as M_GLOBAL_FRAME.	c		e	f	g	a		i		j	k	l	m	n	b

	M_FEATURE_INDEX(AIL_INT LocalFrameIndex)	Specifies the label value of a local frame feature.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters															
		LocalFrameIndex Specifies the local frame index, as a value greater than or equal to 0. The index value 0 refers to the global frame.															
	M_FEATURE_LABEL(AIL_INT LocalFrameLabel)	Specifies the label value of a local frame feature.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters															
		LocalFrameLabel Specifies the local frame label, as a value greater than 0.															
	M_GLOBAL_FRAME	Specifies the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY		Sets whether to validate the area on only one side of the curve, when you specify an M_AREA_BETWEEN_CURVES tolerance. M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY is typically used if the two specified curves (edgel features) that the M_AREA_BETWEEN_CURVES tolerance validates cross over each other, causing areas on both sides of the curves. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that Aurora Imaging Library does not validate the area on only one side of the curve. Areas on both sides are either all added, or added and subtracted, depending on whether M_AREA_BETWEEN_CURVES_OPPOSITES_SUBTRACT is enabled or disabled.	c		e	f	g	a		i		j	k	l	m	n	b

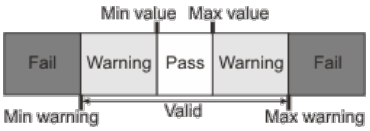
	M_ENABLE	Specifies that Aurora Imaging Library validates the area on only one side of the curve. To enable M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY, you must disable M_AREA_BETWEEN_CURVES_OPPOSITES_SUBTRACT (it is disabled by default). When enabling M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY, Aurora Imaging Library must establish the side on which the area is considered positive, and must add the areas on that side only. Areas on the other side (which is considered the negative side) are ignored. By default, the positive and negative sides are determined by whether the global reference frame's Y-axis initially intersects with either the first or second curve that is specified with the M_AREA_BETWEEN_CURVES tolerance. To determine the positive and negative sides by using a reference frame other than the global frame, use M_AREA_BETWEEN_CURVES_DEFINE_SIDE. For more information about this behavior, see M_AREA_BETWEEN_CURVES_DEFINE_SIDE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_BETWEEN_CURVES_OPPOSITES_SUBTRACT		Sets whether Aurora Imaging Library should subtract opposing areas, when you specify an M_AREA_BETWEEN_CURVES tolerance. M_AREA_BETWEEN_CURVES_OPPOSITES_SUBTRACT only has an effect if the two specified curves (edgel features) that the M_AREA_BETWEEN_CURVES tolerance validates cross over each other, causing areas on both sides of the curves. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that Aurora Imaging Library does not subtract opposing areas. All areas between the curves are added. By default, Aurora Imaging Library adds the areas on both sides of the curves. To add the areas on only one side, enable M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that Aurora Imaging Library subtracts the areas on one side of the curve (opposing areas) from the areas on the other side of the curve, so that the result is positive. The side with the smaller total area is always subtracted from the side with the greater total area. To enable M_AREA_BETWEEN_CURVES_OPPOSITES_SUBTRACT, you must disable M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY (it is disabled by default).	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_UNDER_CURVE_ALLOW_NEGATIVE		Sets whether Aurora Imaging Library allows negative (opposing) areas when establishing the area tolerance for M_AREA_UNDER_CURVE_MAX or M_AREA_UNDER_CURVE_MIN. Negative areas occur when the specified curve (edgel) feature that the tolerance is validating crosses over the specified line feature. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that Aurora Imaging Library does not allow negative areas when establishing the area tolerance. If there are negative areas, the tolerance fails.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that Aurora Imaging Library allows negative areas when establishing the area tolerance. In this case, Aurora Imaging Library subtracts the negative areas from the larger, positive areas when establishing the area tolerance. The negative areas are those located on the side of the line where the sum of the areas is the smallest.	c		e	f	g	a		i		j	k	l	m	n	b
M_CURVE_EDGEL_GAP_SIZE		Sets the maximum gap between edgels for Aurora Imaging Library to consider them part of the same curve of edgels, when you specify an M_AREA_BETWEEN_CURVES, M_AREA_UNDER_CURVE_MAX, or M_AREA_UNDER_CURVE_MIN tolerance. If the gap between edgels is greater than the maximum gap, those edgels will not be part of the same curve. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PIXEL_SIZE_DIAGONAL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL_SIZE_DIAGONAL	Specifies twice the diagonal size of the pixels from which the edgels were taken.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the size, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CURVE_INFO		Sets the information that Aurora Imaging Library uses to establish the curves of edgels, when you specify an M_AREA_BETWEEN_CURVES tolerance. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO_CURVE_OPENED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_INDEX(AIL_INT LocalFrameIndex)	Specifies the index of a reference frame feature (local frame). The reference frame indicates the direction of the curves, and is used to sort the edgels accordingly. That is, M_AREA_BETWEEN_CURVES assumes the direction of the curves' edgels is relative to the reference frame's X-axis. Specifying a direction can make the tolerance faster to calculate, and less sensitive to gaps between edgels. When you set M_CURVE_INFO to a reference frame, M_AREA_BETWEEN_CURVES assumes that the curves are unbounded (open), as edgel direction does not apply to bounded (closed) curves.	c		e	f	g	a		i		j	k	l	m	n	b

		Parameters													
		LocalFrameIndex Specifies the index value of an existing reference frame (local frame). The index value 0 refers to the global frame.													
	M_FEATURE_LABEL(AIL_INT LocalFrameLabel)	Specifies the label of a reference frame feature (local frame). The reference frame indicates the direction of the curves, and is used to sort the edgels accordingly. That is, M_AREA_BETWEEN_CURVES assumes the direction of the curves' edgels is relative to the reference frame's X-axis. Specifying a direction can make the tolerance faster to calculate, and less sensitive to gaps between edgels. When you set M_CURVE_INFO to a reference frame, M_AREA_BETWEEN_CURVES assumes that the curves are unbounded (open), as edgel direction does not apply to bounded (closed) curves.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters													
		LocalFrameLabel Specifies the label value of an existing reference frame (local frame).													
	M_AUTO_CURVE_CLOSED	Specifies that the M_AREA_BETWEEN_CURVES tolerance is validating bounded (closed) curves (edgel features). This setting is typically used when you want to validate closed curves that pass through all the edgels. Such curves should not cross over themselves and must form only one closed shape (for example, the curve cannot form a figure 8, which can be seen as two circular shapes). With this setting, the tolerance does not assume that the curves follow a specific direction (since the shape is closed, direction does not apply). Such curves should have edgels with very minimal gaps between them. If the curves (edgel features) that you specified were externally established with sequential edgels (M_EDGEL_PROVIDED_ORDER set to M_SEQUENTIAL), Aurora Imaging Library maintains the order of the edgels, which can make calculations faster and more accurate (provided that the specified edgel order forms a properly closed curve).	c		e	f	g	a		i		j	k	l	m	n	b

	M_AUTO_CURVE_OPENED	Specifies that the M_AREA_BETWEEN_CURVES tolerance is validating unbounded (opened) curves (edgel features). This setting is typically used when you want to validate opened curves that pass through all the edgels. Such curves should not cross over themselves and must form one fully opened shape (for example, the curve cannot form the number 6, which can be seen as touching and partially closed). With this setting, the tolerance does not assume that the curves follow a specific direction. Such curves should have edgels with very minimal gaps between them. To make the tolerance less sensitive to gaps (and potentially faster to calculate), specify a direction for the curve, by setting M_CURVE_INFO to a reference frame. If the curves (edgel features) that you specified were externally established with sequential edgels (M_EDGEL_PROVIDED_ORDER set to M_SEQUENTIAL), Aurora Imaging Library maintains the order of the edgels, which can make calculations faster and more accurate (provided that the specified edgel order forms a properly opened curve).	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_MODE		Sets how to apply the angle at which to establish the distance between features when adding a distance tolerance. This affects MmetAddTolerance() with M_DISTANCE_MAX or M_DISTANCE_MIN. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ABSOLUTE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABSOLUTE	Specifies that Aurora Imaging Library establishes the angle by taking the distance at which the features are either the farthest (for M_DISTANCE_MAX) or the closest (for M_DISTANCE_MIN).	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_AT_ANGLE	Specifies that Aurora Imaging Library calculates the distance tolerance using the feret at the angle that is set (M_ANGLE). This only applies to M_DISTANCE_MAX. Aurora Imaging Library measures the angle (M_ANGLE) relative to the coordinate system of the first base feature with which to validate the maximum distance tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GAP_AT_ANGLE	Specifies that Aurora Imaging Library calculates the distance tolerance using the gap at the angle that is set (M_ANGLE). This only applies to M_DISTANCE_MIN. Aurora Imaging Library measures the angle (M_ANGLE) relative to the first base feature with which to validate the minimum distance tolerance.	c		e	f	g	a		i		j	k	l	m	n	b

M_GAIN		Sets the gain Aurora Imaging Library uses to correct systematic bias between multiple imaging vision set-ups. Systematic bias can, for example, exist between two different Coordinate Measuring Machines (CMMs). Note that tolerance values, which Aurora Imaging Library uses to determine the status of features (and the status of tolerances themselves), are a composite of measured value, gain (M_GAIN), and offset (M_OFFSET). INQ $ToleranceValue = (Gain\ X\ MeasuredValue) + Offset$	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the gain.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET		Sets the offset Aurora Imaging Library uses to correct systematic bias between multiple imaging vision set-ups. Systematic bias can, for example, exist between two different Coordinate Measuring Machines (CMMs). Note that tolerance values, which Aurora Imaging Library uses to determine the status of features (and the status of tolerances themselves), are a composite of measured value, offset (M_OFFSET), and gain (M_GAIN). INQ $ToleranceValue = (Gain\ X\ MeasuredValue) + Offset$	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the offset.	c		e	f	g	a		i		j	k	l	m	n	b

M_VALUE_MAX	Sets the maximum acceptable value for the geometric tolerance Aurora Imaging Library uses to validate the feature or the relationship between multiple features. For example, if you use a length tolerance to validate the linear dimension of a segment feature, and you set M_VALUE_MAX to 100 pixels, the feature (and the tolerance itself) can only be valid if the segment's length is less than 100 pixels. Similarly, if you are using a perpendicularity tolerance to validate the relationship between two segment features, and you set M_VALUE_MAX to 0.05°, the features (and the tolerance itself) can only be valid if the angle between the segments is between 90.0° and 90.05°. To get the status (M_PASS, M_WARNING, or M_FAIL) of the feature or the geometric tolerance itself, use MmetGetResult() with M_STATUS. Features can only have a passing status if their geometric tolerance has a passing status. Aurora Imaging Library determines the status of geometric tolerances according to the M_VALUE_MAX, M_VALUE_MIN, M_VALUE_WARNING_MAX, and M_VALUE_WARNING_MIN settings. The following is an example of (non-zero) minimum and maximum tolerance values and warning values: INQ  	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALUE_MIN	Sets the minimum value for a geometric tolerance to have a valid status. To get the status of the geometric tolerance (M_PASS, M_WARNING, or M_FAIL), use MmetGetResult() with M_STATUS. The geometric tolerance's status is determined according to the M_VALUE_MAX, M_VALUE_MIN, M_VALUE_WARNING_MAX, and M_VALUE_WARNING_MIN settings. For an example of (non-zero) minimum and maximum tolerance values and warning values, see M_VALUE_MAX. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b

M_VALUE_WARNING_MAX	Sets the maximum warning value for a geometric tolerance to have a valid status. To get the status of the geometric tolerance (M_PASS, M_WARNING, or M_FAIL), use MmetGetResult() with M_STATUS. The geometric tolerance's status is determined according to the M_VALUE_MAX, M_VALUE_MIN, M_VALUE_WARNING_MAX, and M_VALUE_WARNING_MIN settings. For an example of (non-zero) minimum and maximum tolerance values and warning values, see M_VALUE_MAX. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_EQUAL_PASS_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
M_EQUAL_PASS_MAX	Specifies that the maximum warning value is equal to the maximum pass value (M_VALUE_MAX).	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum warning value.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALUE_WARNING_MIN	Sets the minimum warning value for a geometric tolerance to have a valid status. To get the status of the geometric tolerance (M_PASS, M_WARNING, or M_FAIL), use MmetGetResult() with M_STATUS. The geometric tolerance's status is determined according to the M_VALUE_MAX, M_VALUE_MIN, M_VALUE_WARNING_MAX, and M_VALUE_WARNING_MIN settings. For an example of (non-zero) minimum and maximum tolerance values and warning values, see M_VALUE_MAX. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_EQUAL_PASS_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
M_EQUAL_PASS_MIN	Specifies that the minimum warning value is equal to the minimum pass value (M_VALUE_MIN).	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum warning value.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control features and geometric tolerances of a metrology context. For controlling features, the LabelOrIndex parameter can be set to M_GLOBAL_FRAME or one or all features. For controlling geometric tolerances, the LabelOrIndex parameter can be set to one or all tolerances. The MetId parameter must be set to a metrology context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling features or geometric tolerances																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																	
M_ANGLE	Sets the angle used to construct a feature or define a tolerance, when applicable. For more information, refer to the feature or tolerance with which you are using the angle setting (for example, MmetAddFeature() with M_ANGLE / M_ANGLE_ABSOLUTE / M_ANGLE_RELATIVE / M_SECTOR_RELATIVE / M_PARAMETRIC, or MmetAddTolerance() with M_ANGULARITY). INQ															

	M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the angle. Depending on the build operation, you must set the angle in either degrees, or as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAWABLE		Sets whether the feature or tolerance will be drawn using MmetDraw(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the feature or tolerance will not be drawn. In this case, you cannot affect them in an interactive display.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the feature or tolerance will be drawn.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to transform clone-type features (MmetAddFeature() with M_CLONE_FEATURE). For example, M_CLONE_SCALE can specify that the cloned feature will be twice the size of its source. To specify clone transformations, set the LabelOrIndex parameter to the label or index of an existing clone-type feature, using M_FEATURE_LABEL() or M_FEATURE_INDEX(). This is the cloned feature to which the transformation (M_CLONE_...) will apply. The MetId parameter must be set to a metrology context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling clone transformations																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CLONE_ANGLE		Sets the rotation angle, in the counterclockwise direction, that is applied to the cloned feature (LabelOrIndex). INQ															
	M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLONE_OFFSET_X		Sets the translation value, in the X-direction, that is applied to the cloned feature (LabelOrIndex). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-offset for the cloned feature, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLONE_OFFSET_Y		Sets the translation value, in the Y-direction, that is applied to the cloned feature (LabelOrIndex). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the Y-offset for the cloned feature, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLONE_SCALE		Sets the scale factor that is applied to the cloned feature (LabelOrIndex). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the scale factor.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control the results calculated and stored in a metrology result buffer. For controlling results, the LabelOrIndex parameter must be set to M_GENERAL. The MetId parameter must be set to a metrology result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling results																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																	
M_OUTPUT_FRAME		Sets the output reference frame that Aurora Imaging Library uses to return feature results. All results are returned relative to the output reference frame that you set. The output reference frame is reset to M_GLOBAL_FRAME after each call to MmetCalculate(). INQ																
	M_DEFAULT	Same as M_GLOBAL_FRAME.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GLOBAL_FRAME	Specifies that Aurora Imaging Library returns feature results relative to the origin of the global frame. If the target image is not calibrated, the global frame's default origin (0,0) is aligned with the center of the top-left corner pixel of the target image. If the target image is calibrated, the global frame's default origin is aligned with the origin of the relative (world) coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IMAGE_FRAME	Specifies that Aurora Imaging Library returns feature results relative to the origin of the target image's reference frame. If the target image is not calibrated, the origin of the target image's reference frame is at the center of the top-left corner pixel of the target image. If the target image is calibrated, the origin of the target image reference frame is the same as the relative coordinate system of the camera calibration.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REFERENCE_FRAME	Specifies that Aurora Imaging Library returns feature results relative to the reference frame of each feature. By default, a feature's reference frame is the global frame.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0	Specifies the label of an existing local frame feature to use to return feature results.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_OUTPUT_UNITS	Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling MmetGetResult() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to delete features and tolerances from the context. The LabelOrIndex parameter can be set to one of the following: M_ALL_FEATURES, M_ALL_TOLERANCES, an existing feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX(), or an existing tolerance label or index using M_TOLERANCE_LABEL() or M_TOLERANCE_INDEX(). The MetId parameter must be set to a metrology context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For deleting features and tolerances																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_DELETE	Deletes a specific feature, a specific tolerance, all features, or all tolerances. To delete all features, the LabelOrIndex parameter needs to be set to M_ALL_FEATURES. When deleting a feature, all its derived features and associated geometric tolerances are also deleted. To delete all tolerances, the LabelOrIndex parameter needs to be set to M_ALL_TOLERANCES.															
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Deletes a specific feature, a specific tolerance, all features, or all tolerances.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MmetControlInt64 (AIL_ID MetId, AIL_INT LabelOrIndex, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

MetId

See MetId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MmetControlDouble (AIL_ID MetId, AIL_INT LabelOrIndex, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

MetId

See MetId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetDraw

Synopsis

Draw specific features or tolerances of a metrology context or of a metrology result.

Syntax

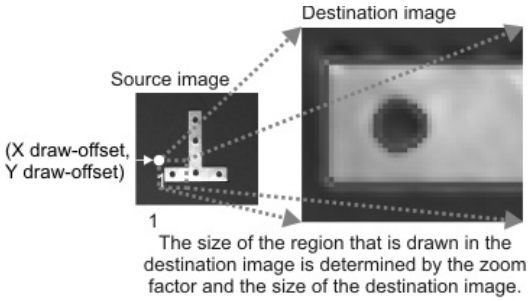
```
void MmetDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ContextOrResultMetId,   //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT LabelOrIndex,          //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific features or tolerances of a metrology context or of a metrology result, in the destination image buffer or 2D graphics list.

If you pass a context, **MmetDraw**() can only draw the specified feature/tolerance if the function can internally compute it without the need of a source image. This is possible if the feature/tolerance is ultimately based on constructed edgel features built with external edgels/points (MmetAddFeature() with M_EDGEL and M_EXTERNAL_FEATURE) and the external edgels/points have been specified using MmetPut(), and/or if it is ultimately based on parametric features. Note that **MmetDraw**() does not modify the context with internally calculated results.

You can draw results and settings, obtained relative to an offset, at the top-left corner of the destination image, using MgraControl() with M_DRAW_OFFSET_X and M_DRAW_OFFSET_Y and zoom them using MgraControl() with M_DRAW_ZOOM_X and M_DRAW_ZOOM_Y. For more information, see the Drawing graphics with offset and zoom subsection of the Drawing graphics section of Chapter 26: Generating graphics.



When zooming, **MmetDraw**() will draw into the destination image buffer even if the buffer is not large enough to contain all of the zoomed image. If necessary, the image will be clipped.

If a template reference has been set using MmetControl() with M_TEMPLATE_REFERENCE_ID, **MmetDraw**() can draw the results of features and tolerances extracted from that template reference. You will need to inquire M_TEMPLATE_REFERENCE_SIZE_X and M_TEMPLATE_REFERENCE_SIZE_Y to allocate an appropriate destination buffer.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use when drawing. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. > Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextOrResultMetId

Specifies the metrology result buffer or context from which to extract the features to draw. The metrology result buffer must have been previously allocated using MmetAllocResult(). The metrology context must have been previously allocated using MmetAlloc().

If you are using **MmetDraw**() with a metrology context, you can specify that a feature or tolerance in the context should not be drawn, using MmetControl() with M_DRAWABLE set to M_DISABLE. This can be useful if you have many features or tolerances and you want to draw all of them in one call, although there are some that you do not want to see. When features or tolerances are not drawn, you cannot affect them in an interactive display.

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. The buffer can be any valid 1- or 3-band image buffer allocated using MbufAlloc...(). The 2D graphics list must be previously allocated using MgraAllocList(). By drawing into a display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

When using **MmetDraw**() to draw features with a source model that is associated with a calibration context and if a calibration context is associated with the destination buffer, the latter must also be calibrated to the same world coordinate system. If no calibration context is associated with the destination buffer, the latter will use the same one as the template reference's. To add a template reference to a metrology context, use MmetControl() with M_TEMPLATE_REFERENCE_ID. **MmetDraw**() draws all the requested features according to whether the destination image buffer is physically corrected or not.

Operation

Specifies the type of drawing operation to perform. This parameter can be set to the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For drawing a context characteristic																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_TEMPLATE_REFERENCE	Draws the template reference. The LabelOrIndex parameter must be set to M_GENERAL. > This operation cannot draw in a 2D graphics list.	c		e	f	g	a		i		j	k	l	m	n	b

For the following constants, the LabelOrIndex parameter can be set to M_FEATURE_LABEL(), M_FEATURE_INDEX(), or M_ALL_FEATURES, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For drawing a feature characteristic																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_ACTIVE_EDGELS	Draws only the edgels that satisfied all edgel constraints.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_ALL_EDGELS	Draws all the edgels that were calculated in the target image. This includes the edgels that were used to measure a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_FEATURE +	Draws the specified feature(s). In this case, you can also set the LabelOrIndex parameter to M_GLOBAL_FRAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_FITTED_EDGELS	Draws only the edgels that satisfied all edgel constraints and fit constraints. The fitted edgels are those used by the fit operation to define a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_NOISY_EDGELS	Draws the edgels before they were denoised. Edgels are denoised using MmetControl() with M_EDGEL_DENOISING_MODE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_REGION	Draws the feature's search region. When drawing in a 2D graphics list, this operation cannot draw ring and ring sector regions when M_REGION_ACCURACY_HIGH is set to M_DISABLE and the 2D graphics context's input units is set to world units (MgraControl() with M_INPUT_UNITS set to M_WORLD).	c		e	f	g	a		i		j	k	l	m	n	b

For the following constants, the LabelOrIndex parameter can be set to M_TOLERANCE_LABEL(), M_TOLERANCE_INDEX(), M_ALL_TOLERANCES, M_ALL_PASS_TOLERANCES, M_ALL_FAIL_TOLERANCES, or M_ALL_WARNING_TOLERANCES.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For drawing a tolerance characteristic																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_TOLERANCE +	Draws the geometric tolerance's icon. To view the icons, see MmetAddTolerance().	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_TOLERANCE_AREA	Draws the area(s) used to define the area-based geometric tolerance. These tolerances are set using MmetAddTolerance() with M_AREA_....	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_TOLERANCE_AREA_NEGATIVE	Draws the negative area(s) used to define the area-based geometric tolerance. Positive and negative areas occur when areas exist on both sides of a curve. The negative areas are those located on the side of the curve where the sum of the areas is the smallest. These tolerances are set using MmetAddTolerance() with M_AREA_.... Note that what is considered the positive side changes if M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY is enabled. In this case, the positive and negative sides are determined by whether the global reference frame's Y-axis initially intersects with either the first or second curve (edgel feature) that is specified with the M_AREA_BETWEEN_CURVES tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_TOLERANCE_AREA_POSITIVE	Draws the positive area(s) used to define the area-based geometric tolerance. Positive and negative areas occur when areas exist on both sides of a curve. The positive areas are those located on the side of the curve where the sum of the areas is the largest. These tolerances are set using MmetAddTolerance() with M_AREA_.... Note that what is considered the positive side changes if M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY is enabled. In this case, the positive and negative sides are determined by whether the global reference frame's Y-axis initially intersects with either the first or second curve (edgel feature) that is specified with the M_AREA_BETWEEN_CURVES tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_TOLERANCE_FEATURES	Draws the features used to define a tolerance. The LabelOrIndex parameter can be set to M_TOLERANCE_LABEL(), M_TOLERANCE_INDEX(), M_ALL_TOLERANCES, M_ALL_PASS_TOLERANCES, M_ALL_FAIL_TOLERANCES, or M_ALL_WARNING_TOLERANCES.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_DRAW_FEATURE; M_DRAW_TOLERANCE.

You can add one of the following values to the above-mentioned values to specify that the label or name of the feature or tolerance will also be drawn.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_DRAW_FEATURE and M_DRAW_TOLERANCE																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_LABEL	Specifies that the label will also be drawn.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_NAME	Specifies that the name will also be drawn.	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Specifies the label or index of the result in the metrology result buffer for which to perform the drawing operation, or specifies that you are performing a general drawing operation for the metrology context. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of the element in the metrology result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default setting. M_DEFAULT is the same as M_ALL_FEATURES, when setting the Operation parameter to a value in the For drawing a feature characteristic table. M_DEFAULT is the same as M_ALL_TOLERANCES, when setting the Operation parameter to a value in the For drawing a tolerance characteristic table. M_DEFAULT is the same as M_GENERAL, when setting the Operation parameter to a value in the For drawing a context characteristic table.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_INDEX (AIL_INT FeatureIndex)	Specifies the index value of an existing individual feature to draw, in either the metrology context or result buffer, depending on whether you provide a context or result buffer to the ContextOrResultMetId parameter.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureIndex This parameter specifies the index of the individual feature to draw. You can set this parameter to the following:															
	Value >= 0 Specifies the index of the individual feature. Note that the index of the global frame is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL (AIL_INT FeatureLabel)	Specifies the label value of an existing individual feature to draw, in either the metrology context or result buffer, depending on whether you provide a context or result buffer to the ContextOrResultMetId parameter.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureLabel This parameter specifies the label of the individual feature to draw. You can set this parameter to the following:															

			Value > 0 Specifies the label of the individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_INDEX (AIL_INT ToleranceIndex)	Specifies the index value of an existing individual tolerance to draw, in either the metrology context or result buffer, depending on whether you provide a context or result buffer to the ContextOrResultMetId parameter.			c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																
		ToleranceIndex This parameter specifies the index of the individual tolerance to draw. You can set this parameter to the following:																
			Value >= 0 Specifies the index of the individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_LABEL (AIL_INT ToleranceLabel)	Specifies the label value of an existing individual tolerance to draw, in either the metrology context or result buffer, depending on whether you provide a context or result buffer to the ContextOrResultMetId parameter.			c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																
		ToleranceLabel This parameter specifies the label of the individual tolerance to draw. You can set this parameter to the following:																
			Value > 0 Specifies the label of the individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_FAIL_TOLERANCES	Specifies that the drawing operation will be performed on all tolerance results that have the status M_FAIL.			c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_FEATURES	Specifies to draw all feature results.			c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_PASS_FEATURES	Specifies that the drawing operation will be performed on all feature results that have the status M_PASS.			c		e	f	g	a		i		j	k	l	m	n	b

M_ALL_PASS_TOLERANCES	Specifies that the drawing operation will be performed on all tolerance results that have the status M_PASS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_TOLERANCES	Specifies to draw all tolerance results.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_WARNING_FEATURES	Specifies that the drawing operation will be performed on all feature results that have the status M_WARNING.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_WARNING_TOLERANCES	Specifies that the drawing operation will be performed on all tolerance results that have the status M_WARNING.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to draw a setting of the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_FRAME	Specifies to draw the global frame of the context.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetFree

Synopsis

Free a metrology context, a derived metrology region object, or a metrology result buffer.

Syntax

```
void MmetFree(
    AIL_ID MetId //in
)
```

Description

This function deletes the specified metrology context, derived metrology region object, or metrology result buffer and releases any memory associated with it.

All metrology contexts, derived metrology region objects, and metrology result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

MetId

Specifies the identifier of the metrology context, derived metrology region object, or metrology result buffer to free. These must have been successfully allocated (with MmetAlloc() or MmetAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetGetResult

Synopsis

Get the specified type of results from an Aurora Imaging Library metrology result buffer.

Syntax

```
void MmetGetResult(
    AIL_ID ResultId,      //in
    AIL_INT LabelOrIndex, //in
    AIL_INT64 ResultType, //in
    void *ResultArrayPtr  //out
)
```

Description

This function retrieves all results of the specified type from a metrology result buffer, after an MmetCalculate() call. The result entries are ordered by label value.

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

> If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use MmetControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, note that if you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, **MmetGetResult**() will generate an error.

All results are returned relative to the output reference frame that you set using MmetControl() with M_OUTPUT_FRAME.

Parameters

ResultId

Specifies the identifier of the metrology result buffer from which to get results.

LabelOrIndex

Specifies the feature or geometric tolerance for which to get results. Labels must be greater than 0 while indices can be equal to 0.

To specify a context, feature, geometric tolerance, or result buffer, select the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a context, feature, tolerance, or result buffer																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_INDEX(AIL_INT FeatureIndex)	Specifies the index value of an existing individual feature for which to get results.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

	FeatureIndex This parameter specifies the index of the individual feature about which to inquire. You can set this parameter to the following:														
	Value >= 0 — Specifies the index of the individual feature. Note that 0 is the index of the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL(AIL_INT FeatureLabel)	Specifies the label of an existing individual feature for which to get results.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	FeatureLabel This parameter specifies the label of the individual feature about which to inquire. You can set this parameter to the following:														
	Value > 0 — Specifies the label of the individual features.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_INDEX(AIL_INT ToleranceIndex)	Specifies the index value of an existing individual tolerance for which to get results.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	ToleranceIndex This parameter specifies the index of the individual tolerance about which to inquire. You can set this parameter to the following:														
	Value >= 0 — Specifies the index of the individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_LABEL(AIL_INT ToleranceLabel)	Specifies the label value of an existing individual tolerance for which to get results.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														

	ToleranceLabel This parameter specifies the label of the individual tolerance about which to inquire. You can set this parameter to the following:															
	Value > 0 Specifies the label of the individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_FEATURES	Specifies to return results about all features. This should be used only with M_STATUS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_TOLERANCES	Specifies to return results about all tolerances. This should be used only with M_STATUS.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONSTRUCTED_FEATURES	Specifies to return results about all constructed features. This should be used only with M_STATUS.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies that the results relating to the entire metrology context will be returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_FRAME	Specifies that information about the global frame of the context will be returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEASURED_FEATURES	Specifies to return results about all measured features. This should be used only with M_STATUS.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the first element of the array in which to write the requested information.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For retrieving general results
- For retrieving results for features
- For retrieving results for tolerances
- For retrieving results for features or tolerances

To retrieve general results, the LabelOrIndex parameter must be set to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving general results																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_NUMBER_OF_CONSTRUCTED_FEATURES +	Retrieves the number of constructed features.																
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FEATURES +	Retrieves the total number of features. This total includes both measured features and constructed features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FEATURES_FAIL +	Retrieves the number of features that are not successfully calculated. The status of those features is M_FAIL.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FEATURES_PASS +	Retrieves the number of features that are successfully calculated. The status of those features is M_PASS.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FEATURES_WARNING +	Retrieves the number of features that are calculated with a warning. The status of those features is M_WARNING.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_MEASURED_FEATURES +	Retrieves the number of measured features.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TOLERANCES +	Retrieves the number of tolerances.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TOLERANCES_FAIL +		Retrieves the number of tolerances that have the status M_FAIL, which is the resulting status when tolerance values are smaller than M_VALUE_MIN or greater than M_VALUE_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TOLERANCES_PASS +		Retrieves the number of tolerances that have the status M_PASS, which is the resulting status when it is not M_FAIL or M_WARNING.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TOLERANCES_WARNING +		Retrieves the number of tolerances that have the status M_WARNING, which is the resulting status when tolerance values are greater or equal to M_VALUE_MIN and lower than M_VALUE_WARNING_MIN. It also occurs when tolerance values are lower or equal to M_VALUE_MAX and greater than M_VALUE_WARNING_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT_END +		Retrieves whether the timeout has been reached. You can set the timeout limit using MmetControl() with M_TIMEOUT. By default, there is no limit.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the timeout has not been reached.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the timeout has been reached.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results for features, the LabelOrIndex parameter can be set to M_GLOBAL_FRAME or an existing feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results for features																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																

M_ANGLE +	Retrieves the aperture of an arc or the angle of a segment, a line, or a local frame. The angle is measured in degrees, relative to the output coordinate system specified using MmetControl() with M_RESULT_OUTPUT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.														
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE_END +	Retrieves the end angle of an arc. The angle is measured in degrees, relative to the output coordinate system specified using MmetControl() with M_RESULT_OUTPUT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE_START +	Retrieves the start angle of an arc. The angle is measured in degrees, relative to the output coordinate system specified using MmetControl() with M_RESULT_OUTPUT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_COVERAGE +	Retrieves the portion of the feature that was covered by the fitted edgels, as a percentage. If the fitted edgels of the feature completely cover all of the possible edgels of the feature (before the fit), M_COVERAGE returns 100%. You can retrieve M_COVERAGE for physically measured or constructed arcs, circles, and segments. For features that have not been fit, the coverage is 100%.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH +	Retrieves the perimeter of a circle or the length of a segment, edgel, or arc feature.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_A +	Retrieves the coefficient A of the equation for a constructed line. The line equation is: $A\,x + B\,y + C = 0$.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_B +	Retrieves the coefficient *B* of the equation for a constructed line. The line equation is: $A\,x + B\,y + C = 0$.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_C +	Retrieves the coefficient *C* of the equation for a constructed line. The line equation is: $A\,x + B\,y + C = 0$.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER +	Retrieves the number of subfeatures in a multiple feature or the number of edgels in an edgel feature.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_GROUPS +	Retrieves the number of groups in the group edgels feature (MmetAddFeature() with M_GROUP_EDGELS).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_END_X +	Retrieves the X-coordinate of the ending point of a segment or arc.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_END_Y +	Retrieves the Y-coordinate of the ending point of a segment or arc.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_START_X +	Retrieves the X-coordinate of the starting point of a segment or arc.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_START_Y +	Retrieves the Y-coordinate of the starting point of a segment or arc.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X +	Retrieves the X-coordinate of a point or edgel, the X-coordinate of the center of a circle or arc, or the X-coordinate of the origin of a local frame. For measured points and edgels, you must retrieve M_NUMBER to allocate the appropriate size of the array.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_POSITION_Y +	Retrieves the Y-coordinate of a point or edgel, the Y-coordinate of the center of a circle or arc, or the Y-coordinate of the origin of a local frame. For measured points and edgels, you must retrieve M_NUMBER to allocate the appropriate size of the array.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_RADIUS +	Retrieves the radius of a circle or arc feature.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve tolerance results, the LabelOrIndex parameter can be set to an existing tolerance label or index using M_TOLERANCE_LABEL() or using M_TOLERANCE_INDEX().

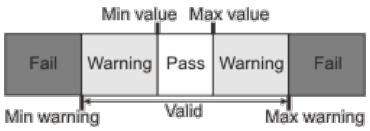
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results for tolerances																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TOLERANCE_VALUE +	Retrieves the calculated geometric tolerance value.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results for features, the LabelOrIndex parameter can be set to M_GLOBAL_FRAME, or an existing feature label or index, using M_FEATURE_LABEL() or M_FEATURE_INDEX(), unless otherwise specified. To retrieve tolerance results, the LabelOrIndex parameter can be set to an existing tolerance label or index, using M_TOLERANCE_LABEL() or M_TOLERANCE_INDEX(), unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results for features or tolerances																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_LABEL_VALUE +	Retrieves the label of a specific feature or a specific tolerance.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_STATUS +		Retrieves the status of the calculation of features and tolerances. The feature's status is determined by whether Aurora Imaging Library was able to successfully calculate it. Some features might be mathematically valid, but their unconventional design can prove problematic. For example, you can theoretically build a valid segment feature using two points at the same location; however, you could not create an extended intersection from such a segment. Aurora Imaging Library considers such a feature to be in a degenerated state and returns its status as M_WARNING. Features that are impossible to construct will have an M_FAIL status; for example, you cannot build a line using two points at the same location (more information is needed to fulfill the requirement that a line extends indefinitely along a directional path). The geometric tolerance's status is determined by whether it (M_TOLERANCE_VALUE) adheres to the thresholds set using MmetControl() with M_VALUE_MAX, M_VALUE_MIN, M_VALUE_WARNING_MAX, and M_VALUE_WARNING_MIN. The following is an example of (non-zero) minimum and maximum tolerance value and warning value thresholds:  To get a global status (the status of multiple statuses), set the LabelOrIndex parameter to M_GENERAL (to get the global status of all features and tolerances), M_ALL_FEATURES (to get the global status of all features), M_CONSTRUCTED_FEATURES (to get the global status of all constructed features), M_MEASURED_FEATURES (to get the global status of all measured features), or M_ALL_TOLERANCES (to get the global status of all tolerances). When retrieving a global status, M_STATUS only returns M_PASS when each individual status is M_PASS. If any individual status is M_FAIL, M_STATUS returns M_FAIL. If there is no individual status with M_FAIL, but one or more with M_WARNING, then M_STATUS returns M_WARNING.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FAIL	Specifies that the feature was not calculated successfully, and/or the tolerance has not passed all restrictions.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PASS	Specifies that the feature was calculated successfully, and/or the geometric tolerance has passed all restrictions.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WARNING	Specifies that the feature was calculated, or the geometric tolerance has passed all restrictions, with a warning. Warnings can be used to advise you that a feature is in a degenerated state, or that a tolerance is at its limit, and is in danger of failing.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for

Tables:

- For retrieving general results
- For retrieving results for features
- For retrieving results for tolerances
- For retrieving results for features or tolerances

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For retrieving general results
- For retrieving results for features
- For retrieving results for tolerances
- For retrieving results for features or tolerances

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetInquire

Synopsis

Inquire information about a metrology context, a derived metrology region object, features, tolerances, or a result buffer.

Syntax

```
AIL_INT MmetInquire(
    AIL_ID MetId,          //in
    AIL_INT LabelOrIndex,  //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

Description

This function inquires information about a metrology context, a derived metrology region object, features, tolerances, or a result buffer. To obtain metrology result buffer values, use MmetGetResult().

If the inquired setting is set to M_DEFAULT (for example, in MmetControl()), **MmetInquire**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

Parameters

MetId

Specifies the identifier of the metrology context, the derived metrology region object, or the result buffer about which to inquire information. The metrology context or derived metrology region object must have been previously allocated on the required system using MmetAlloc(). The metrology result buffer must have been previously allocated on the required system using MmetAllocResult().

LabelOrIndex

Specifies that information will be inquired about the metrology context, the derived metrology region object, features, tolerances, or the result buffer. This parameter should be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to inquire																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_INDEX(AIL_INT FeatureIndex)	Specifies the index value of an existing individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureIndex This parameter specifies the index of the individual feature about which to inquire. You can set this parameter to the following:															

		Value >= 0 Specifies the index of the individual feature. Note that 0 is the index of the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL(AIL_INT FeatureLabel)	Specifies the label value of an existing individual feature about which to inquire.		c		e	f	g	a		i		j	k	l	m	n	b
	Parameters																
	FeatureLabel This parameter specifies the label of the individual feature about which to inquire. You can set this parameter to the following:																
		Value > 0 Specifies the label of the individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_INDEX(AIL_INT ToleranceIndex)	Specifies the index value of an existing individual tolerance.		c		e	f	g	a		i		j	k	l	m	n	b
	Parameters																
	ToleranceIndex This parameter specifies the index of the individual tolerance about which to inquire. You can set this parameter to the following:																
		Value >= 0 Specifies the index of the individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_LABEL(AIL_INT ToleranceLabel)	Specifies the label value of an existing individual tolerance.		c		e	f	g	a		i		j	k	l	m	n	b
	Parameters																
	ToleranceLabel This parameter specifies the label of the individual tolerance about which to inquire. You can set this parameter to the following:																

			Value > 0	Specifies the label of the individual tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Inquires information about the metrology context, which has been set using the MetId parameter.				c		e	f	g	a		i		j	k	l	m	n	b
M_DERIVED_GEOMETRY_REGION	Inquires information about the derived metrology region object, which has been set using the MetId parameter.				c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Inquires information about a setting of a metrology result buffer. This setting applies to all features and tolerances in the metrology result buffer. When using M_GENERAL, MetId must specify a metrology result buffer.				c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_FRAME	Inquires information about the global frame of the metrology template.				c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MmetInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring general context settings
- For inquiring about the system
- For inquiring global processing settings
- For inquiring about features
- For inquiring about constructed edgel features

- For inquiring the geometry of the metrology region
- For inquiring the geometry data of the metrology region
- For inquiring about geometric tolerances
- For inquiring label or index information about a feature or geometric tolerance
- For inquiring about features or geometric tolerances
- For inquiring clone transformations settings
- For inquiring result buffer settings

To inquire about general metrology context settings, set the InquireType parameter to one of the following values. To inquire general context settings, the LabelOrIndex parameter must be set to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring general context settings																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_ASSOCIATED_CALIBRATION +		Inquires the camera calibration context associated with the template reference of the metrology context. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that there is no camera calibration context associated with the template reference.	c		e	f	g	a		i		j	k	l	m	n	b
	Calibration context identifier	Specifies the identifier of the camera calibration context that is associated with the template reference of the metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFICATION_COUNT +		Inquires the current value of the modification counter. The modification counter is increased by one each time settings for the context are modified. Although you cannot identify the modification counter's contents, you can compare them throughout your application to know if the context has been altered. If the modification counter has changed you can, for example, prompt the user to save before closing the application.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the object modification count.	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER +	Inquires the number of external edgels/points. External edgels/points come from MmetPut(), which puts them into the constructed external edgel feature (MmetAddFeature() with M_EDGEL and M_EXTERNAL_FEATURE).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of external edgels/points.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_CONSTRUCTED_FEATURES +	Inquires the number of constructed features in the template of the metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of constructed features.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FEATURES +	Inquires the total number of features in the template of the metrology context. This total includes both measured features and constructed features.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the total number of features.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_MEASURED_FEATURES +	Inquires the number of measured features in the template of the metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of measured features.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TOLERANCES +	Inquires the number of geometric tolerances in the template of the metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of geometric tolerances.	c		e	f	g	a		i		j	k	l	m	n	b

M_TEMPLATE_REFERENCE_ID +	Inquires the identifier of the template reference (image) that was added to the context. Note that this is the identifier of an internal copy of the buffer specified when calling MmetControl() with M_TEMPLATE_REFERENCE_ID. You should not try to free this internal buffer using MbufFree(). If you want to disassociate it from the metrology context, call MmetControl() with M_TEMPLATE_REFERENCE_ID set to M_NULL. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that there is no template reference image associated with the context.	c		e	f	g	a		i		j	k	l	m	n	b
Image identifier	Specifies the identifier of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_REFERENCE_SIZE_BAND +	Inquires the number of bands in the template reference image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 3	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_REFERENCE_SIZE_X +	Inquires the width of the template reference.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the template reference's width. This value must be in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_REFERENCE_SIZE_Y +	Inquires the height of the template reference.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the template reference's height. This value must be in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_REFERENCE_TYPE +	Inquires the data type and the data depth (per band) of the template reference.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the template reference's data type and data depth.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT +		Inquires the maximum measurement-and-validation time for MmetCalculate(), in msec. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies an infinite amount of measurement and validation time.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum measurement and validation time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the system on which either the metrology context or metrology result buffer has been allocated, set the InquireType parameter to the value below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the system																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_OWNER_SYSTEM +		Inquires the identifier of the system on which either the metrology context or metrology result buffer has been allocated. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the processing settings for a measured feature, set the InquireType parameter to one of the following values. To inquire global processing settings, the LabelOrIndex parameter must be set to an existing measured feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring global processing settings																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																

M_CHAIN_ALL_NEIGHBORS +	Inquires whether edge chains are built with as much edgel information as possible. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that edge chains are built with the least amount of edgel information possible. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that edge chains are built with as much edgel information as possible. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRACTION_SCALE +	Inquires the image scale at which to extract the edges. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the extraction scale.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_GAP_ANGLE +	Inquires the aperture angle where the feature extraction searches for edge extremity candidates when filling gaps in features. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 90.0 °.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the aperture angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILL_GAP_DISTANCE +	Inquires the maximum distance radius where the feature extraction searches for edge extremity candidates when filling gaps in features. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies an infinite maximum distance radius.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum distance radius, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_SMOOTHNESS +	Inquires the degree of smoothness (strength of denoising) of the edge extraction filter. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b

0.0 <= *Value* <= 100.0	Specifies the smoothness value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_TYPE +	Inquires the type of filter used when performing the edge extraction. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SHEN.	c		e	f	g	a		i		j	k	l	m	n	b
M_DERICHE	Specifies a Deriche infinite support filter.	c		e	f	g	a		i		j	k	l	m	n	b
M_FREI_CHEN	Specifies a Frei Chen filter.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREWITT	Specifies a Prewitt filter.	c		e	f	g	a		i		j	k	l	m	n	b
M_SHEN	Specifies a Shen-Castan infinite support exponential filter.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOBEL	Specifies a Sobel filter.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOAT_MODE +	Inquires the mode in which to perform the edge extraction filter. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that all edge extractions are not forced to be performed using floating-point precision calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that all edge extractions are forced to be performed using floating-point precision calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAGNITUDE_TYPE +	Inquires the type of magnitude value used to calculate the magnitude of the edge at each edgel position. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SQR_NORM.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORM	Specifies that the magnitude will be used.	c		e	f	g	a		i		j	k	l	m	n	b
M_SQR_NORM	Specifies that the square of the magnitude will be used.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ACCURACY_HIGH +	Inquires the accuracy with which you define the metrology region associated with a feature when dealing with a calibrated image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that high accuracy is not used when defining a metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that high accuracy is used when defining a metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE +		Inquires the threshold mode of the edge extraction. Note that lower threshold values result in a more sensitive edgel detection. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_HIGH.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies no threshold. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_HIGH	Specifies a high threshold. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies a low threshold. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium threshold. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies that the threshold values will be user-defined. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_HIGH	Specifies a very high threshold. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_TYPE +		Inquires the type of the hysteresis threshold used when performing the edge extraction. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_HYSTERESIS.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FULL_HYSTERESIS	Specifies that the lower bound threshold value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_HYSTERESIS	Specifies that both the lower bound threshold value and the upper bound threshold value will be used.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_HYSTERESIS	Specifies that the lower bound threshold value is equal to the upper bound threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE_HIGH +		Inquires the user-defined upper bound of the hysteresis threshold. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the upper bound of the hysteresis threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE_LOW +	Inquires the user-defined lower bound of the hysteresis threshold. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the lower bound of the hysteresis threshold.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the features of a metrology context, set the InquireType parameter to one of the following values. For inquiring feature settings, the LabelOrIndex parameter can be set to one of the following: an existing feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX(), or M_GLOBAL_FRAME.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about features																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ACTIVE +	Inquires whether the feature is enabled for calculation. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE_END +	Inquires the end angle of a constructed arc. The angle is relative to the reference frame's axis. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 360.0 °.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE_START +	Inquires the start angle of a constructed arc. The angle is relative to the reference frame's axis. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies the default value; the default value is 0.0 °.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* < 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPENDENCIES +	Inquires whether other features or tolerances have been constructed or validated based on the specified feature.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that other features or tolerances have not been constructed or validated based on the specified feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that other features or tolerances have been constructed or validated based on the specified feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_ANGLE_RANGE +	Inquires the angular range within which an edgel's gradient angle must fall for Aurora Imaging Library to consider it an active edgel. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 180.0 °.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angular range, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_DENOISING_MODE +	Inquires how to denoise edgels. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies no denoising.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAUSSIAN	Specifies a Gaussian type of denoising.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN	Specifies a mean type of denoising.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIAN	Specifies a median type of denoising.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_DENOISING_RADIUS +	Inquires the radius by which to denoise edgels. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the radius, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_DENOISING_USE_ORDER +		Inquires whether Aurora Imaging Library should use and maintain the order of the edgels when denoising them. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that Aurora Imaging Library should not use and maintain the order of the edgels when denoising them.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that Aurora Imaging Library should use and maintain the order of the edgels when denoising them. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_RELATIVE_ANGLE +		Inquires the relative angle from which to measure the gradient angle range (M_EDGEL_ANGLE_RANGE). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SAME.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REVERSE	Specifies that the gradient angle range is measured relative to the reverse angle (orientation) of the region of interest (+ 180°).	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME	Specifies that the gradient angle range is measured relative to the same angle (orientation) as the region of interest.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME_OR_REVERSE	Specifies that the gradient angle range is measured relative to either the same or the reverse angle (orientation) of the region of interest.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_SELECTION_RANK +		Inquires which edgels to select in each column of the metrology region. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies that all edgels are selected.	c		e	f	g	a		i		j	k	l	m	n	b
M_LAST	Specifies the last edgel in each column, relative to the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies which edgels are selected , based on their rank, relative to the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_TYPE +	Inquires the type of edgels that will be used for constructed edgel features. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACTIVE_EDGELS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACTIVE_EDGELS	Specifies to use the active edgels of the measured feature's metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_EDGELS	Specifies to use all the edgels of the measured feature's metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_FITTED_EDGELS	Specifies to use the fitted edgels of the measured feature's metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_GEOMETRY +	Inquires the geometry of the specified feature. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c	d	e	f	g	a	h	i		j	k	l	m	n	b
M_ARC	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
	//For constructed features Specifies a constructed arc feature.	c		e	f	g	a		i		j	k	l	m	n	b
	//For measured features Specifies a physically measured arc feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCLE	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
	//For constructed features Specifies a constructed circle feature.	c		e	f	g	a		i		j	k	l	m	n	b

		//For measured features Specifies a physically measured circle feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_EDGEL	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
		//For constructed features Specifies a constructed edgel feature.	c		e	f	g	a		i		j	k	l	m	n	b
		//For measured features Specifies a physically measured edgel feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LINE	//For constructed features Specifies a constructed line feature.	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_LOCAL_FRAME	//For constructed features Specifies a constructed local frame feature.	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_POINT	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
		//For constructed features Specifies a constructed point feature.	c		e	f	g	a		i		j	k	l	m	n	b
		//For measured features Specifies a physically measured point feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEGMENT	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
		//For constructed features Specifies a constructed segment feature.	c		e	f	g	a		i		j	k	l	m	n	b
		//For measured features Specifies a physically measured segment feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_TYPE +		Inquires the type of the specified feature. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONSTRUCTED	Specifies a constructed feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEASURED	Specifies a physically measured feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_FIT_COVERAGE_MIN +	Inquires the approximate portion of the feature that must be covered by fitted edgels for a successful fit. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the minimum coverage, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_DISTANCE_MAX +	Inquires the greatest possible gap between an active edgel and an iterative fit for the active edgel to be considered during the next iteration. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies no maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum distance, in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_DISTANCE_OUTLIERS +	Inquires whether to automatically determine the distance that establishes outliers (edgels/points), or to use a user-defined distance. Outliers are excluded from the robust best fit. This only applies to features built using MmetAddFeature() with M_FIT; also, M_FIT_TYPE must be set to M_ROBUST_FIT. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that Aurora Imaging Library automatically determines the distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies that the distance is set with M_FIT_DISTANCE_OUTLIERS_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIT_DISTANCE_OUTLIERS_VALUE +	Inquires the explicit distance value that Aurora Imaging Library uses to establish outliers (outlier edgels/points), which are excluded from the robust best fit. This only applies to features built using MmetAddFeature() with M_FIT; also, M_FIT_TYPE must be set to M_ROBUST_FIT, and M_FIT_DISTANCE_OUTLIERS must be set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 4.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the distance; value must be greater than 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_INTERNAL_NUMBER_OF_POINTS +	Inquires the number of points from which Aurora Imaging Library performs the robust fit to build the feature. This only applies to features built using MmetAddFeature() with M_FIT; also, M_FIT_TYPE must be set to M_ROBUST_FIT. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that Aurora Imaging Library automatically determines the number of points. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of points. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_ITERATIONS_MAX +	Inquires the maximum number of fit iterations used to compute the feature. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the maximum number of fit iterations will be automatically determined.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the maximum number of fit iterations. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_FIT_TYPE +	Inquires the type of best fit with which to build the feature. This only applies to features built using MmetAddFeature() with M_FIT. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_STANDARD_FIT.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROBUST_FIT	Specifies a robust best fit. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STANDARD_FIT	Specifies a standard best fit. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_VARIATION_MAX +	Inquires the maximum allowable difference in the value of the feature's underlying coefficients, from one fit iteration to the next. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the maximum variation will be determined automatically.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the maximum variation, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_A +	Inquires the coefficient *A* of the equation for a parametrically constructed line. The line equation is: $A\,x + B\,y + C = 0$. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the coefficient's value.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_B +	Inquires the coefficient *B* of the equation for a parametrically constructed line. The line equation is: $A\,x + B\,y + C = 0$. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the coefficient's value.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_BISECTOR_TYPE +		Inquires how to construct a line feature when building it as a bisector. This only applies when using MmetAddFeature() with M_CONSTRUCTED set to M_LINE and M_BISECTOR, and specifying two lines as the base features. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_FEATURE_ORDER.	c		e	f	g	a		i		j	k	l	m	n	b
	M_BIGGEST_ANGLE	Specifies that the newly constructed line is built at the middle of the biggest angle formed by the two lines specified as base features.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_ORDER	Specifies that the newly constructed line is built according to the order in which you specified the two line base features; that is, at the middle of the angle formed between the first and second specified lines.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SMALLEST_ANGLE	Specifies that the newly constructed line is built at the middle of the smallest angle formed by the two lines specified as base features.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE_C +		Inquires the coefficient *C* of the equation for a parametrically constructed line. The line equation is: $A\,x + B\,y + C = 0$. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	-0.5 * *AIL_INT32_MAX* <= *Value* <= 0.5 * *AIL_INT32_MAX*	Specifies the coefficient's value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_MAX +		Inquires the maximum number of subfeatures to calculate in a multiple feature (for point features only). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the maximum number of subfeatures. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_MIN +	Inquires the minimum number of subfeatures to calculate in a multiple feature (for point features only). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the minimum number of subfeatures. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OCCURRENCE +	Inquires the point to use if there is more than one candidate when adding a constructed point feature built at the intersection of two base features (MmetAddFeature() with M_POINT and M_INTERSECTION or M_EXTENDED_INTERSECTION). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPERATION +	Inquires the operation used to build the specified feature. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c	d	e	f	g	a	h	i		j	k	l	m	n	b
M_DEFAULT	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
	//For a constructed feature Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	//For a constructed feature Same as M_COPY_FEATURE_EDGELS.	c		e	f	g	a		i		j	k	l	m	n	b
	//For a measured feature Same as M_FIT.	c		e	f	g	a		i		j	k	l	m	n	b
	//For a measured feature Specifies a measured edgel feature built by extracting all the edgels in the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ANGLE	//For a constructed feature Specifies a line feature built at a specified angle from the specified linear base feature (segment or line), and passing through the specified point base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_ANGLE_ABSOLUTE	//For a constructed feature Specifies a point feature built at the specified absolute angle, on the contour of the specified base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_ANGLE_RELATIVE	//For a constructed feature Specifies a point feature built at the specified relative angle, on the contour of the specified base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_BISECTOR	//For a constructed feature Specifies a line feature built by dividing an angle formed by the specified base features. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_CENTER	//For a constructed feature Specifies a point feature built at the center of the specified base feature(s). INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_CLONE_FEATURE	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies an arc feature cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a circle feature cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies an edgel feature cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a line feature cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a local frame feature cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a point feature cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b

		//For a constructed feature Specifies a segment feature cloned from the specified base feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CLOSEST	//For a constructed feature Specifies a point feature built at the point on the first specified base feature that is, by default, closest to the other (one or more) specified base features. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_CLOSEST_TO_INFINITE_POINT	//For a constructed feature Specifies a point feature built, at the position on the base feature, that is the closest to an infinite point. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_CONSTRUCTION	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies an arc feature built from a geometric construction using the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a circle feature built from a geometric construction using the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a line feature built from a geometric construction using the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a local frame feature built from a geometric construction using one or two specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a segment feature built from a geometric construction using the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COPY_FEATURE_EDGELS	//For a constructed feature Specifies an edgel feature constructed by copying the edgels of the specified base features. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_EXTENDED_INTERSECTION	//For a constructed feature Specifies a point feature built at the point of intersection between the extension of the first and second specified base features. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b

	M_EXTERNAL_FEATURE	//For a constructed feature Specifies an edgel feature constructed from edgels or points established using other Aurora Imaging Library modules or third party software. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_FIT	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies an arc feature built from the arc that best fits the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a circle feature built from the circle that best fits the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a line feature built from the resulting best fit of the specified base feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a segment feature built from the line that best fits the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a measured feature Specifies a measured arc feature built using a best fit operation applied to the data (edgels) extracted from the metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a measured feature Specifies a measured circle feature built using a best fit operation applied to the data (edgels) extracted from the metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a measured feature Specifies a measured point feature built using a best fit operation applied to the data (edgels) extracted from the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a measured feature Specifies a measured segment feature built from the line that best fits the data (edgels) extracted from the metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_GROUP_EDGELS	//For a constructed feature Specifies an edgel feature constructed by grouping the edgels of the specified base features. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_INNER_FIT	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a circle feature built from the best inner fit of a circle around the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a segment feature built from the segment that has the best inner fit in the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a measured feature Specifies a measured circle feature built using the best inner fit operation applied to the data (edgels) extracted from the metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a measured feature Specifies a measured segment feature built from the segment that has the best inner fit applied to the data (edgels) extracted from the metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERSECTION	//For a constructed feature Specifies a point feature built at the point of intersection between the first and second specified base features. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_MAX_DISTANCE_POINT	//For a constructed feature Specifies a point feature built at the point on the first specified base feature that is, by default, farthest from any one point on the other (one or more) specified base features. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_MAX_OF_MIN_DISTANCE_POINT	//For a constructed feature Specifies a point feature built at the point on the first specified base feature that is the farthest of all the closest points on the second base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_MIDDLE	//For a constructed feature Specifies a point feature built at the midpoint of the contour (edge) of the specified base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_OUTER_FIT	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b

		//For a constructed feature Specifies a circle feature built from the best outer fit of a circle around the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a segment feature built from the outer fit of the specified base features. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a measured feature Specifies a measured circle feature built using the best outer fit operation applied to the data (edgels) extracted from the metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		//For a measured feature Specifies a measured segment feature built from the segment that has the best outer fit applied to the data (edgels) extracted from the metrology region. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARALLEL	//For a constructed feature Specifies a line feature built parallel to the specified linear base feature (segment or line), and passing through the specified point base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_PARAMETRIC	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies an arc feature built from the following MmetControl() parametric controls: M_POSITION_X, M_POSITION_Y, M_ANGLE_START, M_ANGLE_END, and M_RADIUS.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a circle feature built from the following MmetControl() parametric controls: M_POSITION_X, M_POSITION_Y, and M_RADIUS.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a line feature built from the following MmetControl() parametric controls: M_LINE_A, M_LINE_B, and M_LINE_C.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a local frame feature built from the following MmetControl() parametric controls: M_POSITION_X, M_POSITION_Y, and M_ANGLE. INFO	c		e	f	g	a		i		j	k	l	m	n	b

		//For a constructed feature Specifies a point feature built from the following MmetControl() parametric controls: M_POSITION_X and M_POSITION_Y.	c		e	f	g	a		i		j	k	l	m	n	b
		//For a constructed feature Specifies a segment feature built from the following MmetControl() parametric controls: M_POSITION_START_X, M_POSITION_START_Y, M_POSITION_END_X, and M_POSITION_END_Y.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PERPENDICULAR	//For a constructed feature Specifies a line feature built perpendicular to the specified linear base feature (segment or line), and passing through the specified point base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_POSITION_ABSOLUTE	//For a constructed feature Specifies a point feature built at the specified position on the contour of the specified base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_POSITION_END	//For a constructed feature Specifies a point feature built at the end of the contour of the specified base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_POSITION_RELATIVE	//For a constructed feature Specifies a point feature built at the specified relative position on the contour of the specified base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_POSITION_START	//For a constructed feature Specifies a point feature built at the start position of the contour of the specified base feature. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
	M_SECTOR_RELATIVE	//For a constructed feature Specifies a line feature built where it divides, in two parts, an angle formed by the specified base features. INFO	c	d	e	f	g	a	h	i		j	k	l	m	n	b
M_POSITION +		Inquires the position, along the contour of the specified base feature, at which to construct the point feature (MmetAddFeature() with with M_POINT and M_POSITION_ABSOLUTE or M_POSITION_RELATIVE). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the position value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_END_X +	Inquires the X-coordinate of the end point of a parametrically constructed segment. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_END_Y +	Inquires the Y-coordinate of the end point of a parametrically constructed segment. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_START_X +	Inquires the X-coordinate of the start point of a parametrically constructed segment. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is -1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_START_Y +	Inquires the Y-coordinate of the start point of a parametrically constructed segment. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b

M_POSITION_X +	Inquires the X-coordinate of the center of a constructed circle or arc, or the X-coordinate of the position of a constructed local frame or point. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_Y +	Inquires the Y-coordinate of the center of a constructed circle or arc, or the Y-coordinate of the position of a constructed local frame or point. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_RADIUS +	Inquires the radius of a constructed circle or arc. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the radius, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFERENCE_FRAME +	Inquires the reference frame for any feature. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that the reference frame is the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT +	Inquires the sorting order that Aurora Imaging Library applies to the index of subpoints, when retrieving results of multiple point features. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SORT_UP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SORT_DOWN	Specifies that results of multiple point features are sorted in descending order, according to the subpoint's index.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SORT_UP	Specifies that results of multiple point features are sorted in ascending order, according to the subpoint's index.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about constructed edgel features, set the InquireType parameter to one of the following values. In this case, the LabelOrIndex parameter must be set to the label or index of the feature, using M_FEATURE_LABEL() or M_FEATURE_INDEX(), and the MetId parameter must be set to a metrology context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about constructed edgel features																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_EDGEL_ANGLE +		Inquires the gradient angles (edgel direction) of the edgels of the edgel feature, constructed from externally established edgels/points (using MmetPut()). If the edgels of the edgel feature were constructed from externally established points (MmetPut() with only X and Y coordinates) and their gradient angle was not calculated (M_INTERPOLATE_ANGLE was not specified), the angle of these edgels is returned as M_DISABLE. SET															
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that no angle was specified.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the angles.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_POSITION_X +		Inquires the X-coordinates of the edgels of the edgel feature, constructed from externally established edgels/points (using MmetPut()). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinates, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEL_POSITION_Y +	Inquires the Y-coordinates of the edgels of the edgel feature, constructed from externally established edgels/points (using MmetPut()). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinates, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_PROVIDED_ORDER +	Inquires whether the externally established egdels/points used to construct the edgel feature are listed in a meaningful order. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies unordered edgels/points.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEQUENTIAL	Specifies ordered edgels/points. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_RESAMPLING_LINE_ANGLE +	Inquires the angle of a line along which to resample edgels. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable resampling along a line. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the angle of the line along which to resample edgels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_RESAMPLING_MODE +	Inquires how to resample edgels. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies no resampling.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN	Specifies a mean type of resampling.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIAN	Specifies a median type of resampling.	c		e	f	g	a		i		j	k	l	m	n	b

M_EDGEL_RESAMPLING_RADIUS +	Inquires the radius by which to resample edgels. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the radius.	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGEL_RESAMPLING_USE_ORDER +	Inquires whether Aurora Imaging Library should use and maintain the order of the edgels when resampling them. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that Aurora Imaging Library should not use and maintain the order of the edgels then resampling them.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that Aurora Imaging Library should use and maintain the order of the edgels when resampling them. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUP_EDGEL_GAP_SIZE +	Inquires the maximum allowable gap (distance) between edgels for Aurora Imaging Library to consider them part of the same group. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that Aurora Imaging Library should automatically determine the gap size.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the gap size, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUP_MIN_NUMBER_OF_EDGELS +	Inquires the minimum number of edgels a group must have. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies that there is no minimum number of edgels that a group must have.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the minimum number of edgels that a group must have.	c		e	f	g	a		i		j	k	l	m	n	b

M_GROUP_SELECTION_MODE +	Inquires how to group edgels. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_OCCURRENCE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLOSEST_TO_POINT	Specifies to select one group, based on its proximity to a point or the local frame's origin. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OCCURRENCE	Specifies to select one or all group(s), based on the index of the group of edgels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUP_SELECTION_OCCURRENCE +	Inquires whether to use one or all edgels to establish the group. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to use all groups of edgels.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the index of the group of edgels to select. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GROUP_SELECTION_POINT +	Inquires the point from which Aurora Imaging Library takes the distance to group edgels. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_GLOBAL_FRAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_FRAME	Specifies a virtual point at the location (origin) of the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of an existing point feature or local frame feature.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire the geometry of the metrology region, set the InquireType parameter to the following value. In this case, the LabelOrIndex parameter can be set to an existing measured feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX() (for an explicitly-defined or 2D graphics list metrology region) or M_DERIVED_GEOMETRY_REGION (for a derived metrology region), and the MetId parameter can be set to a metrology context or a derived metrology region object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the geometry of the metrology region																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_REGION_GEOMETRY +		Inquires the geometry of the metrology region. SET SET														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ARC	Specifies an arc region.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies an infinite region.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RECTANGLE	Specifies a rectangle region.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RING	Specifies a ring-shaped region.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RING_SECTOR	Specifies a ring-sector region.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEGMENT	Specifies a linear segment region.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire the geometry data required to define a metrology region, set the InquireType parameter to one of the following values. In this case, the LabelOrIndex parameter can be set to an existing measured feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX() (for an explicitly-defined or 2D graphics list metrology region) or M_DERIVED_GEOMETRY_REGION (for a derived metrology region), and the MetId parameter can be set to a metrology context or a derived metrology region object, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the geometry data of the metrology region																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_REGION_ANGLE +		Inquires the angle of the metrology region. M_REGION_ANGLE applies to M_INFINITE and M_RECTANGLE metrology regions. SET SET SET SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the angle. For an explicitly-defined or 2D graphics list metrology region, M_REGION_ANGLE returns an explicit angle value, in degrees. For a derived metrology region, use M_REGION_ANGLE_TYPE to determine whether M_REGION_ANGLE returns the label of the feature used to establish the angle, or an explicit angle value, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_REGION_ANGLE_END +	Inquires the end angle of the metrology region. M_REGION_ANGLE_END applies to M_ARC and M_RING_SECTOR metrology regions. SET SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the end angle. For an explicitly-defined or 2D graphics list metrology region, M_REGION_ANGLE_END returns an explicit angle value, in degrees. For a derived metrology region, use M_REGION_ANGLE_END_TYPE to determine whether M_REGION_ANGLE_END returns the label of the feature used to establish the angle, or an explicit angle value, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ANGLE_END_TYPE +	Inquires whether the end angle of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_ANGLE_END). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ANGLE_START +	Inquires the start angle of the metrology region. M_REGION_ANGLE_START applies to M_ARC and M_RING_SECTOR metrology regions. SET SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the start angle. For an explicitly-defined or 2D graphics list metrology region, M_REGION_ANGLE_START returns an explicit angle value, in degrees. For a derived metrology region, use M_REGION_ANGLE_START_TYPE to determine whether M_REGION_ANGLE_START returns the label of the feature used to establish the angle, or an explicit angle value, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ANGLE_START_TYPE +	Inquires whether the start angle of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_ANGLE_START). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b

	M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ANGLE_TYPE +		Inquires whether the angle of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_ANGLE). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_END +		Inquires the derived metrology region's end point according to the specified feature. M_REGION_END applies to derived metrology regions with an M_SEGMENT geometry. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label of a point feature, which metrology uses to set the end point of the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_END_TYPE +		Inquires whether the end point of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_END or M_REGION_END_X and M_REGION_END_Y). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_END_X +		Inquires the X-coordinate of the metrology region's end point. M_REGION_END_X applies to M_SEGMENT metrology regions. SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the end point's X-coordinate. For an explicitly-defined or 2D graphics list metrology region, M_REGION_END_X returns an explicit coordinate value, in pixel or world units. For a derived metrology region, use M_REGION_END_TYPE to determine whether M_REGION_END_X returns the label of the feature used to establish the end point, or an explicit coordinate value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_END_Y +		Inquires the Y-coordinate of the metrology region's end point. M_REGION_END_Y applies to M_SEGMENT metrology regions. SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the end point's Y-coordinate. For an explicitly-defined or 2D graphics list metrology region, M_REGION_END_Y returns an explicit coordinate value, in pixel or world units. For a derived metrology region, use M_REGION_END_TYPE to determine whether M_REGION_END_Y returns the label of the feature used to establish the end point, or an explicit coordinate value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_HEIGHT +		Inquires the height of the metrology region. M_REGION_HEIGHT applies to M_RECTANGLE metrology regions. SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the height. For an explicitly-defined or 2D graphics list metrology region, M_REGION_HEIGHT returns an explicit height value, in pixel or world units. For a derived metrology region, use M_REGION_HEIGHT_TYPE to determine whether M_REGION_HEIGHT returns the label of the feature used to establish the height, or an explicit height value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_HEIGHT_TYPE +		Inquires whether the height of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_HEIGHT). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b

M_REGION_POSITION +	Inquires the derived metrology region's position according to the specified feature. M_REGION_POSITION applies to derived metrology regions with any geometry except M_SEGMENT. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of a point feature, which metrology uses to set the position of the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_POSITION_TYPE +	Inquires whether the position of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_POSITION or M_REGION_POSITION_X and M_REGION_POSITION_Y). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_POSITION_X +	Inquires the X-coordinate of the metrology region's position. M_REGION_POSITION_X applies to any geometry except M_SEGMENT, unless otherwise specified. SET SET SET SET SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the position's X-coordinate. For an explicitly-defined or 2D graphics list metrology region, M_REGION_POSITION_X returns an explicit coordinate value, in pixel or world units. For a derived metrology region, use M_REGION_POSITION_TYPE to determine whether M_REGION_POSITION_X returns the label of the feature used to establish the position, or an explicit coordinate value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_POSITION_Y +	Inquires the Y-coordinate of the metrology region's position. M_REGION_POSITION_Y applies to any geometry except M_SEGMENT, unless otherwise specified. SET SET SET SET SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the position's Y-coordinate. For an explicitly-defined or 2D graphics list metrology region, M_REGION_POSITION_Y returns an explicit coordinate value, in pixel or world units. For a derived metrology region, use M_REGION_POSITION_TYPE to determine whether M_REGION_POSITION_Y returns the label of the feature used to establish the position, or an explicit coordinate value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS +		Inquires the radius of the metrology region. M_REGION_RADIUS applies to M_ARC. SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the radius. For an explicitly-defined or 2D graphics list metrology region, M_REGION_RADIUS returns an explicit radius value, in pixel or world units. For a derived metrology region, use M_REGION_RADIUS_TYPE to determine whether M_REGION_RADIUS returns the label of the feature used to establish the radius, or an explicit radius value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS_END +		Inquires the end radius defining the metrology region. M_REGION_RADIUS_END applies to M_RING or M_RING_SECTOR. SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the end radius. For an explicitly-defined metrology region, M_REGION_RADIUS_END returns an explicit radius value, in pixel or world units. For a derived metrology region, use M_REGION_RADIUS_END_TYPE to determine whether M_REGION_RADIUS_END returns the label of the feature used to establish the end radius, or an explicit radius value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS_END_TYPE +		Inquires whether the end radius of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_RADIUS_END). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b

M_REGION_RADIUS_START +		Inquires the start radius defining the metrology region. M_REGION_RADIUS_START applies to M_RING or M_RING_SECTOR. SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the start radius. For an explicitly-defined metrology region, M_REGION_RADIUS_START returns an explicit radius value, in pixel or world units. For a derived metrology region, use M_REGION_RADIUS_START_TYPE to determine whether M_REGION_RADIUS_START returns the label of the feature used to establish the start radius, or an explicit radius value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS_START_TYPE +		Inquires whether the start radius of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_RADIUS_START). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_RADIUS_TYPE +		Inquires whether the radius of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_RADIUS). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_START +		Inquires the derived metrology region's start point according to the specified feature. M_REGION_START applies to derived metrology regions with an M_SEGMENT geometry. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label of a point feature, which metrology uses to set the start point of the metrology region.	c		e	f	g	a		i		j	k	l	m	n	b

M_REGION_START_TYPE +	Inquires whether the start point of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_START or M_REGION_START_X and M_REGION_START_Y). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_START_X +	Inquires the X-coordinate of the metrology region's start point. M_REGION_START_X applies to M_SEGMENT. SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the start point's X-coordinate. For an explicitly-defined or 2D graphics list metrology region, M_REGION_START_X returns an explicit coordinate value, in pixel or world units. For a derived metrology region, use M_REGION_START_TYPE to determine whether M_REGION_START_X returns the label of the feature used to establish the start point, or an explicit coordinate value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_START_Y +	Inquires the Y-coordinate of the metrology region's start point. M_REGION_START_Y applies to M_SEGMENT. SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the start point's Y-coordinate. For an explicitly-defined or 2D graphics list metrology region, M_REGION_START_Y returns an explicit coordinate value, in pixel or world units. For a derived metrology region, use M_REGION_START_TYPE to determine whether M_REGION_START_Y returns the label of the feature used to establish the start point, or an explicit coordinate value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_WIDTH +	Inquires the width of the metrology region. M_REGION_WIDTH applies to M_RECTANGLE. SET SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0.0	Specifies the width. For an explicitly-defined or 2D graphics list metrology region, M_REGION_WIDTH returns an explicit width value, in pixel or world units. For a derived metrology region, use M_REGION_WIDTH_TYPE to determine whether M_REGION_WIDTH returns the label of the feature used to establish the width, or an explicit width value, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_WIDTH_TYPE +	Inquires whether the width of the derived metrology region was specified with the label of a feature or an explicit value (M_REGION_WIDTH). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_PARAMETRIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL_VALUE	Specifies a feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARAMETRIC	Specifies an explicit value.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the geometric tolerance settings of a metrology context, set the InquireType parameter to one of the following values. For inquiring geometric tolerances, the LabelOrIndex parameter can be set to an existing tolerance label or index using M_TOLERANCE_LABEL() or M_TOLERANCE_INDEX(). Unless otherwise specified, the following settings apply to any geometric tolerance (MmetAddTolerance()).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about geometric tolerances																	
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AREA_BETWEEN_CURVES_DEFINE_SIDE +	Inquires the reference frame with which to determine the side of the curve of edgels to use to establish the area, when you specify an M_AREA_BETWEEN_CURVES tolerance, and enable the M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY setting. SET																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_GLOBAL_FRAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_INDEX(AIL_INT LocalFrameIndex)	Specifies the label value of a local frame feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters																

		LocalFrameIndex Specifies the local frame index, as a value greater than or equal to 0. The index value 0 refers to the global frame.													
	M_FEATURE_LABEL(AIL_INT LocalFrameLabel)	Specifies the label value of a local frame feature.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters															
		LocalFrameLabel Specifies the local frame label, as a value greater than 0.															
	M_GLOBAL_FRAME	Specifies the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_BETWEEN_CURVES_ONE_SIDE_ONLY +		Inquires whether to establish the area by considering only one side of the curve of edgels, when you specify an M_AREA_BETWEEN_CURVES tolerance. This only has an effect if M_AREA_BETWEEN_CURVES_OPPOSITES_SUBTRACT is disabled and Aurora Imaging Library only uses positive areas. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that Aurora Imaging Library does not validate the area on only one side of the curve. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that Aurora Imaging Library validates the area on only one side of the curve. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_BETWEEN_CURVES_OPPOSITES_SUBTRACT +		Inquires whether Aurora Imaging Library should subtract opposing areas when establishing the area tolerance for M_AREA_BETWEEN_CURVES. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that Aurora Imaging Library does not subtract opposing areas. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that Aurora Imaging Library subtracts the areas on one side of the curve (opposing areas) from the areas on the other side of the curve, so that the result is positive. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_AREA_UNDER_CURVE_ALLOW_NEGATIVE +		Inquires whether Aurora Imaging Library allows negative (opposing) areas when establishing the area tolerance for M_AREA_UNDER_CURVE_MAX or M_AREA_UNDER_CURVE_MIN. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that Aurora Imaging Library does not allow negative areas when establishing the area tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that Aurora Imaging Library allows negative areas when establishing the area tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CURVE_EDGEL_GAP_SIZE +		Inquires the maximum gap between edgels for Aurora Imaging Library to consider them part of the same curve of edgels, when you specify an M_AREA_BETWEEN_CURVES, M_AREA_UNDER_CURVE_MAX, or M_AREA_UNDER_CURVE_MIN tolerance. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PIXEL_SIZE_DIAGONAL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL_SIZE_DIAGONAL	Specifies twice the diagonal size of the pixels from which the edgels were taken.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the size, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CURVE_INFO +		Inquires the information that Aurora Imaging Library uses to establish the curves of edgels, when you specify an M_AREA_BETWEEN_CURVES tolerance. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO_CURVE_OPENED.	c		e	f	g	a		i		j	k	l	m	n	b

	M_FEATURE_INDEX(AIL_INT LocalFrameIndex)	Specifies the index of a reference frame feature (local frame). The reference frame indicates the direction of the curves, and is used to sort the edgels accordingly. That is, M_AREA_BETWEEN_CURVES assumes the direction of the curves' edgels is relative to the reference frame's X-axis. Specifying a direction can make the tolerance faster to calculate, and less sensitive to gaps between edgels. When you set M_CURVE_INFO to a reference frame, M_AREA_BETWEEN_CURVES assumes that the curves are unbounded (open), as edgel direction does not apply to bounded (closed) curves.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters													
		LocalFrameIndex Specifies the index value of an existing reference frame (local frame). The index value 0 refers to the global frame.													
	M_FEATURE_LABEL(AIL_INT LocalFrameLabel)	Specifies the label of a reference frame feature (local frame). The reference frame indicates the direction of the curves, and is used to sort the edgels accordingly. That is, M_AREA_BETWEEN_CURVES assumes the direction of the curves' edgels is relative to the reference frame's X-axis. Specifying a direction can make the tolerance faster to calculate, and less sensitive to gaps between edgels. When you set M_CURVE_INFO to a reference frame, M_AREA_BETWEEN_CURVES assumes that the curves are unbounded (open), as edgel direction does not apply to bounded (closed) curves.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters													
		LocalFrameLabel Specifies the label value of an existing reference frame (local frame).													
	M_AUTO_CURVE_CLOSED	Specifies that the M_AREA_BETWEEN_CURVES tolerance is validating bounded (closed) curves (edgel features). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_CURVE_OPENED	Specifies that the M_AREA_BETWEEN_CURVES tolerance is validating unbounded (opened) curves (edgel features). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_MODE +		Inquires how to apply the angle at which to establish the distance between features when adding a distance tolerance. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ABSOLUTE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ABSOLUTE	Specifies that Aurora Imaging Library establishes the angle by taking the distance at which the features are either the farthest (for M_DISTANCE_MAX) or the closest (for M_DISTANCE_MIN).	c		e	f	g	a		i		j	k	l	m	n	b
	M_FERET_AT_ANGLE	Specifies that Aurora Imaging Library calculates the distance tolerance using the feret at the angle that is set (M_ANGLE). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GAP_AT_ANGLE	Specifies that Aurora Imaging Library calculates the distance tolerance using the gap at the angle that is set (M_ANGLE). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GAIN +		Inquires the gain Aurora Imaging Library uses to correct systematic bias between multiple imaging vision set-ups. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the gain.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET +		Inquires the offset Aurora Imaging Library uses to correct systematic bias between multiple imaging vision set-ups. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_TYPE +		Inquires the type of the geometric tolerance. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ANGULARITY	Specifies an angularity tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AREA_BETWEEN_CURVES	Specifies an area tolerance for the area between two curves of edgels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AREA_CONVEX_HULL	Specifies an area tolerance, based on the convex hull, for the surface of an edgel, point, or circle feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_AREA_SIMPLE	Specifies a simple area tolerance for the surface of an edgel, point, or circle feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AREA_UNDER_CURVE_MAX	Specifies an area tolerance for the maximum area under a curve of edgels or points. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AREA_UNDER_CURVE_MIN	Specifies an area tolerance for the minimum area under a curve of edgels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONCENTRICITY	Specifies a concentricity tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_MAX	Specifies a maximum distance tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_MIN	Specifies a minimum distance tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LENGTH	Specifies a length tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARALLELISM	Specifies a parallelism tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PERIMETER_CONVEX_HULL	Specifies a perimeter tolerance, based on the convex hull, for the surface of an edgel, point, or circle feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PERIMETER_SIMPLE	Specifies a simple perimeter tolerance for the surface of an edgel, point, or circle feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PERPENDICULARITY	Specifies a perpendicularity tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSITION_X	Specifies a positioning tolerance, along the X-direction of the specified reference frame. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_POSITION_Y	Specifies a positioning tolerance, along the Y-direction of the specified reference frame. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_RADIUS	Specifies a radius tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ROUNDNESS	Specifies a roundness tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_STRAIGHTNESS	Specifies a straightness tolerance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_VALUE_MAX +		Inquires the maximum possible value for a geometric tolerance. If a geometric tolerance value exceeds this maximum threshold, the calculation operation will fail and its status will be M_FAIL. The status of the calculation of a tolerance is found using MmetGetResult() with M_STATUS. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALUE_MIN +		Inquires the minimum required value for a geometric tolerance. If a geometric tolerance value does not meet this minimum requirement, the calculation operation will fail and its status will be M_FAIL. The status of the calculation of a tolerance is found using MmetGetResult() with M_STATUS. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALUE_WARNING_MAX +		Inquires the maximum warning value for a geometric tolerance. Geometric tolerance values that are greater or equal to M_VALUE_MIN and lower than M_VALUE_WARNING_MIN will result in a warning. Geometric tolerance values that are lower or equal to M_VALUE_MAX and greater than M_VALUE_WARNING_MAX will also result in a warning. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_EQUAL_PASS_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EQUAL_PASS_MAX	Specifies that the maximum warning value is equal to the maximum pass value (M_VALUE_MAX).	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the maximum warning value.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALUE_WARNING_MIN +		Inquires the minimum warning value for a geometric tolerance. Geometric tolerance values that are greater or equal to M_VALUE_MIN and lower than M_VALUE_WARNING_MIN will result in a warning. Geometric tolerance values that are lower or equal to M_VALUE_MAX and greater than M_VALUE_WARNING_MAX will also result in a warning. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_EQUAL_PASS_MIN.	c		e	f	g	a		i		j	k	l	m	n	b

	M_EQUAL_PASS_MIN	Specifies that the minimum warning value is equal to the minimum pass value (M_VALUE_MIN).	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the minimum warning value.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire label or index information about a feature or geometric tolerance (LabelOrIndex), set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring label or index information about a feature or geometric tolerance																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																	
	M_BASE_FEATURE_LABEL(AIL_INT BaseFeatureNumber) +	Inquires the label of the base feature, for the feature or tolerance specified by the LabelOrIndex parameter. Base features refer to the elements in the feature array (FeatureLabelArrayPtr) in either MmetAddFeature() or MmetAddTolerance().																
		Parameters																
		BaseFeatureNumber Specifies the number corresponding to the position in which you added the base feature to the feature array (FeatureLabelArrayPtr). For example, if you specify 0 as the base feature number, you are referring to first feature element in the feature array. By identifying the base feature in this way, Aurora Imaging Library can return its corresponding label.																
		0 <= *Value* < M_BASE_FEATURES_ARRAY_SIZE Specifies the base feature index.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_BASE_SUBFEATURE_INDEX(AIL_INT BaseSubfeatureInstance) +	Inquires the index of the base subfeature. Base subfeatures refer to the elements in the subfeature array (SubFeatureIndexArrayPtr) in either MmetAddFeature() or MmetAddTolerance(). You can only retrieve the index of the base subfeature when a measured multiple point feature is used as a base feature with which to build the constructed feature or tolerance specified by the LabelOrIndex parameter. That index makes it possible to isolate and use a specific instance (point subfeature) of the multiple measured point. By default, Aurora Imaging Library uses the first instance of a multiple measured point.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																

		BaseSubfeatureInstance Specifies the instance of the base subfeature. By inquiring about an instance of the base subfeature (for example, the third measured point of a multiple measure point feature), Aurora Imaging Library is able to return the corresponding index.														
		0 <= *Value* < M_BASE_SUBFEATURES_ARRAY_SIZE Specifies the index of the feature.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_BASE_FEATURES_ARRAY_SIZE +		Inquires the number of base features, for the feature or tolerance specified by the LabelOrIndex parameter. This corresponds to the number of elements in the feature array (FeatureLabelArrayPtr) in either MmetAddFeature() or MmetAddTolerance().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of elements in the feature array.	c		e	f	g	a		i		j	k	l	m	n	b
M_BASE_SUBFEATURES_ARRAY_SIZE +		Inquires the number of base subfeatures, for the feature or tolerance specified by the LabelOrIndex parameter. This corresponds to the number of elements in the subfeature array (SubFeatureIndexArrayPtr) in either MmetAddFeature() or MmetAddTolerance().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of elements in the subfeature array.	c		e	f	g	a		i		j	k	l	m	n	b
M_INDEX_FROM_LABEL +		Inquires the index associated with the specified label, if the label is used. Pass the label of either a feature or tolerance using the LabelOrIndex parameter set to M_FEATURE_LABEL() or M_TOLERANCE_LABEL(), respectively.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the label is not associated with any feature (M_FEATURE_LABEL()) or any tolerance (M_TOLERANCE_LABEL()).	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the index that is associated with the specified label.	c		e	f	g	a		i		j	k	l	m	n	b

M_LABEL_VALUE +		Inquires the label value of a specified feature or tolerance. For inquiring the feature's label, the LabelOrIndex parameter can be set to M_FEATURE_INDEX() or M_GLOBAL_FRAME. For inquiring the tolerance's label, the LabelOrIndex parameter can be set to M_TOLERANCE_INDEX().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label value of the specified feature or tolerance.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about features or geometric tolerances of a metrology context, you can set the InquireType parameter to one of the following values. In this case, the LabelOrIndex parameter must be set to an existing feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX() or an existing tolerance label or index using M_TOLERANCE_LABEL() or M_TOLERANCE_INDEX(), and the MetId parameter must be set to a metrology context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about features or geometric tolerances																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_ANGLE +		Inquires the angle used to construct a feature or define a tolerance, when applicable. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0 °.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the angle. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAWABLE +		Inquires whether the feature or tolerance will be drawn using MmetDraw(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the feature or tolerance will not be drawn. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the feature or tolerance will be drawn.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the clone transformation settings used for a cloned feature in a metrology result buffer, set the InquireType parameter to one of the following values. For inquiring clone transformations, the LabelOrIndex parameter must be set to an existing feature label or index using M_FEATURE_LABEL() or M_FEATURE_INDEX().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring clone transformations settings																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

UserVarPtr - Possible values returned																
M_CLONE_ANGLE +	Inquires the rotation angle of the cloned feature in the counterclockwise direction. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 °.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLONE_OFFSET_X +	Inquires the translation value of the cloned feature, in the X-direction. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-offset for the cloned feature, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLONE_OFFSET_Y +	Inquires the translation value of the cloned feature, in the Y-direction. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-offset for the cloned feature, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLONE_SCALE +	Inquires the scale factor of the cloned feature. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the scale factor.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about result buffer settings, set the InquireType parameter to one of the following values. In this case, the LabelOrIndex parameter must be set to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring result buffer settings																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_OUTPUT_FRAME +		Inquires the output reference frame that will be used to return the feature results. SET														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_GLOBAL_FRAME.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GLOBAL_FRAME	Specifies that Aurora Imaging Library returns feature results relative to the origin of the global frame. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_IMAGE_FRAME	Specifies that Aurora Imaging Library returns feature results relative to the origin of the target image's reference frame. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_REFERENCE_FRAME	Specifies that Aurora Imaging Library returns feature results relative to the reference frame of each feature. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label of an existing local frame feature to use to return feature results.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_OUTPUT_UNITS +		Inquires whether results are returned in pixel or world units. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for

Tables:

- For inquiring general context settings
- For inquiring about the system
- For inquiring global processing settings
- For inquiring about features
- For inquiring about constructed edgel features
- For inquiring about geometric tolerances
- For inquiring label or index information about a feature or geometric tolerance
- For inquiring clone transformations settings

- For inquiring result buffer settings

the values listed in all the parameter association tables except **For inquiring the geometry of the metrology region**, **For inquiring the geometry data of the metrology region**, **For inquiring about features or geometric tolerances**.

You can add the following value to the above-mentioned values to determine the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value of an inquire type																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_DEFAULT	Inquires the default value of the specified inquire type.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For inquiring general context settings
- For inquiring about the system
- For inquiring global processing settings
- For inquiring about features
- For inquiring about constructed edgel features
- For inquiring about geometric tolerances
- For inquiring label or index information about a feature or geometric tolerance
- For inquiring clone transformations settings
- For inquiring result buffer settings

the values listed in all the parameter association tables except **For inquiring the geometry of the metrology region**, **For inquiring the geometry data of the metrology region**, **For inquiring about features or geometric tolerances**.

You can add the following value to the above-mentioned values to determine whether a result is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_SUPPORTED	Inquires whether the specified inquire type is supported for the metrology context.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for

Tables:

- For inquiring general context settings
- For inquiring about the system
- For inquiring global processing settings
- For inquiring about features
- For inquiring about constructed edgel features
- For inquiring the geometry of the metrology region
- For inquiring the geometry data of the metrology region
- For inquiring about geometric tolerances
- For inquiring label or index information about a feature or geometric tolerance
- For inquiring about features or geometric tolerances
- For inquiring clone transformations settings
- For inquiring result buffer settings

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.																
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetName

Synopsis

Set or retrieve the name of a feature or tolerance, or retrieve the label or index of a specified feature or tolerance.

Syntax

```
void MmetName(
    AIL_ID ContextOrResultId, //out
    AIL_INT64 Operation,      //in
    AIL_INT LabelOrIndex,     //in
    AIL_TEXT_PTR String,      //in-out
    AIL_INT *ValuePtr,        //out
    AIL_INT64 ControlFlag     //in
)
```

Description

This function allows you to set or retrieve the name of a metrology feature or tolerance. It also allows you to retrieve either the label or index of a feature or tolerance based on its name.

Note that **MmetName**() supports Unicode.

Parameters

ContextOrResultId

Specifies the identifier of the metrology context or result buffer containing the feature or tolerance to read or modify. The metrology context or the metrology result buffer must have been previously allocated on the required system using MmetAlloc() or MmetAllocResult(), respectively.

Operation

Specifies the type of operation to perform.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the operation to perform																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GET_FEATURE_INDEX	Retrieves the index of a feature, based on its name. Set the LabelOrIndex parameter to **M_DEFAULT**. If there is no feature with the specified name, the returned index is -1.	c		e	f	g	a		i		j	k	l	m	n	b
M_GET_FEATURE_LABEL	Retrieves the label of a feature, based on its name. Set the LabelOrIndex parameter to **M_DEFAULT**. If there is no feature with the specified name, the returned label is 0.	c		e	f	g	a		i		j	k	l	m	n	b

M_GET_NAME	Retrieves the name of the feature or tolerance defined by the LabelOrIndex parameter. The name of the feature or tolerance is returned via the String parameter, and the name's length is returned via the ValuePtr parameter. To determine the exact size name, call this function twice, the first time to determine the size of the name, with the String parameter set to **M_NULL**, and the second time to return the name.	c		e	f	g	a		i		j	k	l	m	n	b
M_GET_TOLERANCE_INDEX	Retrieves the index of a tolerance, based on its name. Set the LabelOrIndex parameter to **M_DEFAULT**. If there is no tolerance with the specified name, the returned index is -1.	c		e	f	g	a		i		j	k	l	m	n	b
M_GET_TOLERANCE_LABEL	Retrieves the label of a tolerance, based on its name. Set the LabelOrIndex parameter to **M_DEFAULT**. If there is no tolerance with the specified name, the returned label is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_SET_NAME	Sets the name of the feature or tolerance defined by the LabelOrIndex parameter. This operation requires ContextOrResultId to be a valid context identifier. In addition, the specified name of the feature or tolerance (passed in the String parameter) must be unique, and ValuePtr must be set to **M_NULL**. Note that you can release a name by passing **M_NULL** to String.	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Specifies the label or index value of the metrology feature or tolerance. Set this parameter to **M_DEFAULT** when not required by the name operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the label or index value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FEATURE_INDEX(AIL_INT FeatureIndex)	Specifies a feature by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureIndex Set this parameter to the following:															

			Value >= 0 Specifies the index of the feature. Note that the index of the global frame is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL(AIL_INT FeatureLabel)	Specifies a feature by indicating its label.			c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																
		FeatureLabel Set this parameter to the following:																
			Value > 0 Specifies the label of the feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_INDEX(AIL_INT ToleranceIndex)	Specifies a tolerance by indicating its index.			c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																
		ToleranceIndex Set this parameter to the following:																
			Value >= 0 Specifies the index of the tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOLERANCE_LABEL(AIL_INT ToleranceLabel)	Specifies a tolerance by indicating its label.			c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																
		ToleranceLabel Set this parameter to the following:																

			Value > 0	Specifies the label of the tolerance.	c		e	f	g	a		i		j	k	l	m	n	b

String

Specifies the name to set or retrieve, depending on the Operation parameter.

Set this parameter to **M_NULL** and the Operation parameter to M_SET_NAME to disassociate the name from the tolerance or feature.

Set this parameter to **M_NULL** and the Operation parameter to M_GET_NAME to determine the length of the string.

Note that when setting a feature or tolerance name with the String parameter, the name must be unique (this does not apply when Operation is set to M_GET_NAME).

ValuePtr

Specifies the address at which to write the requested information.

This parameter must be set to **M_NULL** when not in use.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetPut

Synopsis

Put edgels or points from user-supplied arrays into a constructed edgel feature built with external edgels/points.

Syntax

```
void MmetPut(
    AIL_ID ContextMetId,               //out
    AIL_INT LabelOrIndex,              //in
    AIL_INT NumEdgelsOrPoints,         //in
    const AIL_INT *ChainIndexArrayPtr, //in
    const AIL_DOUBLE *PosXArrayPtr,    //in
    const AIL_DOUBLE *PosYArrayPtr,    //in
    const AIL_DOUBLE *AngleArrayPtr,   //in
    const AIL_DOUBLE *ValueArrayPtr,   //in
    AIL_INT64 ControlFlag              //in
)
```

Description

This function adds edgels or points from user-supplied arrays to a constructed edgel feature built with external edgels/points. You can add a constructed edgel feature built with external edgels/points to the metrology context using MmetAddFeature() with M_EDGEL and M_EXTERNAL_FEATURE. Every time **MmetPut**() is called, it adds the specified edgels/points to the constructed feature, without overwriting previously-added edgels/points with the same position. Points and edgels can be added to the same feature, but you must use separate calls.

MmetPut() is useful if you want to construct an edgel feature with edgels established using the Edge Finder module or with the coordinates of chained pixels (points) established using the Blob Analysis module. You can also, for example, use **MmetPut**() to add points from a cross-section of a 3D scanned object, delimited by an extraction box, so that you can analyze the profile of the cross-section. To inquire information about an external edgel feature (M_EXTERNAL_FEATURE), such as the total number of external points/edgels it contains (M_NUMBER), call MmetInquire().

Information about a particular edgel/point must occupy the same index in each array. You specify points by specifying their position (X- and Y-position arrays). You specify edgels by specifying their position and their gradient angle (edgel direction). An edgel's angle must respect the convention:

- The angle is measured between the target image's horizontal axis and the edge's gradient (the direction of the edgel's most sudden change from black to white).
- The angle is measured counter-clockwise.
- You must map the angle to an 8-bit range; that is, 0° corresponds to 0, and 360° corresponds to 256.

A point/edgel's position must be relative to the reference frame of the constructed edgel feature (MmetControl() with M_REFERENCE_FRAME), which can be set to either the global frame or a local frame.

When passing points that are along an edge, but you don't have their gradient angle information (as is the case when obtained using the Blob Analysis module), you can have **MmetPut**() automatically calculate their approximate gradient angle by passing M_INTERPOLATE_ANGLE to the ControlFlag parameter. **MmetPut**() interpolates their gradient angles assuming that points in the arrays are consecutive. As such, the points in the X- and Y-position arrays must be an ordered, connected chain of points. Note that if the points are not from an edge, you should not pass M_INTERPOLATE_ANGLE.

Consecutive edgels in the arrays are considered to be consecutive in their associated edge.

For edgels or points along an edge, you can associate the edgels/points with the index of their edge chain/pixel chain using the ChainIndexArrayPtr parameter. You can add to the end of an edge chain/pixel chain that has already been

added to the feature.

To remove all the edgels/points from the constructed edgel feature, set the ControlFlag parameter to M_RESET.

Note, if a camera calibration context is associated with the expected target image, set values in real-world units (for example, positional values). Otherwise, use pixel units.

> Note that presented above is an example prototype of this function. You can call MmetPut() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use MmetPut() and not, for example, MmetPutFloat(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

ContextMetId

Specifies the identifier of the metrology context. The metrology context must have been previously allocated with MmetAlloc().

LabelOrIndex

Specifies the label or index of a constructed edgel feature with an M_EXTERNAL_FEATURE build operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the label or index value																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FEATURE_INDEX(AIL_INT FeatureIndex)	Specifies the index value of a constructed feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureIndex This parameter specifies the index of the constructed feature to which the edgels or points will be added. You can set this parameter to the following:															
	Value >= 0 — Specifies the index of the constructed feature. Note that the index of the global frame is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL(AIL_INT FeatureLabel)	Specifies the label value of a constructed feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureLabel This parameter specifies the label of the constructed feature to which the edgels or points will be added. You can set this parameter to the following:															

			Value > 0	Specifies the label of the constructed feature.	c		e	f	g	a		i		j	k	l	m	n	b

NumEdgelsOrPoints

Specifies the number of edgels/points to add to the constructed edgel feature.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the PosXArrayPtr, PosYArrayPtr, or AngleArrayPtr parameter.

ChainIndexArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)
- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)

Specifies the address of the array containing the index of the edge chain/pixel chain to which an edgel/point belongs. If specifying edgels/points from several separate chains, this index array identifies the connected edgels/points that are part of the same chain. The order of the edgels in the array determines the shape of the chain. This is especially useful if the gradient angle is not provided, and **MmetPut**() is interpolating the angle value. If unused, set this parameter to **M_NULL**.

PosXArrayPtr INQ
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)
- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)

Specifies the address of the array containing the X-coordinates of the edgels/points. To remove all the edgels/points in the feature, set this parameter to **M_NULL**.

PosYArrayPtr INQ
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)
- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)

Specifies the address of the array containing the Y-coordinates of the edgels/points. To remove all the edgels/points in the feature, set this parameter to **M_NULL**.

AngleArrayPtr INQ
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)
- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)

Specifies the address of the array containing the gradient angle (orientation) of the edgels. If adding points instead of edgels, or removing the edgels/points in the feature, set this parameter to **M_NULL**.

Angle values must be mapped in the range of 0 to 255. That is, 0° corresponds to 0, and 360° corresponds to 256.

ValueArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)
- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>] (and a maximum array rank for C# of 1)
  Required array size:(NumEdgelsOrPoints)

Reserved for future expansion and must be set to **M_NULL**.

ControlFlag

Specifies whether to automatically calculate the expected gradient angle of the edge at the specified points, when a gradient angle array has not been specified. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To specify the function's control flag																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to ignore this parameter, or in the case when points are passed, not to calculate the gradient angle.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATE_ANGLE	Specifies to determine the expected gradient angle of the edge at the specified points. This assumes that the specified points are a connected chain of points. Angles are calculated based on interpolation of consecutive points. This can be useful when limiting the angular range for fitted features.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESET	Specifies to remove all the edgels/points from the specified constructed edgel feature. In this case, the NumEdgelsOrPoints parameter must be set to 0, and the array parameters to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler

```
void MmetPutDouble (AIL_ID ContextMetId, AIL_INT LabelOrIndex, AIL_INT NumEdgelsOrPoints, const AIL_INT
*ChainIndexArrayPtr, const AIL_DOUBLE *PosXArrayPtr, const AIL_DOUBLE *PosYArrayPtr, const AIL_DOUBLE
*AngleArrayPtr, const AIL_DOUBLE *ValueArrayPtr, AIL_INT64 ControlFlag)
```

Details

Parameters

ContextMetId

See ContextMetId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

NumEdgelsOrPoints

See NumEdgelsOrPoints of the main function for a description.

ChainIndexArrayPtr

See ChainIndexArrayPtr of the main function for a description.

PosXArrayPtr

See PosXArrayPtr of the main function for a description.

PosYArrayPtr

See PosYArrayPtr of the main function for a description.

AngleArrayPtr

See AngleArrayPtr of the main function for a description.

ValueArrayPtr

See ValueArrayPtr of the main function for a description.

ControlFlag

See ControlFlag of the main function for a description.

```
void MmetPutFloat (AIL_ID ContextMetId, AIL_INT LabelOrIndex, AIL_INT NumEdgelsOrPoints, const AIL_INT *ChainIndexArrayPtr, const AIL_FLOAT *PosXArrayPtr, const AIL_FLOAT *PosYArrayPtr, const AIL_FLOAT *AngleArrayPtr, const AIL_FLOAT *ValueArrayPtr, AIL_INT64 ControlFlag)
```

Details

Parameters

ContextMetId

See ContextMetId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

NumEdgelsOrPoints

See NumEdgelsOrPoints of the main function for a description.

ChainIndexArrayPtr

See ChainIndexArrayPtr of the main function for a description.

PosXArrayPtr

See PosXArrayPtr of the main function for a description.

PosYArrayPtr

See PosYArrayPtr of the main function for a description.

AngleArrayPtr

See AngleArrayPtr of the main function for a description.

ValueArrayPtr

See ValueArrayPtr of the main function for a description.

ControlFlag

See ControlFlag of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetRestore

Synopsis

Restore a metrology context from disk.

Syntax

```
AIL_ID MmetRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextIdPtr         //out
)
```

Description

This function restores a metrology context that was previously saved to a file, using MmetSave() or MmetStream(). This function restores all the metrology context's settings and names that were in effect when the metrology context was saved.

If you had associated a camera calibration context with the template reference in your metrology context and you did not save it with the context, you must re-associate the camera calibration context, using MmetControl() with M_ASSOCIATED_CALIBRATION. Moreover, if you had previously set drawing control types, using MgraControl() with M_DRAW_..., you must reset them since they were not saved with the context.

When the restored metrology context is no longer required, release it using MmetFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the restored metrology context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the metrology context. The function handles (internally) the opening and closing of the file.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, metrology contexts have an MET extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the metrology context.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies whether to restore, from the metrology context, the camera calibration context associated with the template reference.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to restore the camera calibration context associated with the template reference																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the camera calibration context is not restored.	c		e	f	g	a		i		j	k	l	m	n	b
M_WITH_CALIBRATION	Specifies that the camera calibration context is restored. In this case, the camera calibration context must have been previously saved with the context, using MmetSave() or MmetStream() with M_WITH_CALIBRATION. The camera calibration information is restored from the same file as the metrology context. The camera calibration cannot be managed independently from the metrology context. When the metrology context is freed, the camera calibration is automatically freed as well.	c		e	f	g	a		i		j	k	l	m	n	b

ContextIdPtr

Specifies the address of the variable in which to write the metrology context identifier or specifies the data type that the function should use to return the metrology context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated metrology context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated metrology context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_MET_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the metrology context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored metrology context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the metrology context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_MET_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetSave

Synopsis

Save a metrology context to a file.

Syntax

```
void MmetSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextId,            //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about the previously allocated metrology context to disk, including all of the feature and geometric tolerance settings and names. This information can be reloaded, using MmetRestore() or MmetStream(). However, drawing control type settings are not saved.

Parameters

FileName

Specifies the name and path of the file in which to save the metrology context. It is recommended that you use the MET file extension for easier use with other Aurora Imaging Library software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, metrology context files have an MET extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextId

Specifies the identifier of the metrology context to save.

ControlFlag

Specifies whether to save, with the metrology context, the camera calibration context associated with the template reference.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to save the camera calibration context associated with the template reference																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the camera calibration context is not saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_WITH_CALIBRATION	Specifies that the camera calibration context is saved. The camera calibration information is saved in the same file as the metrology context.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetSetPosition

Synopsis

Set the position of a feature.

Syntax

```
void MmetSetPosition(
    AIL_ID ContextId,             //out
    AIL_INT FeatureLabelOrIndex,  //in
    AIL_INT64 TransformationType, //in
    AIL_DOUBLE Param1,            //in
    AIL_DOUBLE Param2,            //in
    AIL_DOUBLE Param3,            //in
    AIL_DOUBLE Param4,            //in
    AIL_INT64 ControlFlag         //in
)
```

Description

This function moves the specified feature's position in the metrology template of a metrology context.

This function is valid for all measured (MmetAddFeature() with M_MEASURED) and parametrically constructed (MmetAddFeature() with M_CONSTRUCTED and M_PARAMETRIC) features, unless otherwise specified. Typically, this function is used to move the position of a parametrically constructed local frame (which is considered a feature). When specifying a frame, all features and metrology regions that use that frame as their reference frame are moved accordingly. Note that you can also position features using MmetControl() with M_POSITION_X and M_POSITION_Y. However, in this case you must enter exact values; you cannot, for example, position features based on the results of another module, as you can using **MmetSetPosition**() with M_RESULT.

If you want to move the whole metrology template (for example, when moving the global frame), you should use McalFixture() instead of **MmetSetPosition**().

Note that if you save the context using MmetSave(), the positional information set using **MmetSetPosition**() is not saved with the context; after restoring the context, you will need to call **MmetSetPosition**() again.

Aurora Imaging Library moves the specified feature relative to its reference frame, except for the global frame, which Aurora Imaging Library moves relative to its default origin (0,0). If the target image is not calibrated, the global frame's default origin is aligned with the center of the top-left corner pixel of the target image. If the target image is calibrated, the global frame's default origin is aligned with the origin of the relative (world) coordinate system.

If a camera calibration context is associated with the target image, set positional and dimensional values in real-world units. Otherwise, use pixel units.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MmetSetPosition()** internally calls the *AIL_DOUBLE* version of this function (MmetSetPositionDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MmetSetPositionInt64()).

Parameters

ContextId

Specifies the identifier of the metrology context. The metrology context must have been previously allocated on the required system using MmetAlloc().

FeatureLabelOrIndex

Specifies the label or index of the feature you want to position. Set this parameter to one of the following values.

For specifying a feature																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the global frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_INDEX(AIL_INT FeatureIndex)	Specifies the index value of an existing individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureIndex This parameter specifies the index of the feature to position. You can set this parameter to the following:															
	Value >= 0 — Specifies the index. Note that the index of the global frame is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL(AIL_INT FeatureLabel)	Specifies the label value of an existing individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureLabel This parameter specifies the label of the feature to position. You can set this parameter to the following:															
	Value > 0 — Specifies the label.	c		e	f	g	a		i		j	k	l	m	n	b

TransformationType

Specifies the type of transformation that will be used to set the new position of the feature.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies an attribute of the position to set. Its definition is dependent on the type of position chosen.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies an attribute of the position to set. Its definition is dependent on the type of position chosen.

See the Parameter associations section for possible values that can be specified.

Param3

Specifies an attribute of the position to set. Its definition is dependent on the type of position chosen.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Param4

Specifies an attribute of the position to set. Its definition is dependent on the type of position chosen.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **TransformationType**, **Param1**, **Param2**, **Param3**, and **Param4** parameters.

- For selecting the operation that will be used to position the feature

Note that any unused parameters should be set to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For selecting the operation that will be used to position the feature																
TransformationType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Param1																
Param2																
Param3																
Param4																
M_FORWARD_COEFFICIENTS	Specifies to position the feature using an operation based on forward transformation coefficients, using the equations that follow. $x_d = \frac{A}{\sqrt{A^2+B^2}}x_s + \frac{B}{\sqrt{A^2+B^2}}y_s + C$ $y_d = \frac{-B}{\sqrt{A^2+B^2}}x_s + \frac{A}{\sqrt{A^2+B^2}}y_s + D$															
Param1	Sets the forward transformation coefficient *A*.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the coefficient.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the forward transformation coefficient *B*.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the coefficient.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the forward transformation coefficient *C*.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the coefficient.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Sets the forward transformation coefficient *D*.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the coefficient.	c		e	f	g	a		i		j	k	l	m	n	b
M_GEOMETRIC	Specifies to position the feature using an operation based on a translation and rotation.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets the translation value for the feature, in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the distance to move the feature, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the translation value for the feature, in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the distance to move the feature, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the angle of rotation for the feature.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION	Specifies to position the feature at a specific position and angle. This should be used only for a parametrically constructed local frame.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets the X-coordinate of the origin of the local frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. The default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the Y-coordinate of the origin of the local frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. The default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the angle of the local frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT	Specifies to position the feature at the position calculated by another Aurora Imaging Library module. Supported modules are Code, Geometric Model Finder, Pattern Matching, and String Reader. If you are using a Model Finder result, you can combine M_RESULT with M_FORWARD_COEFFICIENTS. In this case, the transformation uses the forward coefficients of Model Finder's result occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets an Aurora Imaging Library result buffer identifier from the Aurora Imaging Library module to use to specify the feature's position.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the index value of the result occurrence to use to set the feature's position. This value must be a valid index that is contained within the result buffer you provided with Param1.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MmetSetPositionInt64 (AIL_ID ContextId, AIL_INT FeatureLabelOrIndex, AIL_INT64 TransformationType, AIL_INT64 Param1, AIL_DOUBLE Param2, AIL_DOUBLE Param3, AIL_DOUBLE Param4, AIL_INT64 ControlFlag)
```

Details

Parameters

ContextId

See ContextId of the main function for a description.

FeatureLabelOrIndex

See FeatureLabelOrIndex of the main function for a description.

TransformationType

See TransformationType of the main function for a description.

Param1

See Param1 of the main function for a description.

Param2

See Param2 of the main function for a description.

Param3

See Param3 of the main function for a description.

Param4

See Param4 of the main function for a description.

ControlFlag

See ControlFlag of the main function for a description.

```
void MmetSetPositionDouble (AIL_ID ContextId, AIL_INT FeatureLabelOrIndex, AIL_INT64 TransformationType, AIL_DOUBLE Param1, AIL_DOUBLE Param2, AIL_DOUBLE Param3, AIL_DOUBLE Param4, AIL_INT64 ControlFlag)
```

Details

Parameters

ContextId

See ContextId of the main function for a description.

FeatureLabelOrIndex

See FeatureLabelOrIndex of the main function for a description.

TransformationType

See TransformationType of the main function for a description.

Param1

See Param1 of the main function for a description.

Param2

See Param2 of the main function for a description.

Param3

See Param3 of the main function for a description.

Param4

See Param4 of the main function for a description.

ControlFlag

See ControlFlag of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetSetRegion

Synopsis

Set the metrology region that delimits the area from which to establish the feature.

Syntax

```
void MmetSetRegion(
    AIL_ID ContextId,                   //out
    AIL_INT FeatureLabelOrIndex,        //in
    AIL_INT ReferenceFrameLabelOrIndex, //in
    AIL_INT64 Geometry,                 //in
    AIL_DOUBLE Param1,                  //in
    AIL_DOUBLE Param2,                  //in
    AIL_DOUBLE Param3,                  //in
    AIL_DOUBLE Param4,                  //in
    AIL_DOUBLE Param5,                  //in
    AIL_DOUBLE Param6                   //in
)
```

Description

This function sets a geometrically-defined ROI, referred to as a metrology region, for a feature within the template of the metrology context. For a physically measured feature, the metrology region delimits the area in the target image from which to establish the edgels of the feature to validate and measure. For a constructed feature, the metrology region delimits the area in the feature itself, which is used as the base feature to build the constructed feature.

When you call this function, you can define the metrology region in one of the following ways.

- Explicitly specify the shape and size of the region. This is referred to as an explicitly-defined metrology region.
- Set the region's shape and size using a graphic in a 2D graphics list. This is referred to as a 2D graphics list metrology region.
- Define the metrology region based on other features by passing a derived metrology region object to this function. This is referred to as a derived metrology region.

When using a 2D graphics list metrology region (M_FROM_GRAPHIC_LIST), the 2D graphics list must contain only one graphic and it must have the same geometric-shape as specified with the Geometry parameter of this function. The advantage of defining a 2D graphics list metrology region is that you can set it with respect to the relative coordinate system as well as set it interactively.

When using a derived metrology region (M_FROM_DERIVED_GEOMETRY_REGION), you must allocate a derived metrology region object using MmetAlloc() with M_DERIVED_GEOMETRY_REGION, and then set it up using MmetControl() with M_REGION_.... Once you specify the geometry of the derived metrology region using the M_REGION_GEOMETRY control type, you can set most of the geometry's attributes either explicitly or derive them based on a feature. Use the corresponding M_REGION_..._TYPE control type to specify which. For example, instead of specifying explicit X- and Y-coordinates to set the position of a rectangular metrology region, you can derive its position according to the position of a point feature.

The metrology region that you choose must be compatible with the geometry and operation of the feature you are adding, with MmetAddFeature(). For example, to add a physically measured circle feature that is built using a best fit operation, you can use an M_RING metrology region.

The metrology region is set according to its feature's reference frame (ReferenceFrameLabelOrIndex). The global frame is the default. If the target image is not calibrated, the global frame's default origin (0,0) is aligned with the center of the top-left corner pixel of the target image. If the target image is calibrated, the global frame's default origin is aligned with the origin of the relative (world) coordinate system. To change the reference frame to a local frame, use MmetControl() with M_REFERENCE_FRAME. Local frames are coordinate systems located anywhere within the metrology template and

defined as metrology features.

A valid feature must not only fall within the metrology region, but its edgels' gradient angle must also fall along the region's orientation. Various control types can be used to set a valid relationship between the metrology region's orientation, and the edgels' gradient angle. For more information, see the Gradient angle subsection of the Eliminating unwanted edgels section of Chapter 21: Metrology.

> Note that you cannot specify a geometrically-defined region for a feature with a region of interest (ROI) set using MbufSetRegion(); you can only use metrology regions established with this function.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MmetSetRegion()** internally calls the *AIL_DOUBLE* version of this function (MmetSetRegionDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MmetSetRegionInt64()).

Parameters

ContextId

Specifies the identifier of the metrology context containing the template in which to set the metrology region. The metrology context must have been previously allocated on the required system using MmetAlloc().

FeatureLabelOrIndex

Specifies the label or index of the feature that will be established from the area delimited by the metrology region. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a feature																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FEATURE_INDEX(AIL_INT FeatureIndex)	Specifies the index of an existing individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureIndex This parameter specifies the index of the feature to establish from the metrology region. You can set this parameter to the following:															
	Value >= 0 — Specifies the index.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL(AIL_INT FeatureLabel)	Specifies the label value of an existing individual feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureLabel This parameter specifies the label of the feature to establish from the metrology region. You can set this parameter to the following:															

			Value > 0 Specifies the label.	c		e	f	g	a		i		j	k	l	m	n	b

ReferenceFrameLabelOrIndex

Specifies the label or index of the reference frame feature of the feature for which you are specifying a metrology region. To change the reference frame, use MmetControl() with M_REFERENCE_FRAME.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the reference frame																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GLOBAL_FRAME.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_INDEX(AIL_INT FeatureIndex)	Specifies the index value of an existing reference frame feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureIndex This parameter specifies the index of the reference frame feature. You can set this parameter to the following:															
	Value >= 0 Specifies the index. Note that the index of the global frame is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_FEATURE_LABEL(AIL_INT FeatureLabel)	Specifies the label value of an existing reference frame feature.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	FeatureLabel This parameter specifies the label of the reference frame feature. You can set this parameter to the following:															
	Value > 0 Specifies the label.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_FRAME	Specifies the global frame as the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b

Geometry INQ

Specifies the geometric shape of the metrology region.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies an attribute of the metrology region to set. Its definition is dependent on the geometry chosen.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies an attribute of the metrology region to set. Its definition is dependent on the geometry chosen.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Param3

Specifies an attribute of the metrology region to set. Its definition is dependent on the geometry chosen.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Param4

Specifies an attribute of the metrology region to set. Its definition is dependent on the geometry chosen.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Param5

Specifies an attribute of the metrology region to set. Its definition is dependent on the geometry chosen.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Param6

Specifies an attribute of the metrology region to set. Its definition is dependent on the geometry chosen.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **Geometry**, **Param1**, **Param2**, **Param3**, **Param4**, **Param5**, and **Param6** parameters.

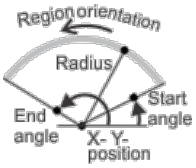
- For setting an explicitly-defined or 2D graphics list metrology region
- For setting a derived metrology region

The following can be specified for setting an explicitly-defined metrology region. To specify a 2D graphics list metrology region, you must add M_FROM_GRAPHIC_LIST to the supported geometry (M_ARC, M_RECTANGLE, and M_SEGMENT).

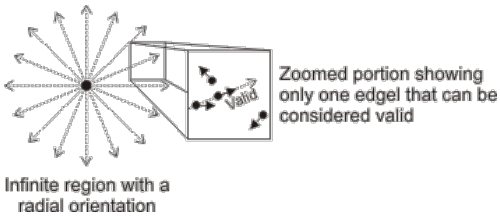
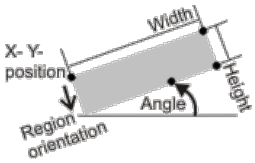
Unused parameters should be set to **M_NULL**.

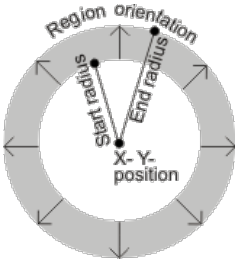
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

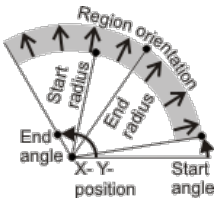
For setting an explicitly-defined or 2D graphics list metrology region

Geometry	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Param1																
Param2																
Param3																
Param4																
Param5																
Param6																
M_DEFAULT	Same as M_INFINITE.															
M_ARC +	Specifies that the feature will be established from an arc metrology region.  M_ARC can be used to establish point (M_POINT) features.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets the X-coordinate of the origin of the arc metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the origin's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the Y-coordinate of the origin of the arc metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the origin's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the radius of the arc metrology region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the radius, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Sets the start angle of the arc metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

Param5	Sets the end angle of the arc metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 360.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that the feature will be established from an infinite metrology region. An infinite metrology region is an X- Y-plane with no boundaries. Use Param3 to specify the angle that defines the orientation of this region. Note that M_INFINITE is similar to M_RECTANGLE, however a rectangular metrology region has a width and a height, which are used to bind the metrology region. M_INFINITE can be used to establish any feature and is the default metrology region.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets the X-coordinate of the origin of the infinite metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the origin's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the Y-coordinate of the origin of the infinite metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the origin's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the angle (orientation) of the infinite metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0°. This indicates that the orientation of the infinite metrology region is the same as the orientation of the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b

	M_RADIAL	Specifies that the infinite metrology region has a radial orientation, from 0.0° to 360.0°, based on the specified origin.  In such a metrology region, edgels can only be considered part of the feature if their gradient angle conforms to the direction of a radii pointing out from the region's origin. M_RADIAL is usually used to define a metrology region for a physically measured circular feature whose radius is unknown. Typically the origin of the metrology region is placed at the feature's center, resulting in valid edgels only being possible if they encircle the origin.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the angle, in degrees, relative to the reference frame. This angle represents the infinite region's orientation.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECTANGLE +		Specifies that the feature will be established from a rectangular metrology region.  Note that M_RECTANGLE is similar to M_INFINITE, however an infinite metrology region does not have a width and a height (boundaries). M_RECTANGLE can be used to establish segment (M_SEGMENT) or edgel (M_EDGEL) features.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Sets the X-coordinate of the origin of the rectangular metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the origin's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Sets the Y-coordinate of the origin of the rectangular metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the origin's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Sets the width of the rectangle. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the rectangle's width, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Sets the height of the rectangle. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the rectangle's height, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param5	Sets the angle (orientation) of the rectangular metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_RING	Specifies that the feature will be established from a ring-shaped metrology region.  M_RING can be used to establish circle (M_CIRCLE) or edgel (M_EDGEL) features.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets the X-coordinate of the center of the ring-shaped metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the origin's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the Y-coordinate of the center of the ring-shaped metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the origin's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the start radius of the ring-shaped metrology region. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the start radius, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Sets the end radius of the ring-shaped metrology region. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the end radius, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_RING_SECTOR	Specifies that the feature will be established from a ring-sector metrology region.  M_RING_SECTOR can be used to establish radial segments (M_SEGMENT), arcs (M_ARC), or edgel (M_EDGEL) features.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets the X-coordinate of the origin of the ring-sector metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the origin's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the Y-coordinate of the origin of the ring-sector metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the origin's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the start radius of the ring-sector metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the start radius, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Sets the end radius of the ring-sector metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies the default value; the default value is 0.0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the end radius, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param5	Sets the start angle of the ring-sector metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
Param6	Sets the end angle of the ring-sector metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 360.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEGMENT +	Specifies that the feature will be established from a linear segment metrology region. M_SEGMENT can be used to establish point (M_POINT) features.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets the X-coordinate of the start point of the segment metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the start point's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the Y-coordinate of the start point of the segment metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the start point's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the X-coordinate of the end point of the segment metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the end point's X-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b
Param4		Sets the Y-coordinate of the end point of the segment metrology region, relative to the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the end point's Y-coordinate, in pixel or world units.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_ARC; M_RECTANGLE; M_SEGMENT.

You can add the following value to the above-mentioned values to set a 2D graphics list metrology region.

Unused parameters should be set to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting a 2D graphics list metrology region			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Geometry		Description															
	Param1																
	Param2																
	Param3																
	Param4																
	Param5																
	Param6																
M_FROM_GRAPHIC_LIST		Specifies a 2D graphics list metrology region. Interactive graphics are allowed. Metrology ignores a graphic's fill color, if specified. When using M_FROM_GRAPHIC_LIST with M_ARC, the 2D graphics list must contain only one arc, created using MgraArc(), MgraArcAngle(), or MgraArcFill(). The arc must be circular; that is, its X- and Y-radius must be the same. When using M_FROM_GRAPHIC_LIST with M_RECTANGLE, the 2D graphics list must contain only one rectangle, created using MgraRect(), MgraRectAngle(), or MgraRectFill(). When using M_FROM_GRAPHIC_LIST with M_SEGMENT, the 2D graphics list must contain only one line, created using MgraLine().															

			c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Sets the identifier of the 2D graphics list with which to delimit the metrology region. The 2D graphics list must have been previously allocated on the required system using MgraAllocList(). The graphic's data (not the graphic itself) is taken as-is and is considered part of the metrology context with which it is used. For example, the graphic's metrology region information is now relative to the feature's reference frame (ReferenceFrameLabelOrIndex), is saved with the metrology context (MmetSave()), and is not affected by subsequent 2D graphics list modifications unless you re-call M_FROM_GRAPHIC_LIST. When using a 2D graphics list metrology region, you can first call McalFixture() with M_MOVE_RELATIVE and M_RESULT_MET, and specify the feature's reference frame. You can then ensure that positions are set and returned according to the specified reference frame. For more information, see the Moving using a metrology reference frame subsection of the Moving the relative coordinate system section of Chapter 29: Fixturing in Aurora Imaging Library.															

The following can be specified for setting a derived metrology region.

Unused parameters should be set to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting a derived metrology region			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Geometry		Description															
	Param1																
	Param2																
	Param3																
	Param4																
	Param5																
	Param6																
M_FROM_DERIVED_GEOMETRY_REGION		Specifies a derived metrology region. To set up this metrology region, use MmetControl() with M_REGION_.... It is only recommended to use a derived metrology region when you need at least one other feature to set the metrology region. However, you can use a derived metrology region to set the metrology region using features, explicit values, or a combination of the two. Once you define this type of metrology region, the region information (not the derived metrology region object itself) is considered part of the metrology context with which it is used; for example, that information will be saved along with the context if you call MmetSave(). You can use M_FROM_DERIVED_GEOMETRY_REGION with any geometry.															

	Param1	Sets the identifier of the derived metrology region object. The derived metrology region object must have been previously allocated on the required system using MmetAlloc() with M_DERIVED_GEOMETRY_REGION.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MmetSetRegionInt64 (AIL_ID ContextId, AIL_INT FeatureLabelOrIndex, AIL_INT ReferenceFrameLabelOrIndex,
AIL_INT64 Geometry, AIL_INT64 Param1, AIL_DOUBLE Param2, AIL_DOUBLE Param3, AIL_DOUBLE Param4, AIL_DOUBLE Param5,
AIL_DOUBLE Param6)
```

Details

Parameters

ContextId

See ContextId of the main function for a description.

FeatureLabelOrIndex

See FeatureLabelOrIndex of the main function for a description.

ReferenceFrameLabelOrIndex

See ReferenceFrameLabelOrIndex of the main function for a description.

Geometry

See Geometry of the main function for a description.

Param1

See Param1 of the main function for a description.

Param2

See Param2 of the main function for a description.

Param3

See Param3 of the main function for a description.

Param4

See Param4 of the main function for a description.

Param5

See Param5 of the main function for a description.

Param6

See Param6 of the main function for a description.

```
void MmetSetRegionDouble (AIL_ID ContextId, AIL_INT FeatureLabelOrIndex, AIL_INT ReferenceFrameLabelOrIndex,
AIL_INT64 Geometry, AIL_DOUBLE Param1, AIL_DOUBLE Param2, AIL_DOUBLE Param3, AIL_DOUBLE Param4, AIL_DOUBLE Param5,
AIL_DOUBLE Param6)
```

Details

Parameters

ContextId

See ContextId of the main function for a description.

FeatureLabelOrIndex

See FeatureLabelOrIndex of the main function for a description.

ReferenceFrameLabelOrIndex

See ReferenceFrameLabelOrIndex of the main function for a description.

Geometry

See Geometry of the main function for a description.

Param1

See Param1 of the main function for a description.

Param2

See Param2 of the main function for a description.

Param3

See Param3 of the main function for a description.

Param4

See Param4 of the main function for a description.

Param5

See Param5 of the main function for a description.

Param6

See Param6 of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.

MmetStream

Synopsis

Load, restore, or save an Aurora Imaging Library metrology context from/to a file or memory stream.

Syntax

```
void MmetStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextMetIdPtr,       //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save an Aurora Imaging Library metrology context from/to a file or memory stream.

To inquire the number of bytes necessary to save a Model Finder context to memory stream, you should first call this function (**MmetStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. This function is equivalent to a file saved using MmetSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file. All information about the previously allocated metrology context is saved, including all of the feature settings and geometric tolerance settings. However, any associated camera calibration contexts and drawing control type settings are not saved.

All of the metrology context's settings and names that were in effect when the metrology context was saved will be restored. If you had associated a camera calibration context with the template reference in your metrology context and you did not save it with the context, you must re-associate the camera calibration context, using MmetControl() with M_ASSOCIATED_CALIBRATION. Also, if you had previously set drawing control types, using MgraControl() with M_DRAW_..., you must reset them since they were not saved with the context.

Using **MmetStream**(), you can choose to save a backwards-compatible version of an Aurora Imaging Library metrology context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore Aurora Imaging Library metrology context saved using Aurora Imaging Library version 8.0 or above. Settings that do not exist in the lower version will be filled with default values when the metrology context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the metrology context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the metrology context.

This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the metrology context. When performing an M_LOAD or M_RESTORE operation, this parameter must be set to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies whether to restore, from the file or memory stream, the camera calibration context associated with the template reference, or specifies to save, with the metrology context, the camera calibration context associated with the template reference.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to restore or save the camera calibration contexts																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the camera calibration contexts are not saved or restored.	c		e	f	g	a		i		j	k	l	m	n	b

M_WITH_CALIBRATION	Specifies that the camera calibration context is saved or restored, depending on the Operation parameter. When restored, the camera calibration context must have been previously saved with the context, using MmetSave() or **MmetStream**() with M_WITH_CALIBRATION. The camera calibration information is restored from/ saved to the same file or memory stream as the metrology context. The camera calibration cannot be managed independently from the metrology context. When the metrology context is freed, the camera calibration is automatically freed as well.	c		e	f	g	a		i		j	k	l	m	n	b

ContextMetIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the metrology context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the metrology context, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of the metrology context will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextMetIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																	
Operation		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	MemPtrOrFileName - Possible values passed or returned																
	SysId																
	Version																
	ContextMetIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE		Inquires the number of bytes required to save a metrology context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
	MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

Version	Specifies the Aurora Imaging Library version with which the metrology context must be compatible when saved. The size of a metrology context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextMetIdPtr	Specifies the address of the variable containing the identifier of the metrology context. The metrology context must have been previously allocated on the required system using MmetAlloc() with M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Metrology contexts typically have an MET file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextMetIdPtr		Specifies the address of the variable containing the identifier of the metrology context. The identifier returned is similar to the one returned if allocated using MmetAlloc() with M_CONTEXT. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE			Specifies the address of the variable in which to write the identifier of the metrology context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Metrology contexts typically have an MET file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b

SysId	Specifies the system on which to allocate the metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextMetIdPtr	Specifies the address of the variable in which to write the identifier of the metrology context. The identifier returned is similar to the one returned if allocated using MmetAlloc() with M_CONTEXT. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves a metrology context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Aurora Imaging Library software products, when saving a metrology context to a file, use the MET file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

Version	Specifies the Aurora Imaging Library version with which the metrology context must be compatible when saved. Note that the metrology context can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextMetIdPtr	Specifies the address of the variable containing the identifier of the metrology context to save. The metrology context must have been previously allocated on the required system using MmetAlloc() with M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailmetrol11.lib.
DLL	Requires ail11.dll; ailmetrol11.dll.